

Computer Science NEA

3D Gravity Simulator

Max Baker

Contents

Analysis	7
Problem Identification	7
Why is the problem solvable by computational means?	8
Features that make it amenable to a computational approach	8
Object Oriented Programming	8
Abstraction	9
Problem Decomposition	9
Divide and Conquer	10
Heuristics	10
Stakeholders	10
Stakeholder Identification and Justification	10
Student Questions	11
Teacher Interview:	14
Research Of Other Solutions	16
1: PhET Interactive Simulations gravity/orbit simulator	16
2: NSTMF Lab Gravity Experiment	20
3: Gravity Simulator	24
4: PhET - Kepler's Laws	28
The Proposed Solution	31
Overview	31
Features	31
Limitations of Proposed Solution	34
Requirements	35
Hardware and Software Requirements	35
Success Criteria	36
Design	41
Top-Down Design	41
Subproblem 1: Rendering	41
Subproblem 2: Educational Value	45
Subproblem 3: Physics Engine	46
Subproblem 4: Simulation Controls	47
Subproblem 5: Web Based	50
Structure of the Solution	51
Justification of my Choice of Coding Language	51
Rendering	51
Educational Value	52
Physics Engine	53
Simulation Control	53
Web Based	54
Variables	55
Pseudocode Classes	55
Planets Class	56

Label	59
Space-time grid	61
Vertices	63
Trail Path	64
Trail Path Points	66
Design of the Main Loop Code	66
User Interface	67
Usability Features	69
Stakeholder Feedback	70
Validation	71
Testing to Inform Iterative Development	72
Validation Testing	72
Stage 1: Creating a Working Window	73
Stage 2: The Planet Class	74
Stage 3: Prototype 1	79
Stage 4: Going 3D	82
Stage 5: Adding Functionality	83
Stage 6: Usability Features	87
Post Development Tests	88
Tests PD1-25	88
Test of Gravitational Accuracy - Test PDA1	93
Development	94
Stage 1: Creating a Working Window	94
Stage 2: The Planet Class	98
Planet Class Version 1	98
Plant Class Version 2	106
Stage 3: Prototype 1	110
Prototype 2: Circle	111
Prototype 1 Version 1 - Using Planet Class	112
Prototype 1 Version 2 - Movement	113
Prototype 1 Version 3 - Multiple Circles	120
Prototype 1 Version 4 - Planets List	122
Prototype 1 Version 5 - Gravity	125
Prototype 1 Version 6 - Front End	128
Stage 4: Going 3D	147
Prototype 3: Sphere	147
Main Program	157
Stage 5: Adding Functionality	163
Trail Paths	163
The Space-Time Grid	170
Stage 6: Usability Features	189
Evaluation	202
Evaluating Success Criteria	202
Unmet Success Criteria and How they Could be Addressed	207

SC15	207
SC25	209
Usability Feature Evaluation	210
Aesthetic Features	212
How I could address unmet usability features?	213
Educational Messages - SC12	213
Limitations of the Solution	214
Maintenance Issues	215

Analysis

Problem Identification

While many students may think they understand how gravity works and what effect it has on matter, this is often full of misconceptions. For example, most students have the idea that gravity is a force between two large objects, when in reality gravity is curvature in space-time that causes the attraction between objects.

Gravity being one of the most important concepts in physics, it should be essential that students have an accurate understanding of what it is and how it acts on a macro scale. However, it is impossible to show students the gravitational interaction between objects in real life as we inhabit an area of space-time in which we are constantly acted upon by the gravitational 'force' of earth.

In this project I hope to make a simulator of gravity in 3D which utilises a grid to represent space-time, with the target of showing students how curvature in space-time is really how gravity works. I want to make the simulator interactive, allowing students to see how the space-time grid is affected when they make certain changes to it.

Why is the problem solvable by computational means?

1. Number of calculations- in gravitational simulations with two or more bodies, many calculations are required a second to plot the velocity of the bodies. If I want to create and run a simulation in which several objects are all exerting a force on each other, there will be a large number of interaction forces, velocities and positions to take into consideration. In addition to this, any visualisation of space-time will require calculations of all vertices in a grid in a single time-step. Therefore, computation is suitable as it can perform these calculations efficiently and quickly, allowing for a functional simulator that can simulate events in a suitable time-frame.
2. Visualisation and experimentation. Using computation is suitable for this problem because it is essentially impossible to run a gravity related experiment in real life. In almost all other areas of physics, experimentation can be carried out and give accurate results. Because the gravitational interaction between planets happens on such a huge scale, and accurate models cannot be created in real life, a computational model provides the most accurate and suitable way of visualising and running experiments on the gravitational interaction between two large bodies.

Features that make it amenable to a computational approach

Object Oriented Programming

Object Oriented Programming (OOP) is a programming paradigm that structures software around data, or objects, rather than just function and logic. In my simulator I can use OOP to create a class for planets I add into the simulator.

- Encapsulation: Within the planet class, I can have attributes such as mass or colour which will dictate the properties of the object. I can make the attributes within my class private so they can only be edited by the class's methods. This makes code a lot more maintainable as it stops variables from being changed by accident.
- Instantiation: I can use this class to instantiate planets when I need to, on command of the user, who will be able to edit these features. I can use a constructor method within each class to allow the user to make many instances of the class as they wish from their input.
- Composition: In addition to this, if I want to link instances of the planet with other functionalities such as property labels or orbit trails, I can do this using composition within the planet class, linking it to the other classes I create.
- Polymorphism: If I want to create several different classes with methods of the same name, I can do this using polymorphism. For example, if I want a planet to have a draw function and an orbit path to have a draw function, these can have the same name but contain different code for each class.

Abstraction

Abstraction is the removal of unnecessary components of a problem to allow focus on only those that are necessary. The gravity simulator lends itself to abstraction because in order to make a simulator that runs without instability and crashing. To enhance the performance of the simulator I will remove details to reduce the memory use of the program. I hope this will make the function calls faster. Some ways in which I might use abstraction include removing details from planets; I will likely model all the planets as spheres for simplicity of coding and reduce the memory the program uses.

Problem Decomposition

Problem decomposition is breaking down a big problem into smaller problems so that they can be solved independently. In the case of the gravity simulator, the overall system can be broken down into separate problems. This is a rough outline of the modules that will be necessary in the gravity simulator.

1. Object representation - using Object Oriented Programming to create classes of planets, stars, potentially black holes and allowing separate instances of each class to be created with separate masses, and any other properties that may be necessary to change.

2. Physics engine - a module for performing calculations to handle the gravitational forces. This module will update the velocity and position of the planets and apply acceleration to them.
3. Rendering/visualization - a large part of my project will involve creating the graphics and rendering that will make the simulation effective and educational. This will likely be done after the initial code has been written.
4. User input - a module to allow the user to control and configure the planets, gravity constant, time step, etc is essential for an interesting simulation.

Divide and Conquer

Divide and conquer is a strategy to make a complex task easier by breaking it into smaller, more manageable tasks. After I split the problem into modules, I will try to solve each module independently and follow this by combining the modules together to create a solution to the larger problem. The benefits of this is that solving each subproblem/module separately should lead to more efficient modules which come together to form an overall more efficient solution.

Heuristics

Heuristics are used to find a solution that is 'good enough' rather than perfect. I can use heuristics to approximate calculations of the interactions between planets in the simulation, as well as using heuristic rules for collisions, allowing me to approximate bounce angles and velocities without full rigid-body physics calculations. The benefits of using heuristics is that it should make the program run faster and more efficiently.

Stakeholders

Stakeholder Identification and Justification

Students

As my project is designed for the use of students, I've chosen A level 5 physics students to be stakeholders for me. They are suitable stakeholders because they will have an understanding of what A level physics requires, including the topic of gravity. This means they will be able to assess whether or not my product will be a helpful educational tool. In addition to this, these students will know in what areas they struggle and I can use this to identify possible solutions to my problem. I can use their feedback to create a simulator that helps them in their areas of struggle.

I have decided to use 5 students because I want an accurate representation of all physics students. My stakeholders will have different abilities and weaknesses when it comes to

physics and using 5 allows me to get a good sense of physics students in general, rather than a single physics student who may not be representative of students on the whole.

For my project I have asked these 5 students a series of questions about their understanding of gravity and how they have used gravity simulators in the past.

Teacher

Dr Mahoney - a physics teacher who teaches both A level and GCSE physics. He is an appropriate stakeholder because he will have an understanding of where students struggle when it comes to gravity from his teaching experience. He will be able to give me advice and feedback on how the simulator should function and what features should be included.

Dr Mahoney could make use of the product by showing it to classes, using it to demonstrate how gravity works to students. This adds to his suitability as a stakeholder because he can inform me of features he wants to have on the simulator.

Student Questions

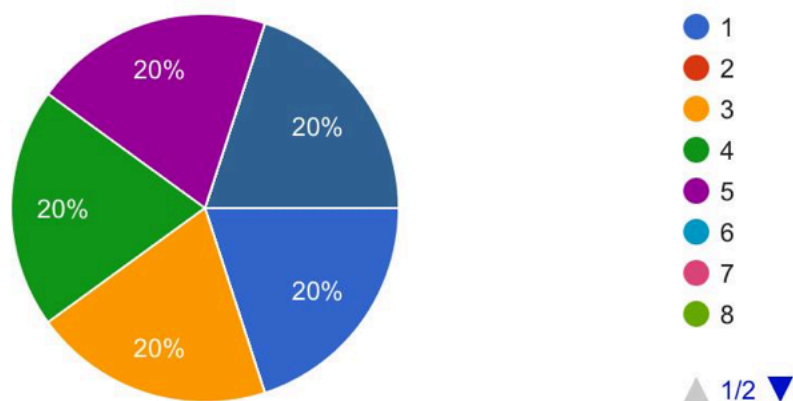
The first part of getting input from stakeholders was sending out a google form to collect data to give a better picture of how I could make the simulator to be helpful for students.

Question 1

On a scale of 1 to 10 how well do you understand how gravity acts between planets?

1 being no understanding and 10 being full understanding.

5 responses

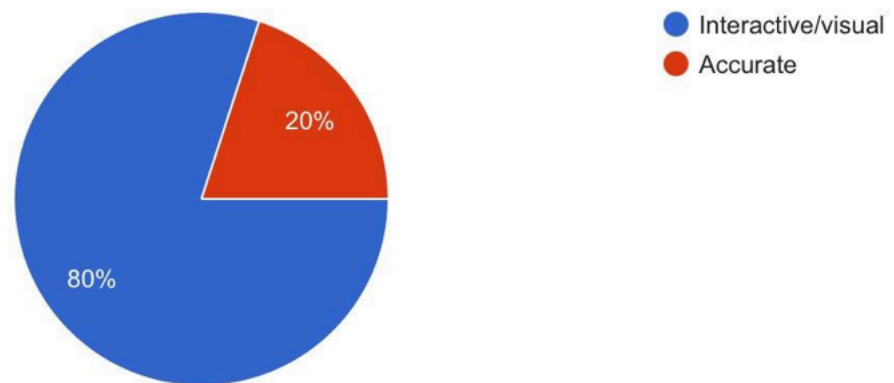


In question 1 I wanted to get a rough idea of how well students understood gravity. All the students I asked were A level physics students, however, I feel gravity is a weak area of many students' knowledge. The 5 students gave 5 different answers: 10, 5, 4, 3, 1. From this I can conclude that many students have a less than satisfactory knowledge of gravity. Despite one student thinking they have a complete understanding of gravity the mean was still 4.6/10. This result tells me that a more educational simulator which talks through what is happening may be more suitable to enable students to gain understanding of how gravity works.

Question 2

Would you prefer a more accurate simulation or a more interactive/visual one?

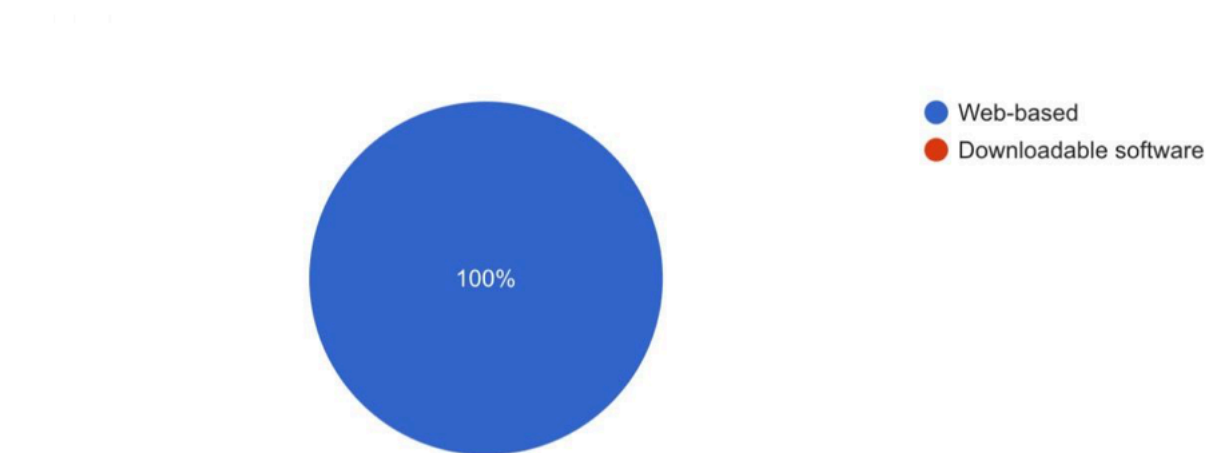
Figure 2.1



In this next question I wanted to get an idea of whether users would prefer the simulator to be accurate, allowing them to make measurements from it, or to be more visual, showing and describing the way in which gravity works. The result is quite conclusive, with 80% of stakeholders agreeing that a more interactive and visual approach would be more suitable.

Question 3

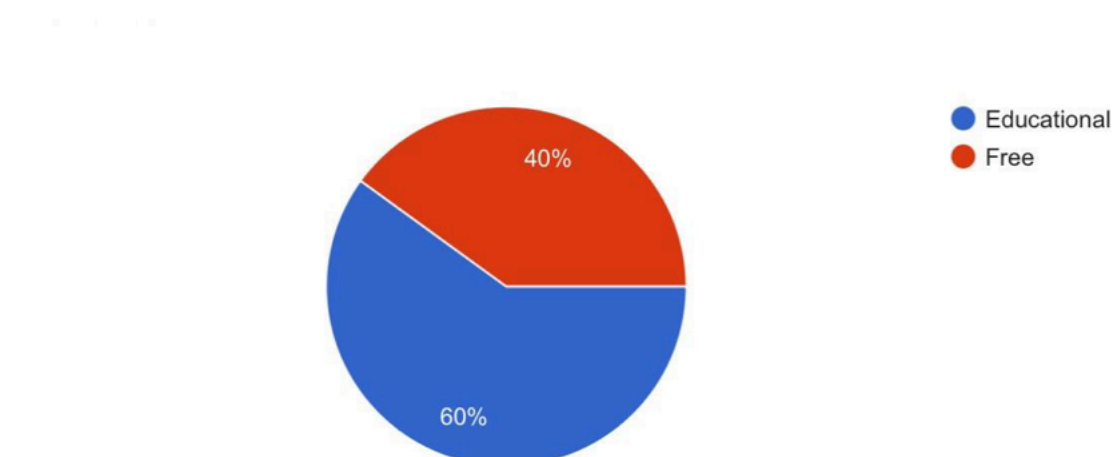
Would you prefer a web-based simulator that you can access via the internet or downloadable software (e.g app)?



Question 3 shows that all my users would prefer a web-based solution rather than one with downloadable software. I will therefore add web-based to the success criteria of the project, to ensure the solution is practical and usable for my stakeholders.

Question 4

Would you prefer a simulator that is more education and talks the user through what is happening the simulation or a simulator that is more free and allows the user to create whatever events they wish?

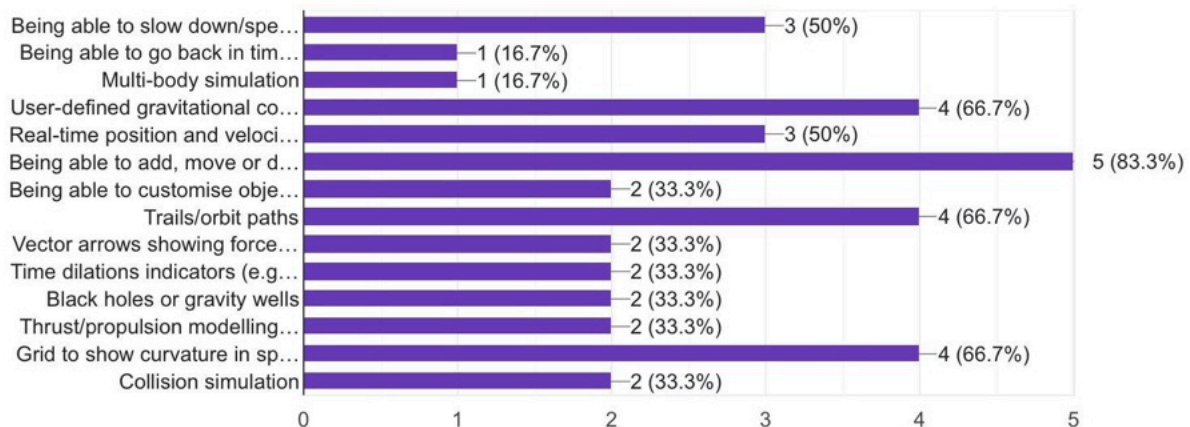


With question 4 I aimed to deduce whether my stakeholders would prefer a simulator that is free from instructions and allows the user to do whatever they wish or a simulator that provides educational value and talks the user through what is happening. As I only surveyed

5 people this result is quite split, potentially indicating that the solution between the options is the most suitable.

Question 5

Out of the list below which features do you think are important for the gravity simulator?



This question aimed to outline key features that my simulator could include. With a wide range of possible options, I wanted to highlight the key features my stakeholders want so I can focus on delivering these. I can also remove possible features that were less popular amongst my shareholders. The 4 most popular features were: User-defined gravitational constant; being able to add, move or delete objects (planets); trails/orbit paths and a grid to show curvature in space time.

Teacher Interview:

The next step in outlining how I can make the simulator suitable for my stakeholders was interviewing my teacher stakeholder.

I first asked him to select features they thought were important out of the list of features I submitted to the students, these are the features he picked out:

- Being able to slow down/speed up time
- Multi-body simulation
- User-defined gravitational constant
- Real-time position and velocity updates
- Being able to add, move or delete objects (planets)
- Trails/orbit paths
- Grid to show curvature in spacetime

Question	Response	My thoughts
1. How do you currently aid students in understanding when it comes to teaching gravity?	Review what already known: uniform fields, gravity weaker further, often have seen inverse square rule. Teach in steps Kepler's laws and relation between gravitational strength and distance. Definitions before Linking force to gradient of U against r (then link radial force to inverse square law), show it retrieves $g = 9.81 \text{ N/kg}$ on some parts of Earth. Link to circular motion and Kepler's laws. Apply to SHM problems for stretch and challenge Lots of calculations / modelling answers / practice and constant assessment for learning.	1. I need to simulate the relationship between gravitational strength and distance using Newton's law of universal gravitation. 2. I can show the link between gravity and circular motion by creating a system that allows users to set up circular orbits. 3. It might be harder to bring Kepler's laws into the simulator, but could talk about this in educational messages displayed to the user.
2. Do you use any online resources for this?	There are some simulators online, but it depends what you want to achieve. Some things are well simulated (Eg Kepler's laws), while things like space-time are not well simulated.	1. Avoid simulating Kepler's laws as they are already well simulated. 2. Focus on space-time simulation.
3. What do you need to understand to do well in the gravity topic in physics?	It's not an easy topic and you cannot coast. You need to have an appreciation of mathematics and an ability to distinguish between grav potential and forces, students often mix when to use one over another. Need to be clear on how bodies interact (space-time rather than a mysterious interaction) Clear on definitions and able to apply problem solving skills / link to other parts of physics.	1. I will need to carefully consider the maths associated with gravity in order to simulate it successfully. 2. In order to help students with definitions I could display educational definitions to them while the simulation is running, which might make it more engaging.

After asking him what his thoughts were about the project and if they had any comments and suggestions, Dr Mahoney said:

'I think if it were me; I'd start with narrowing the scope to Newtonian mechanics. If you succeed there reproducing Kepler's laws of motion, then you could try incorporating Lagrangian mechanics to try modelling a solar system. Lagrangian mechanics would be an interesting spin as you could identify Lagrange points, regions of space where objects experience no force, despite not being intuitive positions. If you master that, then you could

think about collisions / effects of GR. I'd imagine incorporating GR would be equivalent to an ambitious 4th year university level project; I can't pretend I fully understand GR myself.

An alternative route could be to go SR to begin with and try modelling a rotating black hole with material feeding in by an accretion disk or see if you could mimic the jet-like emissions from SMBH - a relativistic effect that results from length contraction due to BH rotation. It would be a very different simulation to the first suggestion.'

Conclusion

From 3 initial questions, I have concluded that it is best to focus on simulating space-time to make this an effective educational tool. Dr Mahoney mentioned how Kepler's laws are already well simulated with online simulations so focusing my simulator on these may not bring anything new as this is a problem already solved.

I will follow Dr Mahoney's advice to start with and use Newtonian mechanics. However, I won't reproduce Kepler's laws of motion but instead focus on producing an accurate simulation of general relativity (GR). As he said that incorporating general relativity in my calculations may be ambitious, I won't attempt this but rather focus on making an accurate simulator using Newton's universal law of gravitation.

Key

- Newtonian mechanics in the context of gravity - describes gravity as a force by stating that every particle attracts every other particle in the universe with a force that is proportional to the product of their masses and inversely proportional to the square of the distance between their centres of mass.
- Kepler's law of motion - three laws of motion that dictate how planets orbit the sun. The three laws are:
 1. The orbit of a planet is an ellipse, with the Sun at one of the two foci.
 2. A line segment joining the Sun to a planet sweeps out equal areas in equal time intervals.
 3. The square of the orbital time Period T is directly proportional to the cube of the orbital radius r .
- Lagrangian mechanics - Lagrangian mechanics is a formulation of classical mechanics that describes the motion of a system using a scalar function called the Lagrangian. It's based on the principle of stationary action, which states that a system will follow the path that minimizes the "action," a quantity calculated from the system's kinetic and potential energies.
- GR - general relativity, Einstein's understanding of how gravity affects the fabric of space-time.
- SR - special relativity, scientific relationship between space and time.

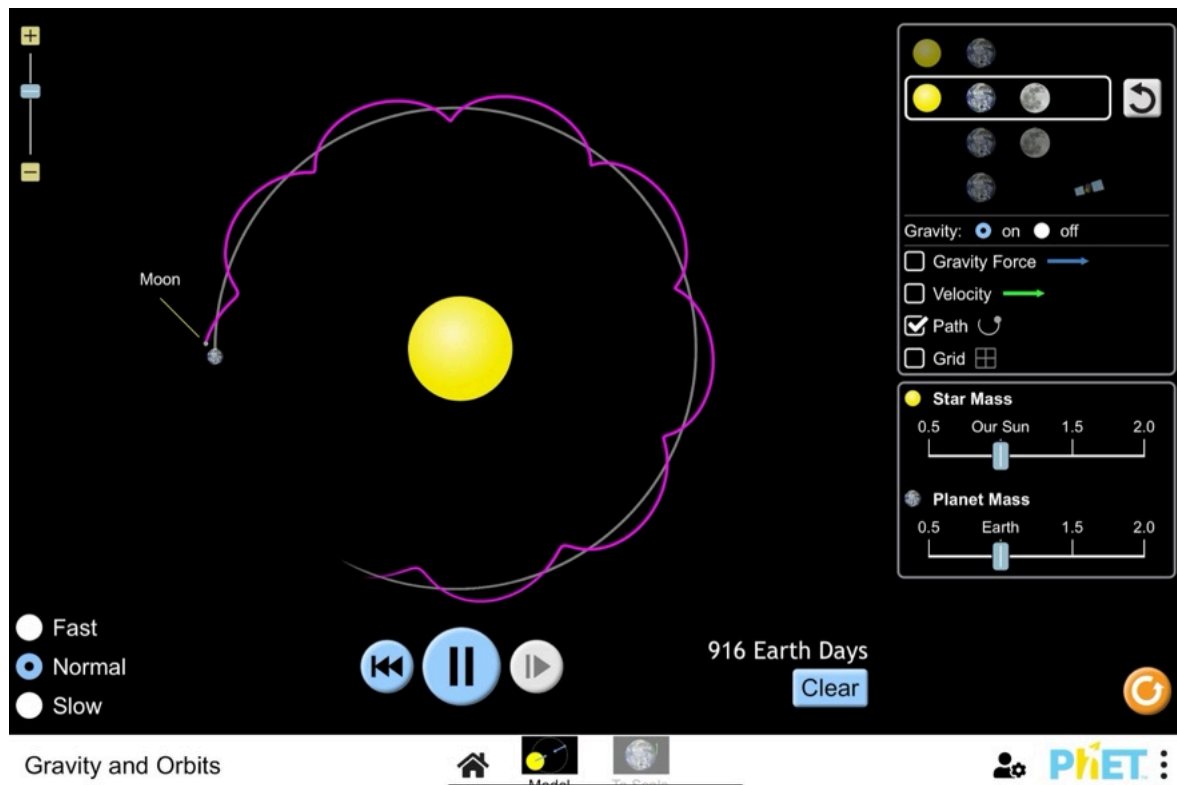
- SMBH - supermassive black hole
- BH - black hole

Research Of Other Solutions

1: PhET Interactive Simulations gravity/orbit simulator

https://phet.colorado.edu/sims/html/gravity-and-orbits/latest/gravity-and-orbits_all.html

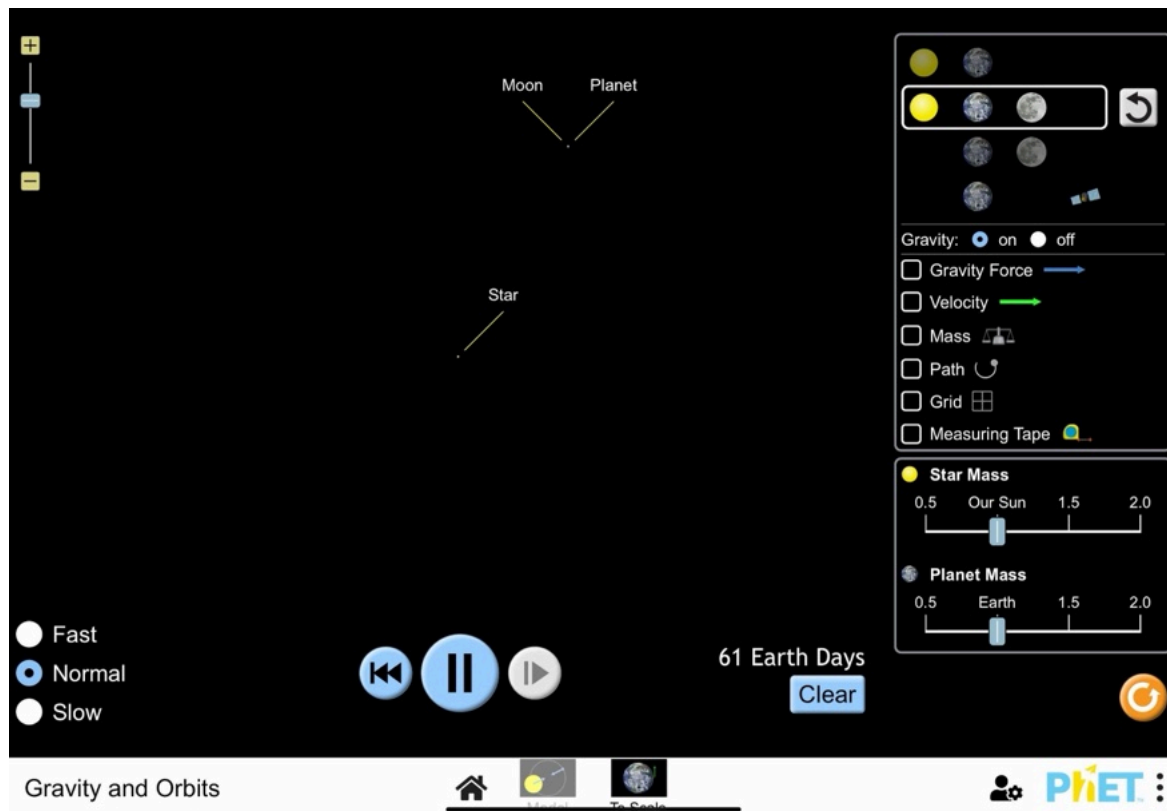
This fairly basic simulator from PhET Interactive Simulations has two main 'modes'; a model and a to scale version. After selecting one, you are presented with a setup similar to the screen below. It allows you to pick between 4 different simulations, as seen on the upper right hand corner of the screen. Each simulation contains different planets and masses and in each you can move, adjust the masses of and change the velocity of some of the masses.



The picture above shows a model system, while the one below shows the to scale version.

Something I find interesting about this simulation is that you can turn gravity on and off as you please. Although this may not be realistic, it is interesting to see how the system is affected by zero gravity. While I find this feature interesting it is not something I think is very helpful educationally and therefore I won't consider including this in my simulator.

The simulator allows you to move the planets while the simulator is going. This allows the user to freely move planets in the plane around and see what effect this has on the bodies.



Features I like	Justification
Simplicity of the layout	It is easy to navigate for the user and it is clear what each toggle and option does. This makes it a very usable simulator. The sliders make the editing of the masses of each planet very easy and the way the sliders are labelled with the relative masses of each object makes it very clear to the user how massive each object is without them having to know the mass of each object.
Ability to change the mass of each planet during the simulation	This allows the user to see directly how larger masses increase the force of gravity and smaller masses reduce the force of gravity. This makes it a helpful educational tool in this regard.
The ability to switch between a model system and a to scale system	The model with larger planets is suitable for giving the user a visual representation of the planetary system and allows the user to come to grips with the workings of gravity,

	while the to scale model gives the user a more realistic view of the scale that gravity acts on.
--	--

Features I dislike	Justification
Lack of mass customisation	You can only change the mass of the sun and the planet from 0.5 times to 2 times the mass of our solar system's relative equivalents. The lack of possible masses is frustrating because it is unrealistic; intermediate mass stars can be as large as 8 times the mass of our sun, while massive stars can range from 8 times to above 50 times the mass of our sun. This really limits the possibilities of what you can do with the simulator and it makes it centred entirely on portraying our solar system.
Inability to add other planets to the simulation	Not being able to add other planets means that the user cannot see how many bodies all interact together. This limits the user's ability to explore gravity as they can only explore so many scenarios.
Lack of educational value	This simulator is aimed at students, but lacks any real description of what is happening in the simulator or any description of how gravity works. This limits its usability for students who don't know what is happening and doesn't necessarily help students understand how gravity works.

Conclusion

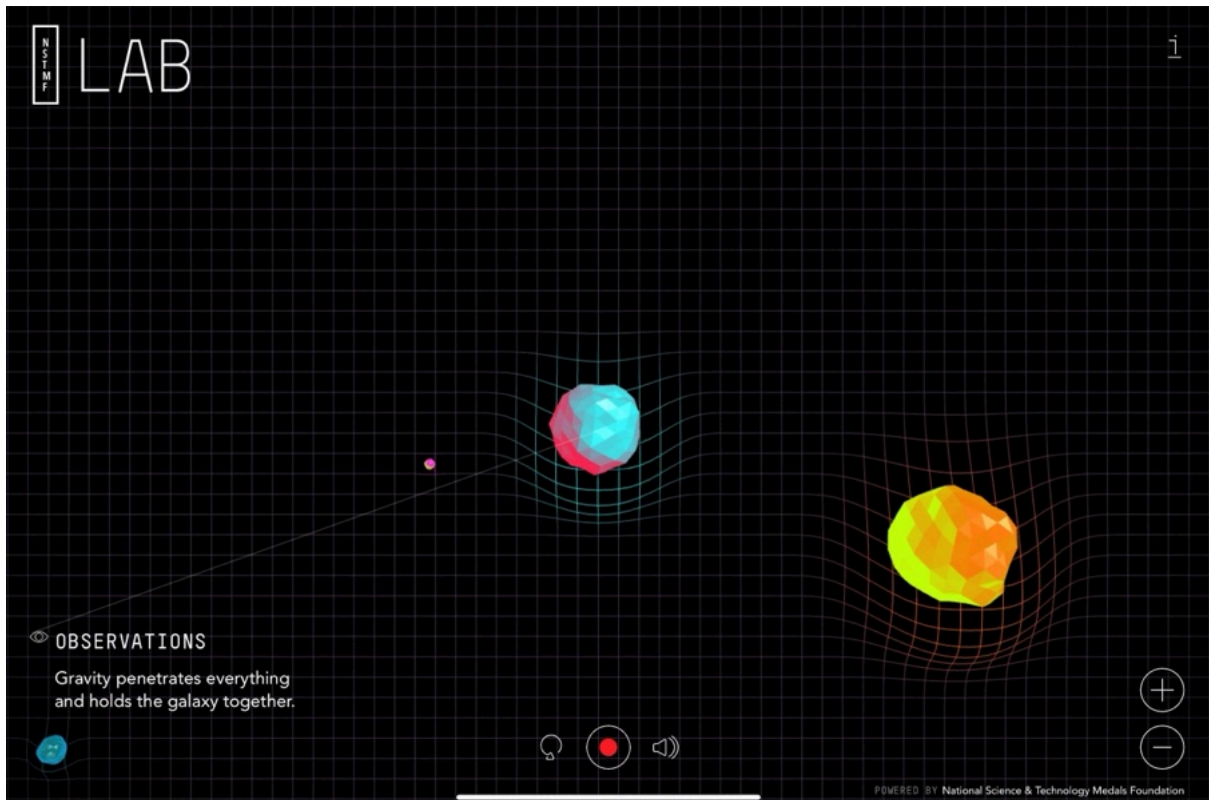
My user interface should be simple for the user. The simplicity made this simulation very approachable for new users. There is no reason for my user interface to be complex because it is aimed at students and doesn't need loads of unnecessary customisation. However, I think that the situations that the user can set up using this simulator are very limited, so I'd like to give my user a lot more free reign to set up systems as they wish. In addition to this, I think some sort of educational value in this simulation would have helped the user to learn more from it. Therefore, I think including some sort of educational value into my simulator would be beneficial.

2: NSTMF Lab Gravity Experiment

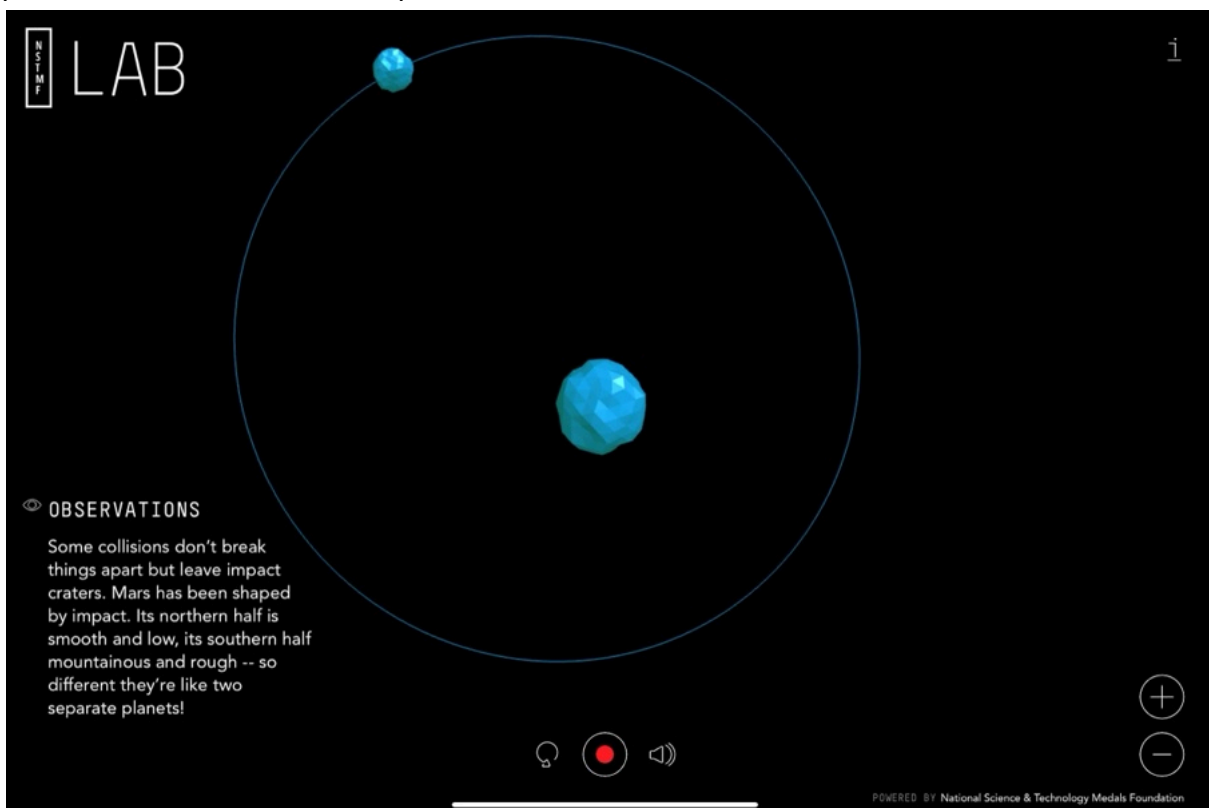
<https://lab.nationalmedals.org/gravity.php>



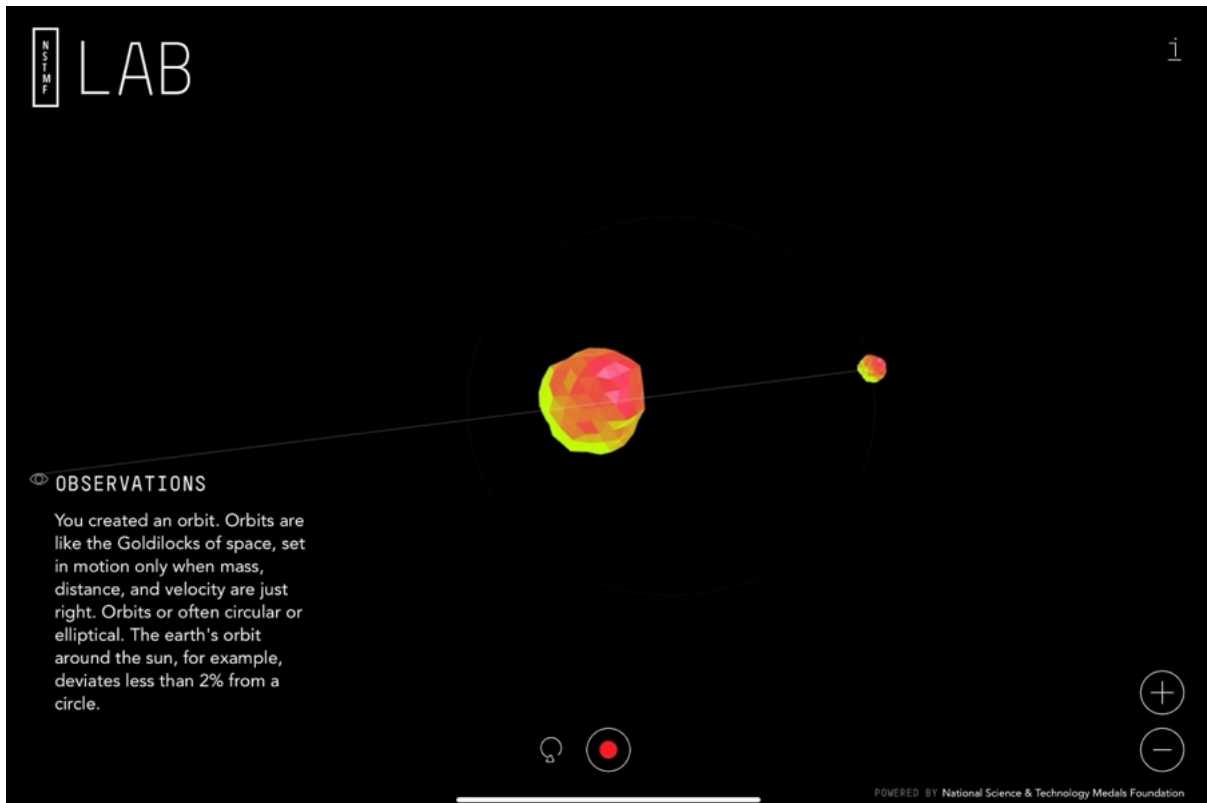
When you first open the gravity simulator you are presented with this screen. The simulator allows you to press on the screen to add planets. Holding for a longer period of time creates more massive planets.



After adding planets, the grid that represents space-time appears, showing the curvature the planets cause in the fabric of space-time.

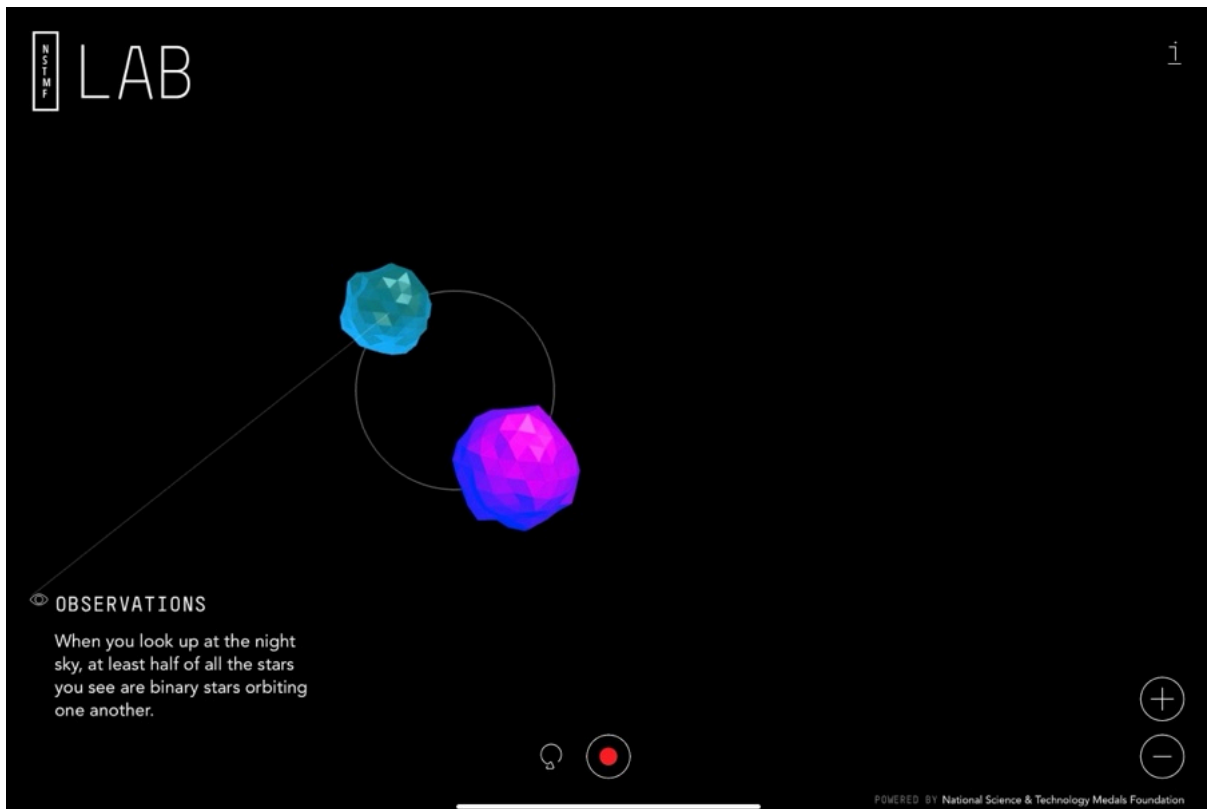


While the simulator is simple it also allows you to create orbits, which can be done by releasing planets with a 'flick'.



When an orbit is created in the example above, the observation section effectively provides information about what orbits are, in the context of space.

Binary system example:



Features I like	Justification
Minimal design	Being minimal allows the simulation to be focused entirely on the gravity in this simulator. It is very clear for the user and not intimidating at all. This gives it more of a game-like feel which makes it more engaging for users.
Space-time grid	I think this is a good way of showing how the planets interact with space-time and why it causes the objects to attract each other. The user can see that massive objects warp the space-time around it.
Educational prompts	Something I think this simulator does flawlessly is the observations in the bottom left hand corner. For users who aren't interested in the educational side of it, these observations can be ignored. However, for users who want to learn more about gravity, these observations provide interesting and helpful information about what is currently happening in the simulator. The way that they change depending on the orbits you set up and point at planets in the simulation is such a great feature because it makes the simulator such a good educational tool.

Features I dislike	Justification
Lack of accuracy when it comes to customisation	Lack of accuracy limits the user's ability to create meaningful and educational situations. Take velocity for example. The only way to add a planet with a given velocity is to 'flick' with your mouse. In the simulator, it is very easy to add more or less velocity than you intend. I think this can be frustrating for users who want to learn from the simulation and set up meaningful scenarios.
Lack of accuracy in the gravitational interactions	The scale of the 'universe' that you create is not very accurate.

	Planets in a binary system orbit roughly between 10 to 100 times the diameter of the planets. In the example above, the planets barely orbit 1 diameter of the planets. In this respect, I think this simulator lacks because the space between objects in orbit tends to be unrealistic. While this may be suitable as a visualisation tool, this lack of accuracy may be misleading to students when they go on to find out more about the nature of gravity.
--	---

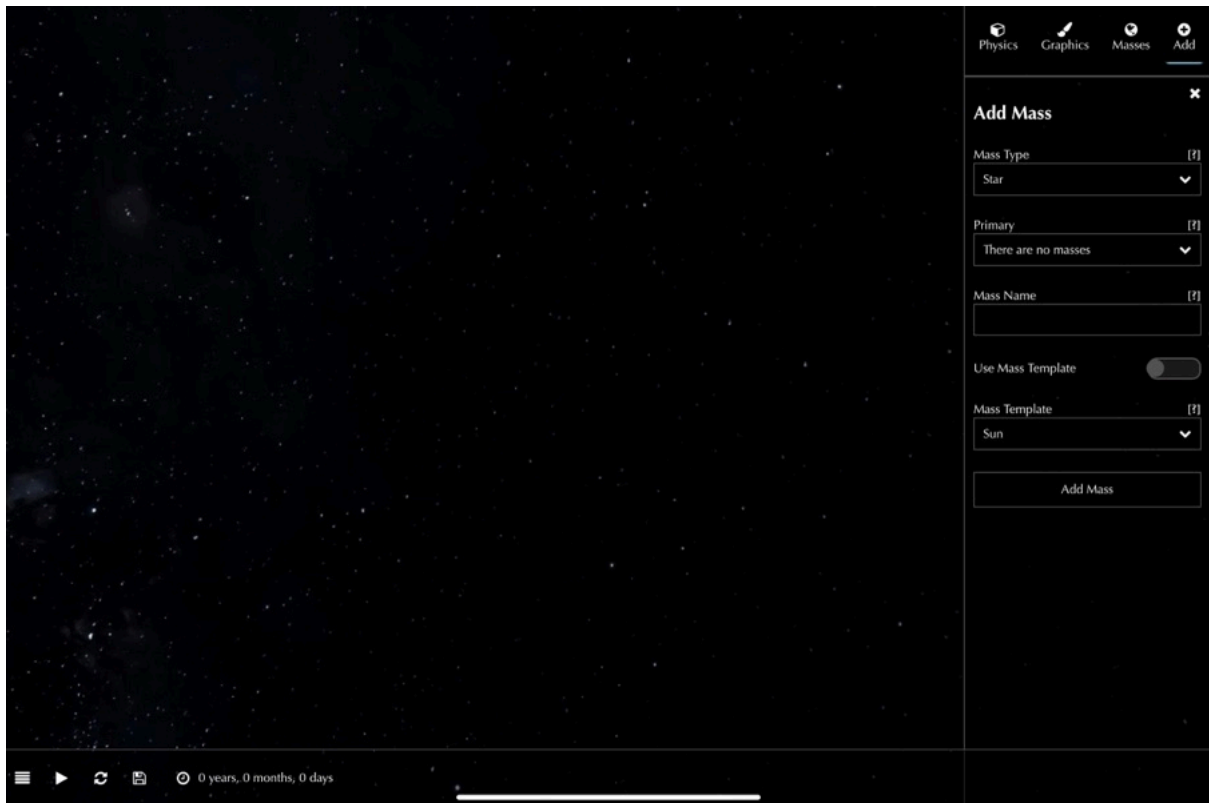
Conclusion

All of the educational parts of this simulator were executed very well. To start, I think the educational prompts displayed are something I'd like to capture in my simulator. The space-time grid is also a great feature, allowing the user to see how masses affect the fabric of space-time. This is a feature I would like to add to my simulation. That being said, after using this simulator, I think that sliders are the best way to customise the objects I add to my simulation as I thought the 'hold' and 'flick' was inaccurate and frustrating for the user.

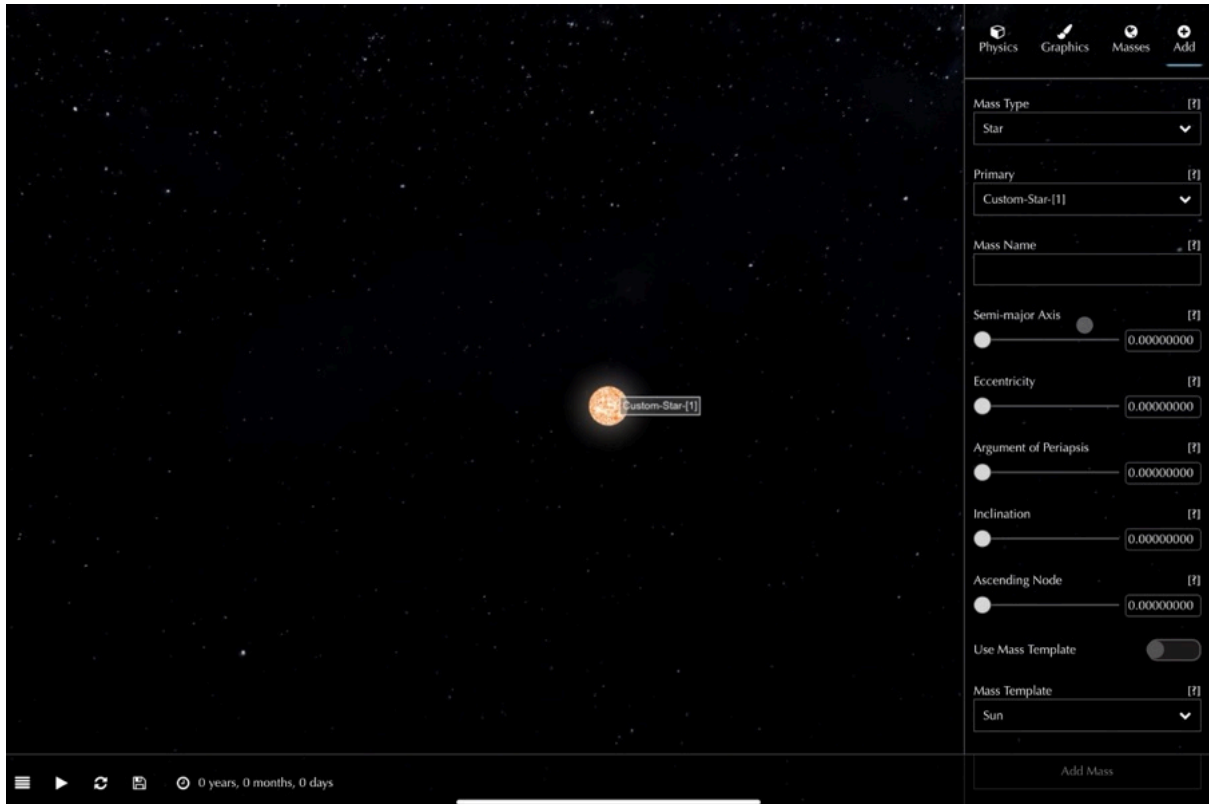
3: Gravity Simulator

<https://gravitysimulator.org/misc/create-new-gravity-simulation>

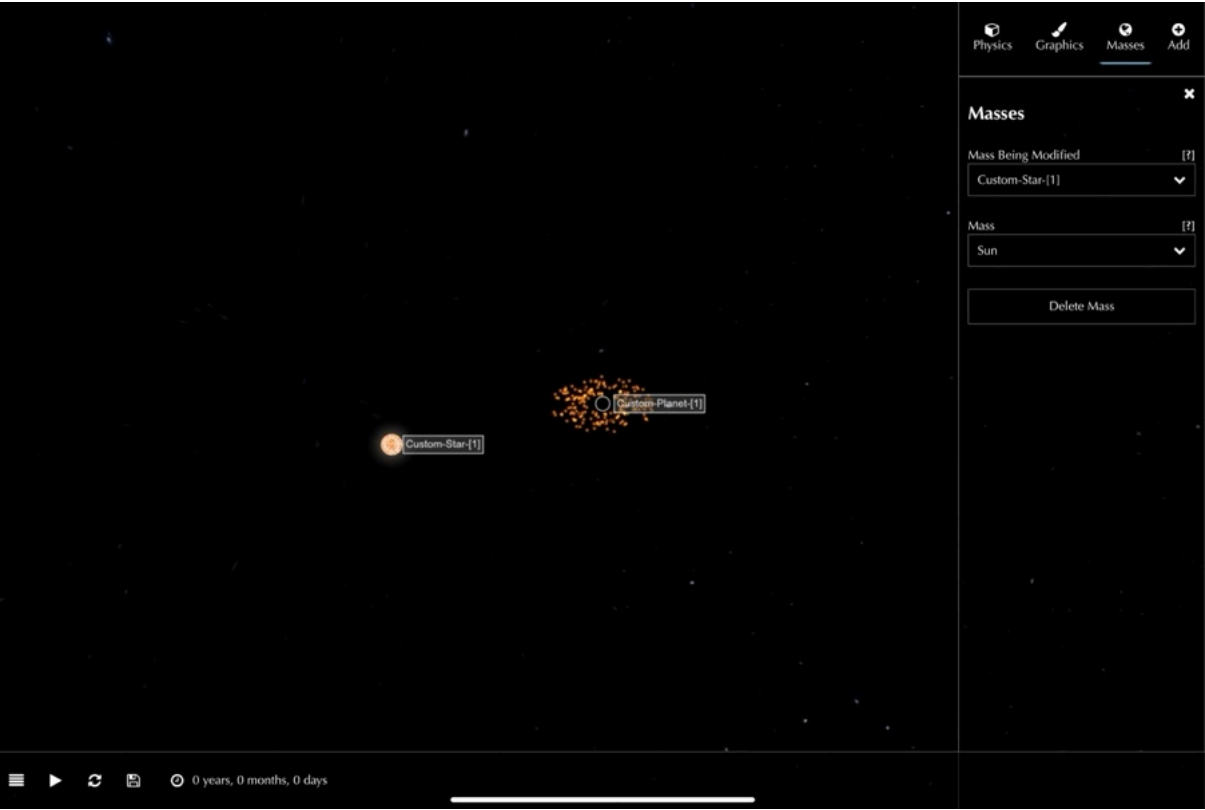
This gravity simulator titled: 'Gravity Simulator' contains many different scenarios all based around gravity on the home page. Instead of focusing on any single one of these, I'm going to base my research on the 'create new simulation' section instead.



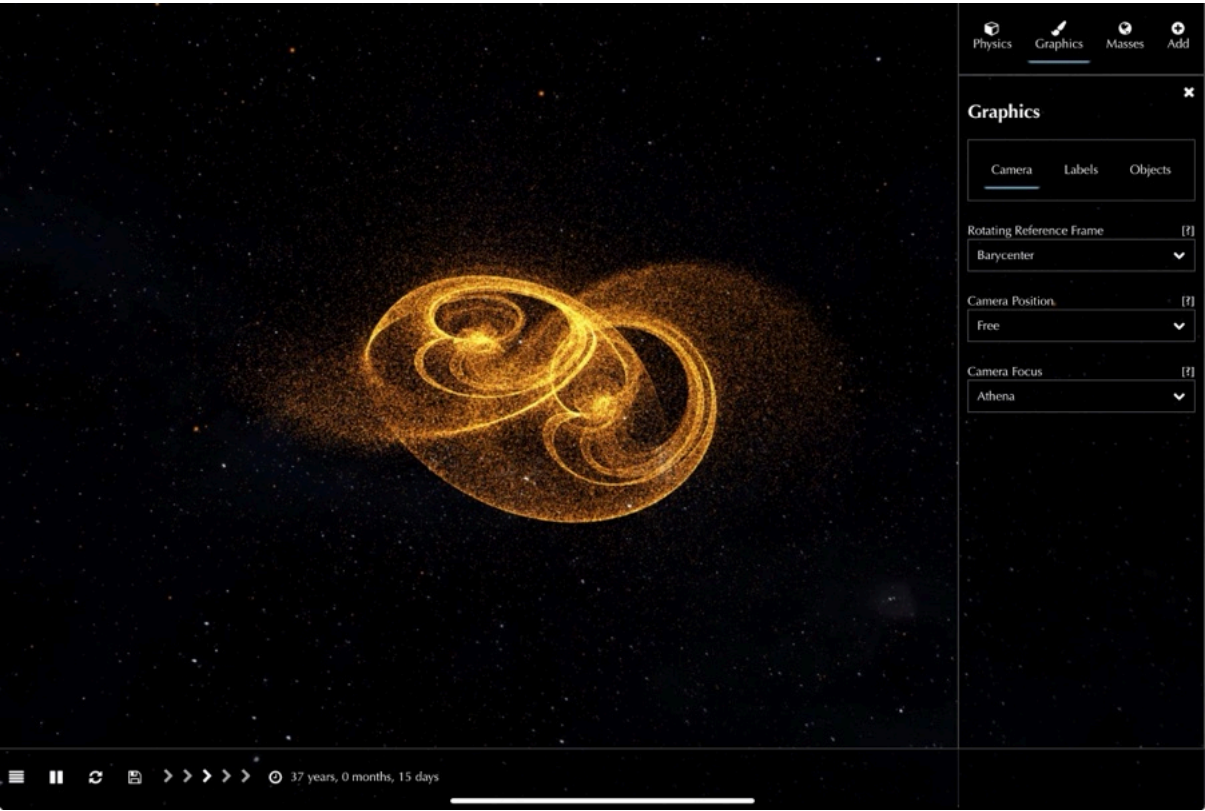
When you first open the 'create new simulation' section, you are presented with the above screen. This allows you to add new masses, with the following choices: star, ring, moon, planet and dwarf planet.



The picture below shows a simple system I was able to create containing a star and an orbiting planet with a ring surrounding it.



Galaxies colliding:



Features I like	Justification
You can add several different types of masses	Our universe is not just made up of spherical objects, so being able to add different types of objects feels realistic. Allows users to create really interesting systems. For example, one of the pre-made simulations uses the ring feature to create a simulation of two galaxies colliding.
The sliders	I like how the sliders allow a wide range of customisation but nothing which is unrealistic. They also display the value which you choose, allowing you to pick values accurately.

Features I dislike	Justification
There is too much customization	This is not suitable for users whose understanding of gravity is less advanced. I think this level of customisation is unnecessary for my simulator as my end users have a relatively limited understanding of gravity and orbits. The sheer amount of customisation makes the simulator intimidating for new users which could put them off.
Complicated user interface	Adding planets was complicated and adding planets at a chosen distance from a central point even more so. This wasn't helped by the fact that there were no instructions to help the user create what they desired. I also felt that the 'camera' was often in the wrong position when in the 'create new simulation' section of the simulator. This was also difficult to adjust, being new to the software. The graphics section of the menu can be seen above. It was frustrating that none of the options were clearly defined.

Conclusion

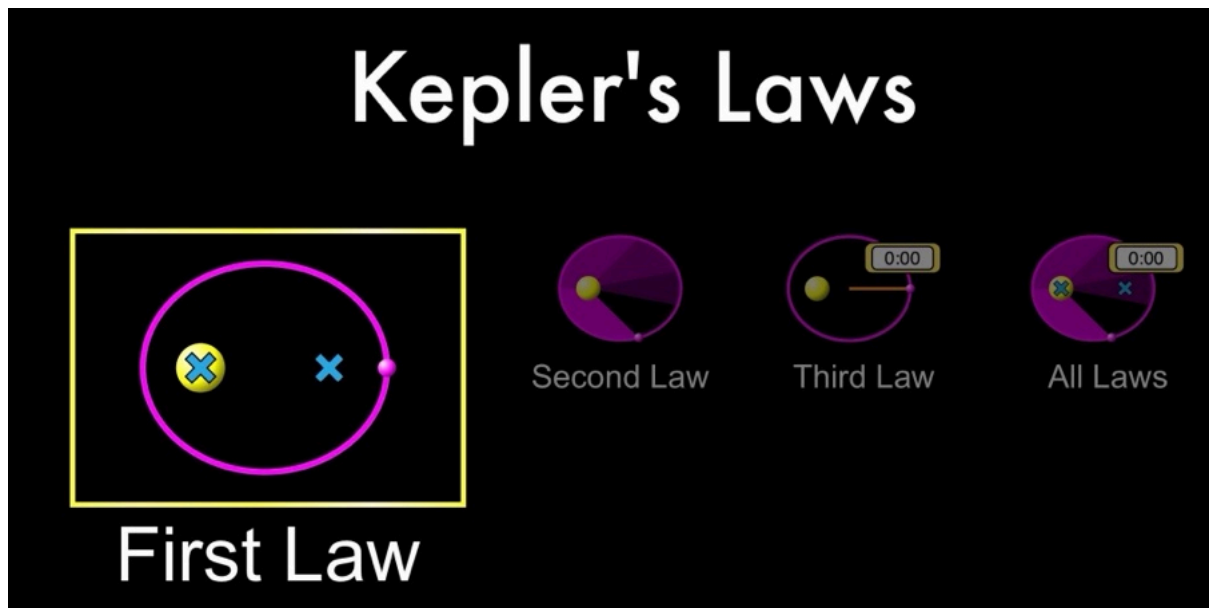
Implementing sliders similar to the ones used here is a good idea. They are a good medium between suitable customisation and usability. This simulator shows that I should not add too many customisable features or it will be intimidating for the user. In addition to this it would be complicated and impact the ability of users to learn from the simulation.

4: PhET - Kepler's Laws

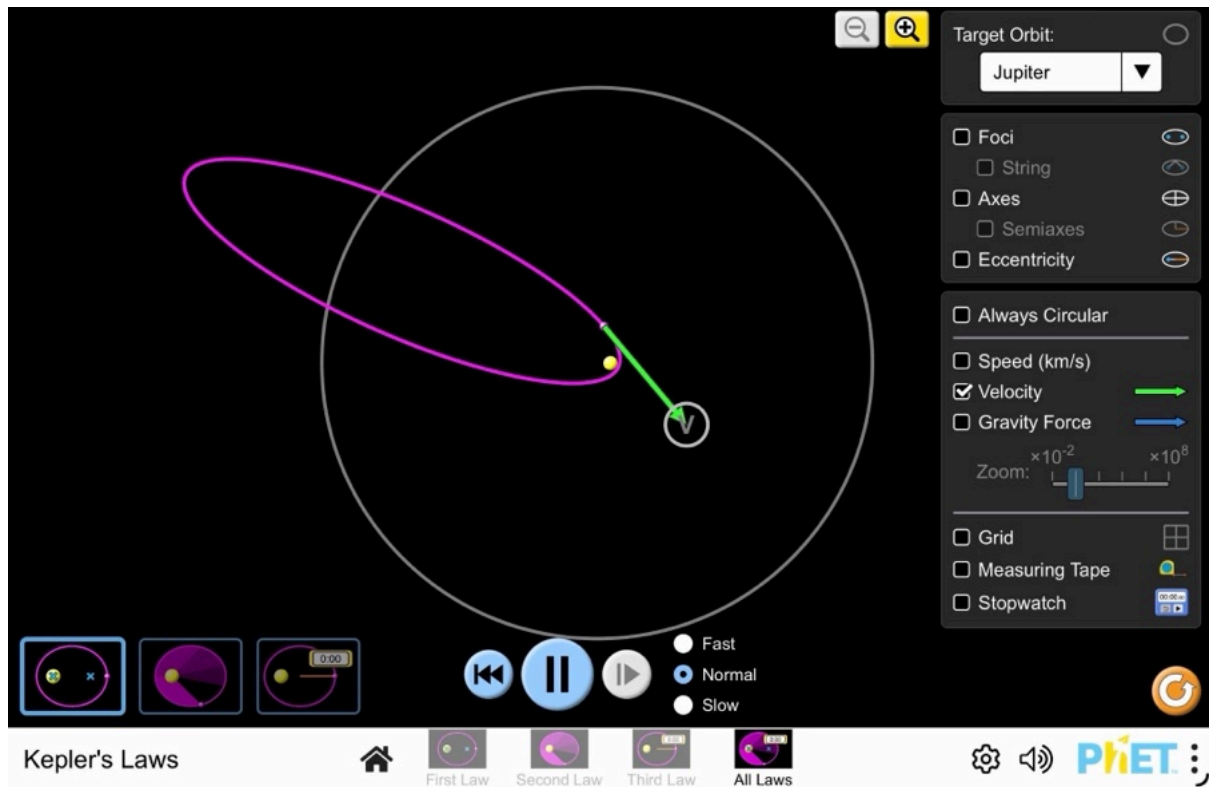
As Dr Mahoney talked about Kepler's laws being very well simulated, I wanted to investigate one of these simulations. The PhET Kepler's law simulation allows the user to pick which one they would like to simulate or if they would like to simulate them all. I will use the mode where all are simulated as it gives the best representation of the simulation.

This simulation is very similar to the other PhET simulation, but with more focus on Kepler's laws. The user can select parts to add to the simulation such as 'Foci', 'Axes' and 'Eccentricity'. This can all be seen in screenshots below.

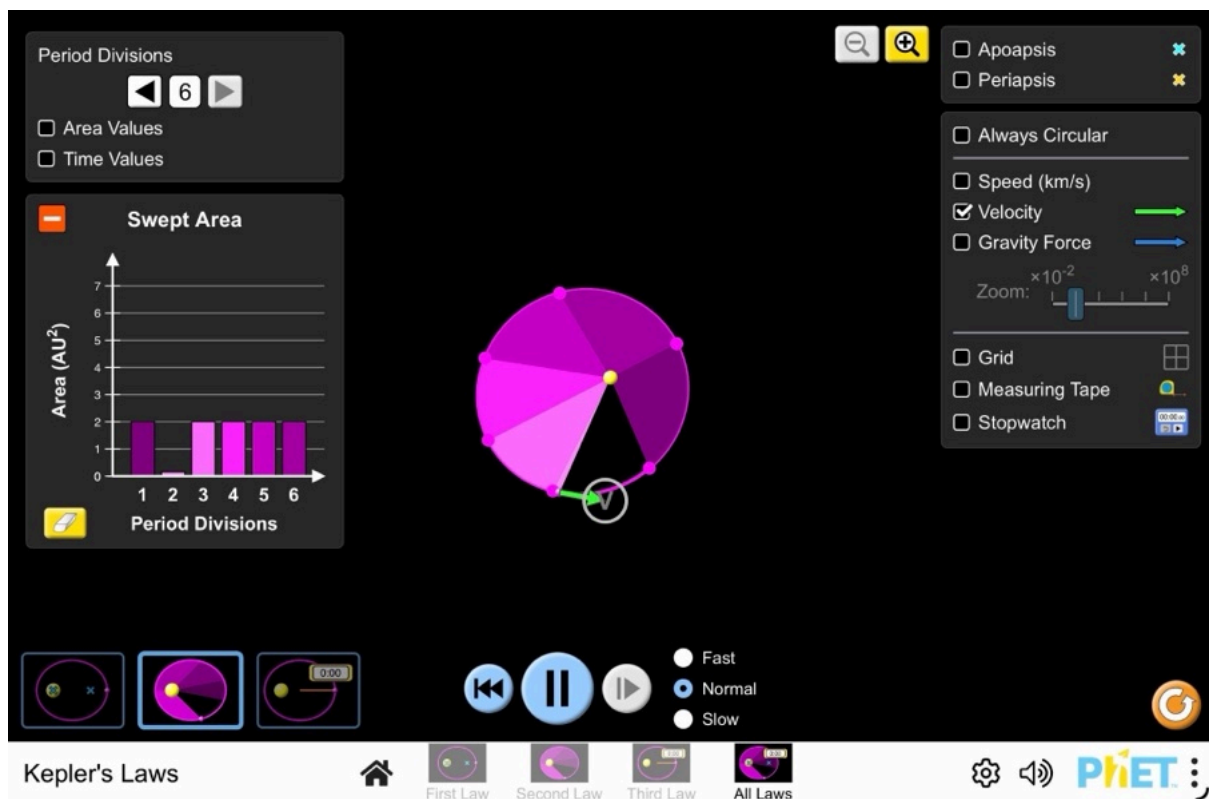
This simulation allows the user to edit the orbit of the planet by dragging it across the screen. Each law has a different interface that comes along with it, the first and second laws are screenshot below.



First Law:



Second Law:



Features I like	Justification
Being able to simulate all 3 of Kepler's laws.	Only being able to simulate one of Kepler's laws would make the simulator much less usable because users would have to find other simulators for the other laws. In addition to this, all the laws are derived from the same principles so it is intuitive to have them all simulated in 1.
Being able to pause/play the simulation.	Allows the user to view what is happening in more detail.

Features I dislike	Justification
The user interface is very complicated and has no explanation.	This means that users by themselves cannot understand what is going on. For students new to this topic, this serves very little educational value because they won't know what each part means. Therefore, the simulation is reliant on the student already knowing the content or a teaching being present to explain the simulation. This takes away from the simulation as much of the time students will be revising or learning by themselves at home without the assistance of a teacher.
The lack of customisation.	This simulation has very little in terms of interaction. Other than being able to toggle parts of the simulation on and off and move the planet around somewhat (it can only be put in certain positions), there is no customisation. This makes it seem much more like something that a student would watch rather than engage with, making it a less effective learning tool.

Conclusion

Although I am not simulating Kepler's laws, looking at this simulation was still valuable. I can see how this is a good simulation from the perspective of a teacher, as they can be there to talk their students through each part of the simulation. However, my simulation should not have to rely on a teacher being present for it to be a helpful tool. This is something that I will consider as I go through my design process and development.

The Proposed Solution

Overview

Initial concept

My initial solution is to create a 3D simulation which allows the user to add masses/planets as they wish. Any customisation to planets will happen before they are added to the simulation and will mostly be dictated by sliders. Properties such as initial position, initial velocity and mass can all be edited using these sliders. Position and velocity will have to be split into 3 sliders (x, y and z) to account for all directions in 3D. When a planet is added it appears on the screen in the position and velocity it was created with.

While I will use Newton's law of universal gravitation for calculating the gravitational force between planets, the space-time grid will give a visual representation of space-time which is a concept of General Relativity. It will allow the user to see how space-time warps around planets, rather than being an accurate general relativity simulation. This is part of making it educational for the user and allowing them to learn about General Relativity. The other educational value should be provided by messages displayed to the user.

Features

Below is the initial list of features that I have outlined as essential for a successful simulator.

Feature	Description	Justification	Source
Ability to adjust masses/planets	Being able to adjust the mass of planets in the simulator is essential, whether that be using a slider or pressing and holding for a longer time.	It is important that a user is able to see what happens when the masses of planets change for them to get a better idea of how gravity works.	PhET Gravity Simulator NSTMF Gravity Gravity Simulator Student stakeholder form
Ability to adjust the initial position of the planets	The user should be able to use a user interface to place planets into the simulation where they want.	In order to set up orbits, planets will have to be set up with different distances between them.	PhET Gravity Simulator NSTMF Gravity Gravity Simulator
Ability to adjust the initial velocity of the planets	The user should be able to create planets with different initial velocities.	This is necessary in order to set up stable orbits, as for a planet to orbit	PhET Gravity Simulator

		another planet it must have velocity.	NSTMF Gravity Gravity Simulator
Ability to remove a given mass	The user should be able to use the UI to remove a given mass.	This allows the user to see the effect on a planet's movement if other masses are removed from the simulation, an important demonstration of how gravity works.	Gravity Simulator
Ability to reset the entire simulation	The user should be able to use the UI to reset the simulation, removing all planets and returning the space-time grid to its original form.	This allows the user to return to an unmodified environment when they are done with their current situation that they have set up.	PhET Gravity Simulator NSTMF Gravity
Ability to pause the simulation	The user should be able to use the UI to pause the simulation.	This allows the user to add planets in this pause, allowing for the ability to add many planets before playing the simulation. It also allows the user to pause the simulation and move the camera to view the space-time grid with more detail.	PhET Gravity Simulator Gravity Simulator PhET Kepler's Law Simulation Student stakeholder form
Space-time grid	A grid that shows how space-time is affected by the planets that are added.	It allows the user to more easily grasp the concept of general relativity by showing how planets affect the fabric of space-time.	NSTMF Gravity Student stakeholder form
Educational Prompts	Giving the user interesting information relevant to what is happening in the simulator at any given moment.	Without this feature, users may use the simulator without knowing what is going on. For an educational tool, I think this is essential.	NSTMF Gravity Student stakeholder form

Ability to create objects and scenarios	Allows the user to add, delete and edit planets as they please.	In order for a simulator to be effective, the user should be able to add planets and create orbits using these orbits. This gives the user an interesting and educational experience.	NSTMF Gravity Gravity Simulator Student stakeholder form
Simplicity	Keeping the layout of the simulator simple by removing unnecessary details that the average student won't understand.	In order for the simulator to be effective for the average person, it needs to be understandable and usable, which can be achieved by simplicity.	PhET Gravity Simulator NSTMF Gravity
Trails/orbit paths	A path following each planet in the simulator that shows where they have been. This should also show the orbit path of planets once they have a stable orbit.	This helps visualise the orbits of planets, allowing the user to more clearly see the movement of planets.	PhET Gravity Simulator NSTMF Gravity Gravity Simulator PhET Kepler's Law Simulation Student stakeholder form
Web-based	Being able to access the simulator via the web.	All my stakeholders had the preference of a web-based simulator rather than a downloadable one.	PhET Gravity Simulator NSTMF Gravity Gravity Simulator PhET Kepler's Law Simulation Student stakeholder form
Ability to see the simulation run in 3D	When the planets are added, they appear in the 3D	The 3D element will allow the user to clearly see	Gravity Simulator

	environment where they interact with other planets and the space-time grid	gravitational attraction between planets.	Teacher interview
Ability to add multiple masses	The user will be able to add several masses to the simulation.	This is necessary for the user to be able to see the gravitational influence between the planets	PhET Gravity Simulator NSTMF Gravity Gravity Simulator
Camera Controls	The user should be able to explore the 3D environment as they please	This is important for the user to explore and investigate gravitation affects themselves. It will be particularly necessary for seeing how the space-time grid distorts due the influence of planets.	PhET Gravity Simulator NSTMF Gravity Gravity Simulator
Planet labels	Each planet should have a corresponding label which shows the planet's name.	This allows the user to track given planets across the environment easily.	Gravity Simulator

Limitations of Proposed Solution

Accuracy

When modelling the orbits of the planets, integration errors may occur over time. Simple numerical integration methods like Euler's methods can become less accurate over time, while more complex methods could hinder the performance of the simulator.

Performance

When using the [Gravity Simulator](#) my computer was very slow, sometimes lagging in the process of a simulation. One concern I have is that the simulator I create will be slow and/or lag. Running a simulation in real time requires a lot of computational power because of the number of calculations that have to happen per second. This could really affect the educational value of the simulator if the user struggles to run the simulation without it being slow.

Abstractions/Realism Limitations

1. Firstly, I will completely simplify the collision of planets in the simulation, most likely choosing to delete both planets automatically if they collide. This is because I am focusing on gravitational physics rather than incorporating any collision physics. While this simplifies the problem, it also retracts from the realism of the simulation.
2. Secondly, I will ignore the effects of a planet's spin, and tidal forces as they will be too complicated to take into account. These factors would have had a slight impact on the behaviour of a planet in space.

Customisation

Third, I will be modelling all planets as spheres because modeling gravity for a non-spherical object is significantly more complex. Modelling as a point mass or perfect sphere instead allows me to use Newtonian mechanics which is much simpler than any approach required for non-uniform bodies. This means that objects such as satellites and rockets will not be able to be simulated using my simulator.

Requirements

Hardware and Software Requirements

Hardware requirements

Requirement	Explanation
Dual-core CPU, 2.0GHz	Simulations involve many calculations, having a computer with at least a 2.0GHz clock speed and two cores will allow real-time calculations and a functioning simulator that does not lag. This is fairly standard for computers to have, so shouldn't be an issue. I've set this theoretical minimum processing power because my simulation will perform many calculations per second and will require a fair amount of processing power.
4GB of RAM	A computer with minimum 4GB of RAM will avoid any performance issues. This should not be a problem because 4GB of RAM is typically considered the absolute bare minimum for a computer.

	I've set this as a requirement because to do the many calculations my simulation requires, the computer must have enough data storage to be able to process all of it.
Mouse/Trackpad	Users will need to zoom, pan, select objects; a mouse allows them to do this.
Computer with stable access to the internet	The simulation will run through the internet, so the user's computer must have internet access.

Software requirements

Requirement	Explanation
Modern Web browser	Will be used to access the simulator on the web. If the user cannot access the web, they won't be able to use the simulator.
Javascript enabled	Users may in a rare case have Javascript and other web languages disabled. It is important that these are enabled to run my simulation to allow it to run on the web.
HTML5 enabled	HTML will be used to structure my web page, so it is important that users have a browser that supports this and it is enabled.

Success Criteria

ID	Description	Justification	How is it measurable?
SC1	Planet's name can be customised using a text box.	Essential for the planet labels and the delete planet drop-down menu, which will require the user to select a planet based on its name.	I will carry out a series of tests to ensure this textbox correctly customises the name of planets.
SC2	Planets mass can be customised using a slider.	It is important to see the relationship between gravitational attraction and mass of planets.	I will carry out a series of tests to ensure the slider correctly customises the mass of planets.
SC3	The planet's initial position can be customised using a	Allows the user to set up orbits however they wish.	I will carry out a series of tests to ensure the slider correctly customises the

	slider for each component.		position of planets.
SC4	The planet's initial velocity can be customised using a slider for each component.	Allows the user to set up orbits and investigate gravitational interaction however they wish.	I will carry out a series of tests to ensure the slider correctly customises the velocity of planets.
SC5	The user can select whether or not they want a planet to have a trail path.	Allows the user to be able to see the trail path of desired planets, but can turn it off to avoid the environment being cluttered.	I will carry out a series of tests to ensure the toggle correctly customises the planet and renders the environment correctly. This can be evidenced using screen recordings
SC6	Once the user has customised a planet, they can add it to the environment using a button.	This allows users to add planets to the simulation intuitively.	I will carry out a series of tests to ensure that this button correctly adds planets to the system. This can be evidenced using screen recordings.
SC7	Ability to pause and play the simulation using a button.	Allows users to add planets before playing simulation and allows the user to pause and view the simulation in more detail.	I can evidence whether or not this works using screen recordings. This will go alongside tests.
SC8	Ability to reset the simulation using a button.	Allows users to return the environment to its original state when done with their current system.	Video evidence of a system with multiple planets being reset with the press of the button.
SC9	Ability to delete a given planet using a drop-down menu and button.	Allows users to delete planets they want to in order to see the effect this has on other planets.	Video evidence of the drop-down menu being used to delete a given planet.
SC10	Ability to move the camera position around the environment.	Allows the user to view the system as they wish and focus in on parts of the simulation.	Video evidence of the camera moving around the environment and focusing on a given planet/system. The stakeholders rank camera movement as at least a 4 for simplicity on average.
SC11	Educational messages are displayed to the user.	The simulation is designed to be a learning tool, this enhances that.	Screenshot evidence of messages being displayed to the user.

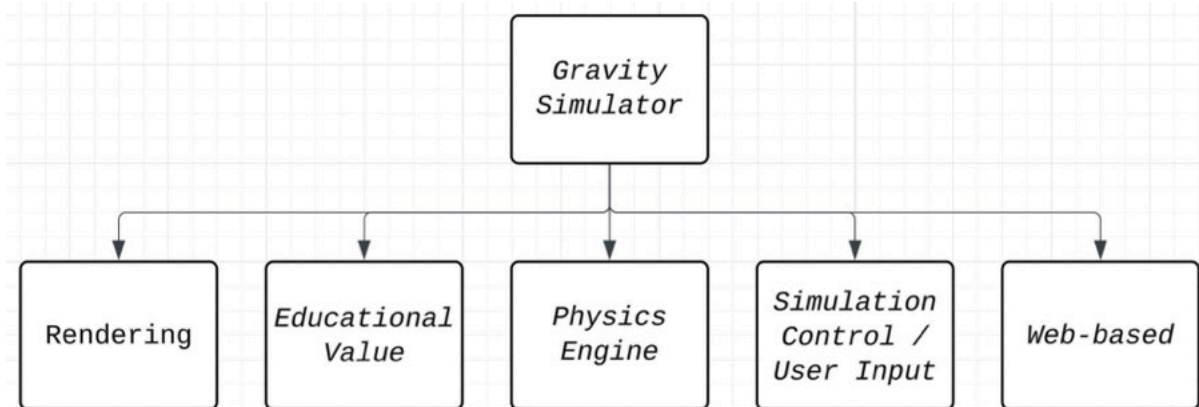
SC12	Educational messages change depending on what is in the simulation.	This makes the messages more relevant to what is currently happening in the environment.	Video evidence of the message changing depending on what is in the environment.
SC13	Labels for all planets in the simulation.	Allows users to track the planet's movement around the screen.	Video evidence of the label rendering and following the planet it is assigned to.
SC14	The simulator must be able to render the planets in the correct position on the screen.	The primary purpose of the simulator is to visualise gravity for the user to learn from. If the planets don't render in the correct place, this means that the simulator itself is flawed. The rendered position of the planet must reflect the backend computing of its position.	Screenshot evidence of planets being rendered to the screen, in the correct position.
SC15	The simulator must accurately simulate gravitational attraction between multiple planets using Newton's law of universal gravitation.	If the calculation is incorrect or poorly implemented, the resulting movement, trajectories, and orbits will be fundamentally wrong, rendering the application useless as a simulator.	Video evidence of planets in the environment attracting gravitationally to each other. If Dr Mahoney rates the simulation a % on accuracy or higher.
SC16	The simulator must update each planet's velocity, position and acceleration in real-time based on the forces acting on it.	If the updating of each planet's properties is not done successfully, then this affects their position on the screen. If these are not correct, the simulator does not serve its purpose as an educational tool.	Video evidence of the planets moving around the screen, with their position and velocity changing due to gravitational attraction with other planets.
SC17	Show planets with lighting and shading to emphasise their 3D form.	Gives the simulation a clear 3D aspect.	Screenshot evidence of 3D planets in the simulation.
SC18	The customisation of planets should be captured in the simulator by differences in the visual	Allows the user to see how more massive objects interact differently to less massive objects.	Video evidence of planets created with smaller masses having a smaller size and planets with larger initial

	characteristics of the planets such as size, position and velocity/speed.		velocities moving across the window faster.
SC19	Maintain a frame rate high enough for smooth animation.	Makes the simulation clear and usable for users.	Screenshot/video evidence of a smooth simulation. If stakeholders rate the simulation % for performance.
SC20	A space-time grid is rendered which initially is just a flat grid.	For the warps in the grid to be visible, it must first be rendered flat.	Screenshot evidence of the initial space-time grid rendering.
SC21	The space-time grid warps around planets which are added to the simulation.	Necessary for the educational value of the simulator.	Video evidence of the space-time grid warping around planets as they move across the simulation.
SC22	The space-time grid gives an accurate depiction of how gravity works.	If it is not accurate, its educational value is severely limited.	Dr Mahoney ranks the simulation at least a % for accuracy.
SC23	Trail paths render behind planets that the user has selected to have a trail path.	Allows the user to see where the planet has been, this maps gravitational interaction.	Video evidence of trail paths being rendered when selected.
SC24	Trail paths have a set length and points are removed as necessary to maintain this set length.	Prevents the simulation from being too crowded and preventing a clear view.	Video evidence of a planet moving with a trail path that is staying the same length.
SC25	A web-based simulation that users can access via the web.	Easier for users to access.	Screenshot evidence of the simulation running on the web.
SC26	The simulator is simple and clear.	Enhances user experience.	It ranks at least a % for simplicity on average by stakeholders. At least 80% of stakeholders say that it is clear what each part of the control panel does.
SC27	The user interface controls are responsive and work without delay.	Enhances user experience	It ranks at least a % for responsiveness on average by stakeholders.

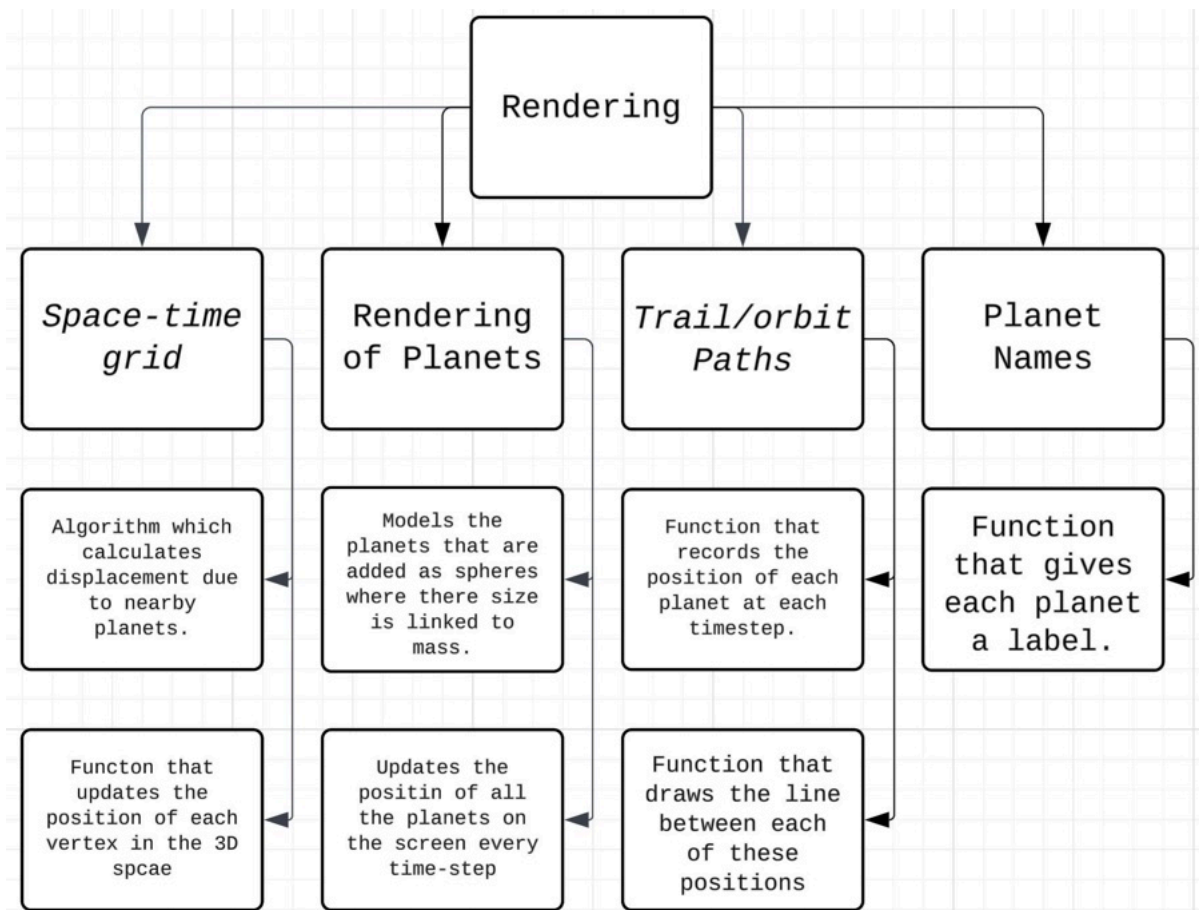
			There is video evidence of the user interface functioning without delay.
--	--	--	--

Design

Top-Down Design



Subproblem 1: Rendering



Space-time grid

Subcomponent	Explanation	Justification	How will I make it
--------------	-------------	---------------	--------------------

			computational
Position calculator	The space-time grid will be made up of vertex's. Each vertex will be able to move down from its original position, but only in the single dimension. I will use a formula based on the proximity and mass of nearby planets to calculate the displacement from its original position.	I've decided to create this separately from the physics engine, despite being very calculation heavy, because when creating the physics engine I can focus on accurately simulating gravity. When it comes to creating this subcomponent, I can then entirely focus on the calculations that are related to it.	This is suitable for a computational approach because I can use object oriented programming to create each vertex. I can use formulas and calculations to update the position of each vertex every time-step. This would be impossible without the speed of computation.
Rendering of the grid	Between adjacent vertices will be straight lines, forming a large grid. This is what the user will be able to view. A function is needed to render these lines.	I've separated this from the position calculator because this requires the use of the rendering code to plot the lines.	As this will be done using an API that allows me to open and manipulate a window. I need to use this API to take the position of each vertex from the backend and plot the relevant lines.

Rendering of Planets

Subcomponent	Explanation	Justification	How will I make it computational
Creation of planets	I'm going to use object oriented programming to create the planets. I'm going to create a class for planets from which I can create individual planets.	This is a subcomponent of the rendering because while I'm using an API for the front end, this will be done using backend code.	The planet class will have the attributes: - Name - mass - radius - initial velocity (x and y) - initial position (x and y) - Trail paths (on/off) In addition to these it will also have a method which calculates the radius

			based on the mass of the planet. This will be based on a simple formula.
Position of planets	This function will take the updated position of each planet from the back end physics engine and update this on the window. All the planets will have different attributes, such as mass and	This needs to be separate from the physics engine because this is a front end function that will use the rendering API.	Use the input from the physics engine and the rendering API to change the position of each planet. This will occur every time step.

Trail/Orbit Paths

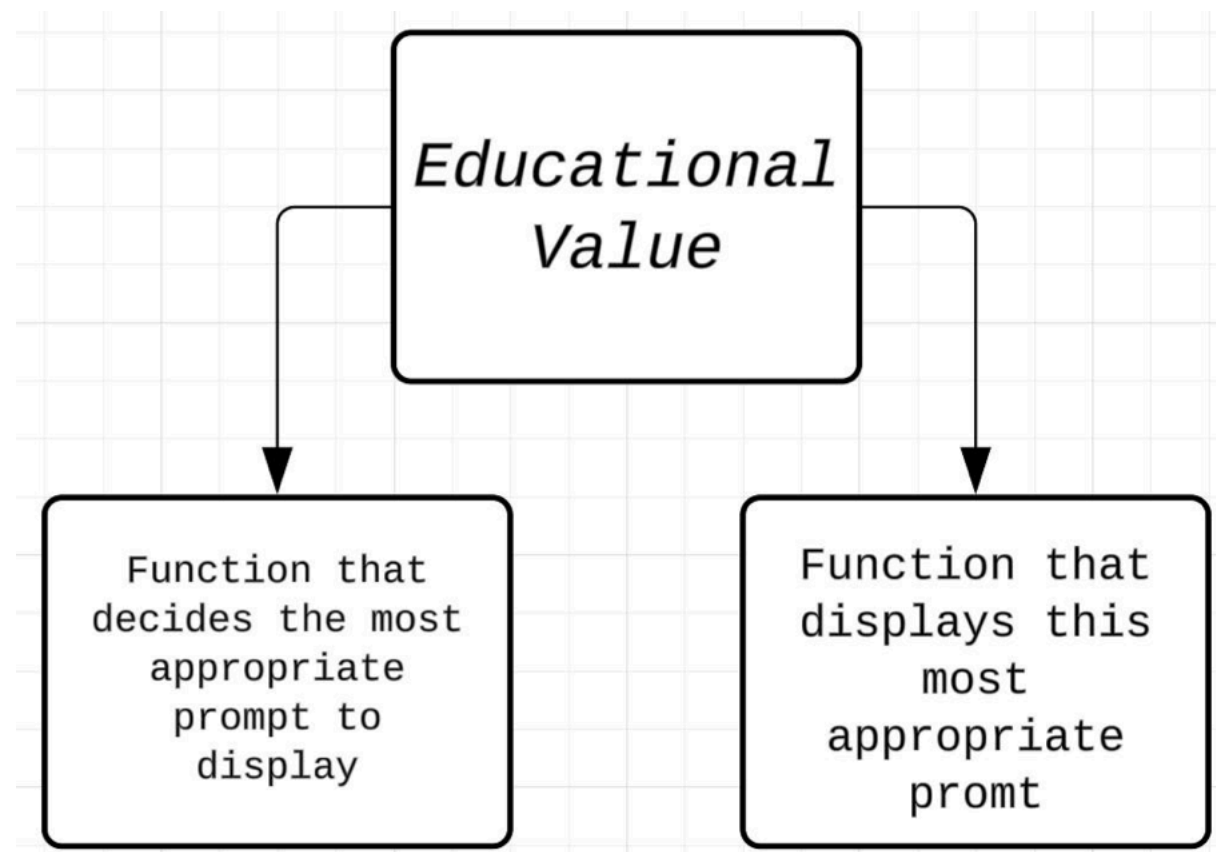
Subcomponent	Explanation	Justification	How will I make it computational
Storage of positions	The trail-path must only be a certain number of time-steps long to limit the length of it. If the trails got too long, they would disrupt the view of the simulation for the user.	This needs to be a separate algorithm that takes input from the main physics engine and adds this to the queue.	I will use a queue data structure to store the positions of the planet at each time-step. This queue will be fixed sized. Every time-step, all the items will be shifted back one place in the queue, with a new position added to the front and the last position deleted. This means that the trail path can only be a certain number of time-steps long.
Line plotting	On the front end, an algorithm is needed to take each position in the queue and plot lines between adjacent points.	This needs its own subcomponent, which takes the queue as an input, and plots the lines.	I will use the rendering API to draw visible lines in between the points. This should create a smooth line because the time-steps will be often enough that the space between points is small, even

			if the object is moving at a large velocity.
--	--	--	--

Planet Names

Subcomponent	Explanation	Justification	How will I make it computational
Rendering the planet names	Next to each planet will be a label which shows the name of the planet which the user inputted.	This is separate to the other rendering subcomponents because this will involve text-based rendering, rather than shapes, points and lines. Therefore, when I come around to creating this, I can focus on rendering the	I will take input from the simulation controls subcomponent. Then, I can use the rendering language to render this label. I will use the position of the planet in the current time-step and plot the text here or slightly offset.

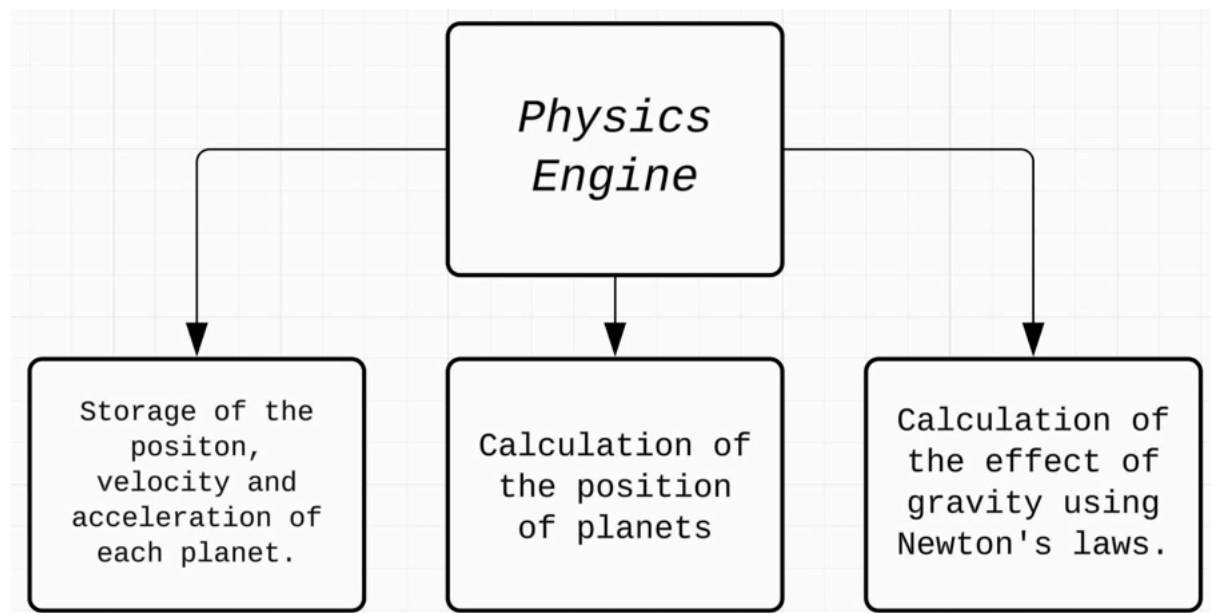
Subproblem 2: Educational Value



Subcomponent	Explanation	Justification	How will I make it computational
Deciding the most appropriate prompt	There will be several different prompts stored in their own file. Each prompt will have related elements which dictate when they will be displayed. This subcomponent will make a decision on which prompt is most relevant to what is happening on screen, which it will input into the next subcomponent, which will display the prompt. When a prompt has been displayed, it shouldn't repeat.	I've chosen to make this separate to the function which displays the prompt because that subcomponent will be more focused on accessing files.	In order to make this computational, I will have to take several inputs from the physics engine. For example, if two planets are exerting a large gravitational force on each other, then a prompt that is about gravity will be displayed. I will use if/elif statements to do this. In order to not repeat prompts, the algorithm will keep a list of prompts which have already been used. This list will be referenced when

			selecting the most relevant prompt.
Displaying the selected prompt	Once the most relevant prompt has been selected, this subcomponent will display this on the user interface.	This subcomponent is separate from the previous one because it can take a simple input from it in order to display the prompt. It will be simpler to make if I can focus on the interface module separately.	I will use HTML to display the prompt. The back end will take the prompt from the file it is in and input it into the front end. I will use CSS to style it if necessary

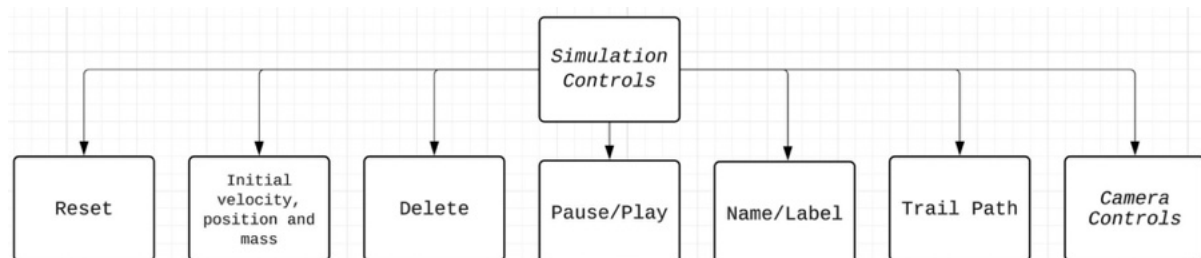
Subproblem 3: Physics Engine



Subcomponent	Explanation	Justification	How will I make it computational
Storage of user input	The user input will come from buttons and sliders on the user interface. These will need to be stored so they can be used by the physics engine.	Using a separate subcomponent for storage of inputs gives the solution modularity and will make it easier when I get to the development stage.	This will require the use of JavaScript to take inputs from the buttons and sliders. These inputs will be stored as variables.
Calculation of the effect of gravity	Each planet will have a mass and	I've broken the calculation side of	I will use formulas to calculate the force

	therefore will affect other planets in the simulation through gravity.	the physics engine into two parts. Calculating the effect of gravity has its own subcomponent because I can test each separately. Furthermore, it makes it easier to debug if the module is simpler.	that each planet exerts on a planet. I then calculate the magnitude and direction of the magnitude that acts on a planet. I will use this to calculate the resultant acceleration on the planet, which will affect its velocity.
Calculation of the position of the planets	This subcomponent will take the velocity and acceleration of each planet from the previous subcomponent and use this to update the position of the planet.	The reasons for this being separate are very similar to the reasons I've given above. The main ones being: easier to debug; easier to test and makes the project more manageable.	This will be created by using SUVAT equations in 2D. I will therefore have to run two calculations, one in x direction and one in the y direction.

Subproblem 4: Simulation Controls



Subproblem 4 contains the backend functions that are necessary to allow the user to have a customisable experience. All these functions will be linked to input from the user interface.

I've split user input into 7 different subcomponents.

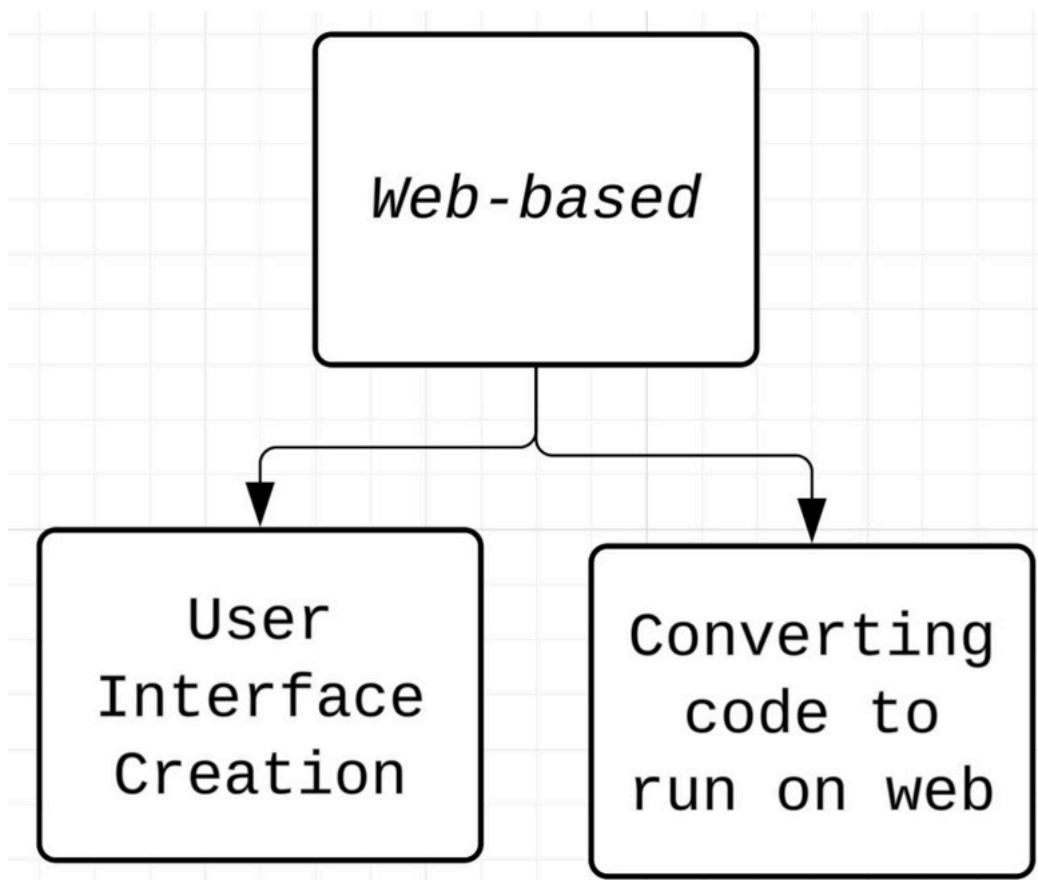
Justification: I've split them up because each subcomponent has a different way of the user entering data into them.

Subcomponent	Explanation	Justification	How will I make it computational
Reset	This function will revert the simulation window to its	This is separate as it will have to use functions from other	This function will call the delete function on all the planets. I

	original state. This involves deleting all planets and returning the space-time grid to its original position.	areas in the program. For example, the delete function will have to be used several times when resetting the simulation because planets will have to be removed from the simulation.	can do this by iterating through each added planet and calling the delete function. For the space-time grid I can iterate through each vertex and set its displacement back to 0. I can iterate through them by using loops.
Initial velocity, position and mass	This function is simple. It will take data from the user interface and use it to create a new planet from the planet class. This new planet can then be used by the rendering and physics engine components.	I've decided to make this separate from the physics engine and rendering components because when creating those, I won't be focused on creating the OOP for the planets. By focusing on this separately, it will be easier to debug and test.	I will use object oriented programming. The data will come in from the user interface, and a planet will be instantiated from the planet class using this data.
Delete	The delete subcomponent will allow users to delete planets from the simulation as it is running.	This function has its own subcomponent because it can be reused wherever necessary in the project. For example the reset subcomponent will use it to remove all the planets from the simulation.	In order to delete a planet, I will have to remove it from the data structure - likely an array or list. The physics engine and rendering will be linked to this central data structure. Therefore, when I remove an object from it, no aspects of the simulator will have access to it.
Pause/Play	The user will be able to press a button on the control panel and pause/play the simulation.	This contained subcomponent can manage the simulation state - paused/playing. I therefore won't have to think about this when creating other subcomponents	In order to implement this feature, I will create a boolean variable for whether or not the simulation is running. On the user's input, this variable will

		such as the physics engine. This subcomponent also acts as a mediator between the user input and the physics engine.	change. If this variable is true, the physics engine will run. If not, the physics engine will not run.
Name/label	While the simulation is running, each planet will have a label that the user can choose.	Acts as a mediator between the user input from the UI and the rendering function which makes this visible to the user.	The user's input will become an attribute of the planet, which can be used by the front-end to render this.
Trail path	This subcomponent will enable the user to toggle whether or not they want a planet to have a plotted trail path.	As trail paths are a visual feature, which require rendering, this subcomponent keeps the front end separate from the backend, acting as a mediator. This makes the project easier to debug, test and keep modularised.	The user will select whether or not they want the trail path on the UI. This will become an attribute of the planet, stored as a boolean variable.
Camera Controls	This subcomponent will allow the user to explore the simulation by moving the camera around the worldspace.	Camera controls require a totally different way of handling user input as they will use their mouse and scroll wheel to move the camera around.	The camera will be centred around the origin and when the user left clicks and drag the camera will move around the origin. The scroll-wheel can then be used to zoom in and out.

Subproblem 5: Web Based



Subcomponent	Explanation	Justification	How will I make it computational
User Interface Creation	This subcomponent will introduce the user interface for the user.	It is separate from the other modules because I will want to focus on this separately after I already have a somewhat working simulation.	I will need to edit the HTML, CSS and Javascript files to display a user interface and allow the user to input text, edit mass, position and velocity.
Converting code to run on web	I will need to create code that has the ability to run on the web. This may involve converting code from the language I choose into these web languages.	This is separate because again I want to have a working backend before compiling this into web-compatible languages.	Make any necessary changes to my backend code. Convert backend into the necessary languages or use any necessary APIs.

Structure of the Solution

Justification of my Choice of Coding Language

For my project I'm going to use C++. This is because C++ supports object-oriented programming. This will allow me to create classes for my planets, labels, trail paths and space-time grid all with their own attributes and methods.

In addition to this, when using some of the other simulators in my analysis stage, I found that they didn't perform very well and would have a low frame rate and not respond very quickly to user input. In addition to this, my simulator will be performing many calculations continuously to create smooth, real-time motion. C++ is a low level language that compiles directly into machine code, allowing it to perform faster than languages such as Python or Javascript. This will maintain good performance for the simulator.

Finally, C++ integrates with my chosen rendering APIs, OpenGL and GLFW.

'OpenGL is a cross-platform, open standard graphics API used for rendering 2D and 3D graphics' - from OpenGL website.

OpenGL gives me the commands I need to draw points, lines and triangles, apply transformations and add lighting to scenes within a window. It also allows me direct communication with the GPU, enabling smooth visualisation and a reduced load on the CPU. It is also cross-platform which means my program can run on different operating systems without needing to make changes to the code.

'GLFW is an Open Source, multi-platform library for OpenGL... It provides a simple API for creating windows, contexts and surfaces' - GLFW website.

This provides a simple way to create a window which OpenGL can then use to draw onto. GLFW also allows the user to interact with the simulation. GLFW is also cross-platform.

Rendering

The rendering module of the problem comes in 3 different forms: rendering of the space-time grid; rendering of trail/orbit paths and rendering of the objects.

1. Space-time grid: the space-time grid will be a 2D grid of vertices with edges between adjacent ones, forming a grid of squares. The displacement of each vertex will be affected by the proximity of massive objects nearby, if more massive objects are nearer, there will be greater displacement. The displacement of a vertex can only be up and down, so will only be plotted in a single direction. To incorporate this I will require the following functions:
 - A function that calculates the displacement of each vertex every time-step. As the vertices can only move up and down, the value of a vertex's displacement can be stored as a double.
 - A function that updates the position of each vertex in the rendered 3D space. This provides the user with a grid that reacts to the conditions of the 3D space, allowing the user to see how space-time is affected by the addition of objects.

2. Rendering of objects: a very important aspect of the simulator is the visual aspect of adding objects and seeing how they interact with each other. Therefore, ensuring that the rendering of these objects is effective is essential. In order to achieve this I will use the following functions:
 - A function which initially renders objects in the 3D space. The size of the object will be directly proportional to the mass. As a result, if a user creates an object with a large mass, it will have a greater size than an object with a smaller mass.
 - A function that takes input from the physics engine and updates the position of each object on the screen every time-step. This is again crucial for a user experience that will be educational, as without this feature, the interaction between objects may be unclear.
3. Trail/orbit paths: in order to plot the paths of objects in the 3D space I will need to keep track of the previous positions of planets and plot these positions in the space. I haven't come across any 3D simulators which use trail/orbit paths so I have nothing to base it off of. However, I think the best way to do it is to take a record of the position every time step and plot a line between each of these points. The line shouldn't stay in the space forever, so part of the functionality should make the line only so long, with points disappearing after a certain amount of time steps. The plotted trail is important because it will allow users to see clearly how objects move through a space when under the influence of gravity. This will require the following functions:
 - A function that records the position of each object and stores this for use for plotting the line. These points can be stored in a list. This will not be limited to a single object, but however many objects there are in the simulator at any given time. This function also should have the ability to delete positions from the front of the list to ensure the trail path doesn't exceed a certain length and make effective use of memory.
 - A function is also required to do the plotting of the line by plotting points and connecting them with lines.

Educational Value

In order to provide educational value to the user the simulator will provide educational prompts to the user in the form of text. This text box will provide information about what is currently being displayed on the screen. This feature will require these functions:

- A function that decides the most appropriate prompt to display. I will achieve this by taking data from the physics engine. For example, when one object is exerting significant force on another, a prompt can be displayed about how gravity affects objects. I can do this for the space-time grid, creation of orbits and other interesting events. This is important because it will increase the educational value for the user more effectively than random prompts, as well as making it a more immersive experience.
- A function that displays this prompt is necessary. This will display a pre-made prompt based on input from the previous function. This text will be displayed in one of the corners, in a non-invasive way that allows the user to ignore them should they wish. This function will also record which prompts have and have not been used and use

this to ensure that no prompts are repeated. This will mean that at some point, no more prompts will be displayed as all have been used, but I think this is suitable as users may wish to use the simulator without prompts beyond this point.

Physics Engine

The functionality of the gravity simulator will come from the engine that runs the calculations. Each object will have an initial velocity, position and mass. The velocity of an object will be affected by forces acting upon it by other objects. These forces will affect the acceleration of an object. The following functions are required:

- A function to store all necessary data about an object. Velocity, position and acceleration will all be stored as 3D vectors, which I can store using vector doubles in the backend. This is important because it allows for all these aspects to function in a 3 dimensional space. It is also important because this data will be used in other parts of the project. For example, the position of the object is used in the space-time grid and rendering the objects themselves.
- A function that calculates the effect of gravity using Newtonian mechanics. This function will implement Newton's formula to calculate these effects. This is essential because it does the work of simulating gravity, allowing objects to interact correctly with each other. This is crucial for providing the user with a realistic idea of how gravity works.
- A function that takes the calculated effect of gravity and applies it to the acceleration, velocity and position of each planet. This will use SUVAT mechanics with an assumed time of 1, updating each of these every frame. This process of updating position and velocity in this way is called Euler's method of numerical integration. There are several different ways of carrying out numerical calculations like this. I have chosen to use Euler's method because it is the simplest to implement. Although other methods such as Verlet or Runge-Kutta integration might be more accurate, as I am creating an educational tool rather than an accuracy simulation or prediction model, these methods are unnecessary.

Simulation Control

The user will be able to control several aspects of the simulation. Before the simulation starts, the user will be able to edit the characteristics of each object and then add them to the space. The simulator will allow the user to enter multiple objects. When the user has finished adding objects to the simulation, they will be able to play the simulation and observe what happens. When they wish, they will be able to pause the simulation and delete or add masses. Users can input play on the simulation and see how their changes affected the simulation. In order to create this I require the following functions:

- **Initial velocity, position and mass:** A function that takes and stores the input of the user about the characteristics of each object that is added to the simulation. I will do this by instantiating objects from a class of objects using OOP. The class will have the attributes: mass, position, velocity and radius. Radius will not be decided by user input, however, but instead will depend on the mass. This function is important

because it allows to customise simulations, making the simulator an overall more inspiring educational experience.

- **Delete:** A function that removes objects from the simulation. The user should be able to select an object from a drop down menu and have the option to delete it. The function will then remove it from the simulation and can remove the instantiated object from memory.
- **Pause/Play:** A function that allows the user to pause/play the simulation freely. I can do this by introducing a boolean variable and only running the simulation while this variable is true. This is an important feature because it will allow the user to edit the simulation while it isn't running. This enables users to edit several objects in a single time-step and see the effect this has. Another reason this function is crucial is because it lets users pause the simulation and explore the simulation and observe the effect gravity is having on the objects as well as the space-time grid.
- **Reset:** A function that resets the simulation. This will involve deleting all objects and resetting the displacement of the space-time grid. I can do this by clearing the list of planets. The reset simulation is then ready for the user to add another set of objects and run the simulation again. This is important because it will allow the user to run the simulation multiple times with different scenarios.
- **Name/Label:** The user will be able to name each planet. These names will show up as a label next to each planet. This allows the user to easily see how the position of each different planet changes, which is important to make the simulator user friendly.
- **Trail paths:** Users will be able to choose whether or not each planet has a trail path. This is important because I want users to be able to set up systems in which a planet orbits another planet and this will work best if the central planet doesn't have a trail path and the orbiting one does. This also increases the customisation of the simulator, which makes it more user friendly.
- **Camera Controls:** I will use an orbit camera that circulates around the origin of the worldspace. The user will be able to use their mouse to navigate this. This allows the user to look at systems in more detail and observe the space-time grid at angles different from the default camera position.

Web Based

In order to run my code on the web I will have to convert it into the necessary languages (HTML, Java, Webassembly). This is because I won't use javascript for creating my simulation. There are several ways I can do this. If I choose to use python, I can have two options:

- Use server-side web development: my Python code runs on a web server and handles the logic and sends HTML/CSS/JavaScript to the user's browser.
- Use browser-side web development: allows my Python code to run directly within the user's web browser, similar to how JavaScript works, enabling interactive elements without constant communication with a server.

However, if I use a low-level language such as C, C#, C++ or Rust I will probably have to compile this into webassembly, HTML and Javascript and then run this on the web. I can then run this compiled code on the web.

Variables

In terms of what naming convention I'm going to use, I will use camel case. As there is different version of camel case, these are the exact two rules I will follow when naming variables:

- All variables, functions and methods will start with a lowercase letter, any other words in the variable, function or method name will start with an uppercase letter.
- All classes will start with an uppercase letter, any words in the class name will also start with an uppercase letter.

Here are some of the key variables I will use and as explanation of them:

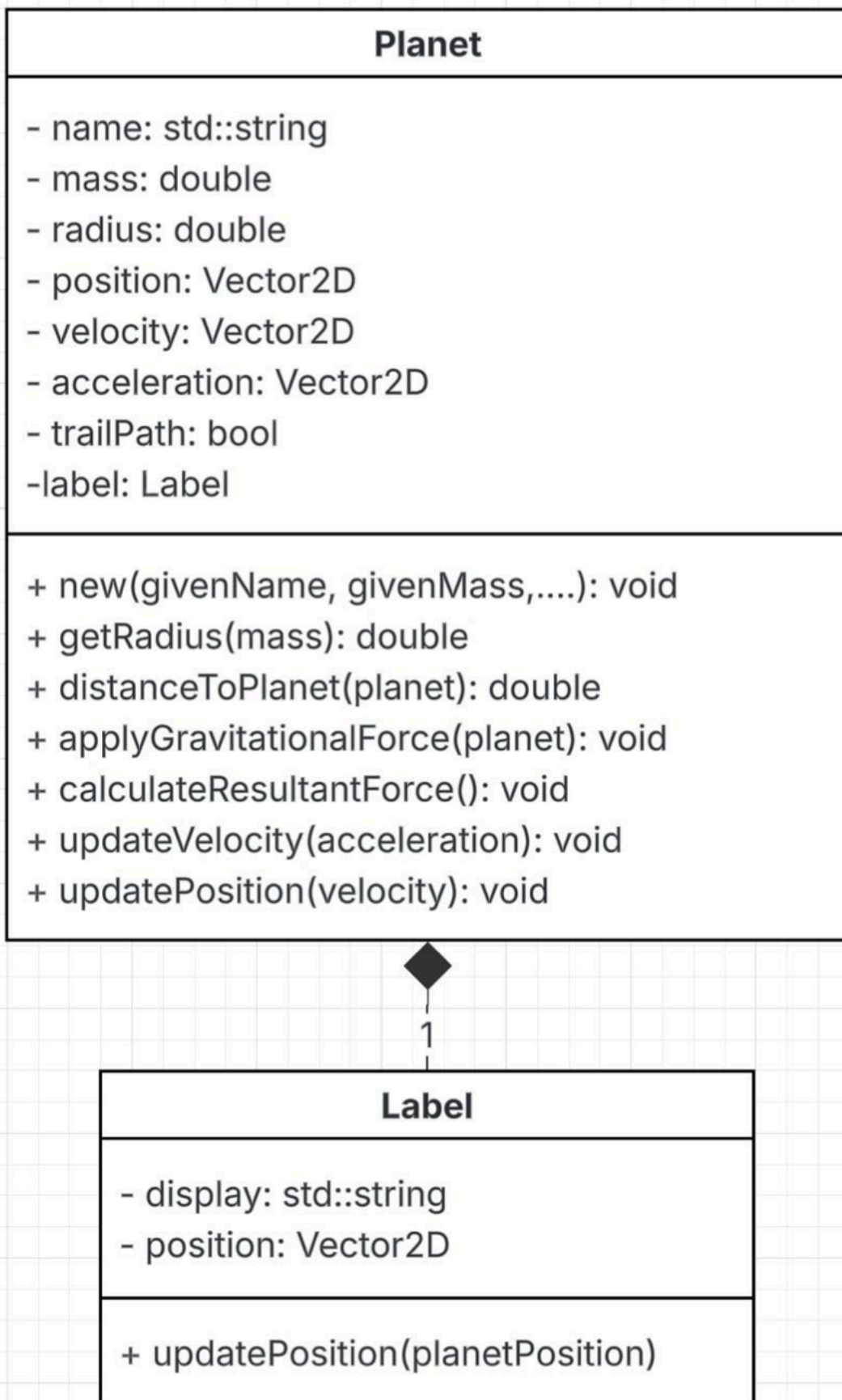
- **dt** - stores the time-step that will occur every frame (double), this will be 1 as I have talked about previously unless I modify it.
- **isRunning** - keeps track of whether the simulation is paused or played (boolean).
- **G** - the gravitational constant. This will be used in the gravitational calculations (double).
- **windowWidth** - defines width of the window (double).
- **windowHeight** - defines height of the window (double).

Pseudocode Classes

I'm going to start with the physics engine as this will form the backend of the simulator. Firstly, I will design the object oriented side of the simulator, which involves the planet class that will act as a blueprint from which planets can be created from. I will also need to create a vertex class for the space-time grid, a points class for the trail-paths and a label paths to do along with the planet class.

It is important to note that I haven't included any draw or render functions into my class definitions at this point. This is because OCR pseudocode does not contain any rendering commands (I'm using this guide as a reference [OCR Pseudocode Guide](#)). I will refer to drawing functions in the layout of my main lope code but will not specify them in the following section.

Planets Class



The planet class is used to create planets which can be added to the simulation. The class will be used as a template from which I can create instances of this class. This is a suitable way of doing it as the simulator is designed to be used with only a few planets. This means each planet can have its own properties and can be stored separately.

```

1  class Planet
2      private name
3      private mass
4      private radius
5      private position
6      private velocity
7      private acceleration
8      private trailPath
9      private label
10
11     public procedure new(givenName, givenMass, givenPosition, givenVelocity, givenTP)
12         name = givenName
13         mass = givenMass
14         position = givenPosition
15         velocity = givenVelocity
16         acceleration = 0
17         trailPath = givenTP
18         label = new Label(name, position)
19     end procedure
20
21     public procedure getRadius(mass)
22         radius = mass * k
23         //k is an undecided constant
24     end procedure
25
26     public procedure distanceToPlanet(otherPlanetPosition)
27         distance = -1
28         distance = sqrt((otherPlanetPosition.x - position.x)^2 + (otherPlanetPosition.y - position.y)^2)
29         return distance
30     end procedure
31
32     public procedure applyGravitationalForce(otherPlanetMass)
33         force = -1
34         force = (G * mass * otherPlanetMass) / distanceToPlanet(otherPlanetPosition)^2
35         return force
36     end procedure
37
38     public procedure calculateResultantForce(planetList)
39         resultantForce = 0
40         for planet in planetList
41             resultantForce += applyGravitationalForce(otherPlanetMass)
42         next planet
43         acceleration = resultantForce/mass
44     end procedure
45
46     public procedure updateVelocity()
47         velocity = velocity + acceleration * dt
48     end procedure
49
50     public procedure updatePosition()
51         position = position + velocity * dt
52     end procedure
53
54 end class

```

Attributes: The attributes name and mass will be chosen by the user. The radius is calculated by the getRadius() method. These attributes will not be able to be changed once added to the simulation.

The attributes position, velocity and acceleration are stored at 2D vectors. The user inputs the values for position and velocity using the user interface. The acceleration is calculated using physics equations within the class. This will have an affect on the velocity and therefore position.

The attribute trailPath is a boolean value that stores whether or not the given instance of the planet class will display a trail path in the simulation. This is decided by user input from the user interface.

new(): The constructor method takes all the user input from the user interface and uses it to instantiate a new planet.

getRadius(): This method uses the inputted mass from the user to calculate the radius of the planet. As mass and radius will be directly proportional, I'm going to use the equation: $\text{radius} = \text{mass} * k$. Where k is a constant I will decide in the development stage of the project.

distanceToPlanet(): This method calculates the distance between the centres of two planets. This is necessary as this value is needed in the applyGravitationalForce() method. To find the distance between two planets, I will use the equation:

$$\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}.$$

Where (x1, y1) are the coordinates of the centre of the first planet and (x2, y2) are the coordinates of the centre of the second planet.

applyGravitationForce(): This method will calculate the force that another single planet exerts on the planet that this method belongs to. I will do this by using the equation:

$$F = G * m_1 * m_2 / r^2.$$

Where:

- F is the force on the planet, measured in Newtons, N.
- G is the gravitational constant. This is a constant which gives the strength of the universe's gravitational field induced by a mass. The value of this is 6.6743×10^{-11} . However, in order to make the simulator easy to understand and use, I might change this value in the development stage of the project.
- m1 and m2 are the masses of the two planets in this interaction. This will be obtained from the classes of each planet and is measured in kilograms, kg.
- r is the distance between the centres of the mass, measured in metres, m.

calculateResultantForce(): This method uses the previous method to calculate the force of each other planet on the current planet. Then, this method will use these results to calculate a total resultant force. This resultant force is the force due to the combination of all of the forces. This resultant force can then be used to calculate an acceleration using the equation:

$$F = ma.$$

Where:

- F is the resultant force, measured in N.
- m is the mass of the planet, measured in kg.
- a is the acceleration, measured in ms^{-2} .

updateVelocity(): This method will take the acceleration that was calculated in the previous method and use this to change the velocity. This can be achieved using the equation:

$$a = (v - u) / t.$$

Where:

- a is the acceleration, measured in ms^{-2} .
- v is the final velocity, measured in ms^{-1} .
- u is the initial velocity, measured in ms^{-1} .
- t is the time, measured in s, stored in the variable **dt**.

As this calculation will be carried out every time-step, the time is 1. Subbing this into the equation gives: $a = v - u$. As we want to calculate the final velocity, we can rearrange the equation to give: $v = a + u$. Therefore, in order to calculate the final velocity, the algorithm will have to add the acceleration to the initial velocity.

updatePosition(): This method will take the updated velocity from the previous method.

Then, using the equation:

$$p2 = p1 + vt$$

Where:

- $p2$ is the final position
- $p1$ is the initial position
- v is the velocity
- t is the time, stored in the variable **dt**.

Similarly to the previous method, t is 1, due to this calculation happening every time-step. Therefore, $p2 = p1 + v$. The algorithm will add initial position and add the velocity to obtain the final position.

Label

```

1  class Label
2      private display
3      private position
4
5      public procedure new(givenDisplay, initialPositon)
6          display = givenDisplay
7          position = intialPosition
8      end procedure
9
10     public procedure updatePosition(planetPositon)
11         position = planetPosition
12     end procedure
13
14 end class

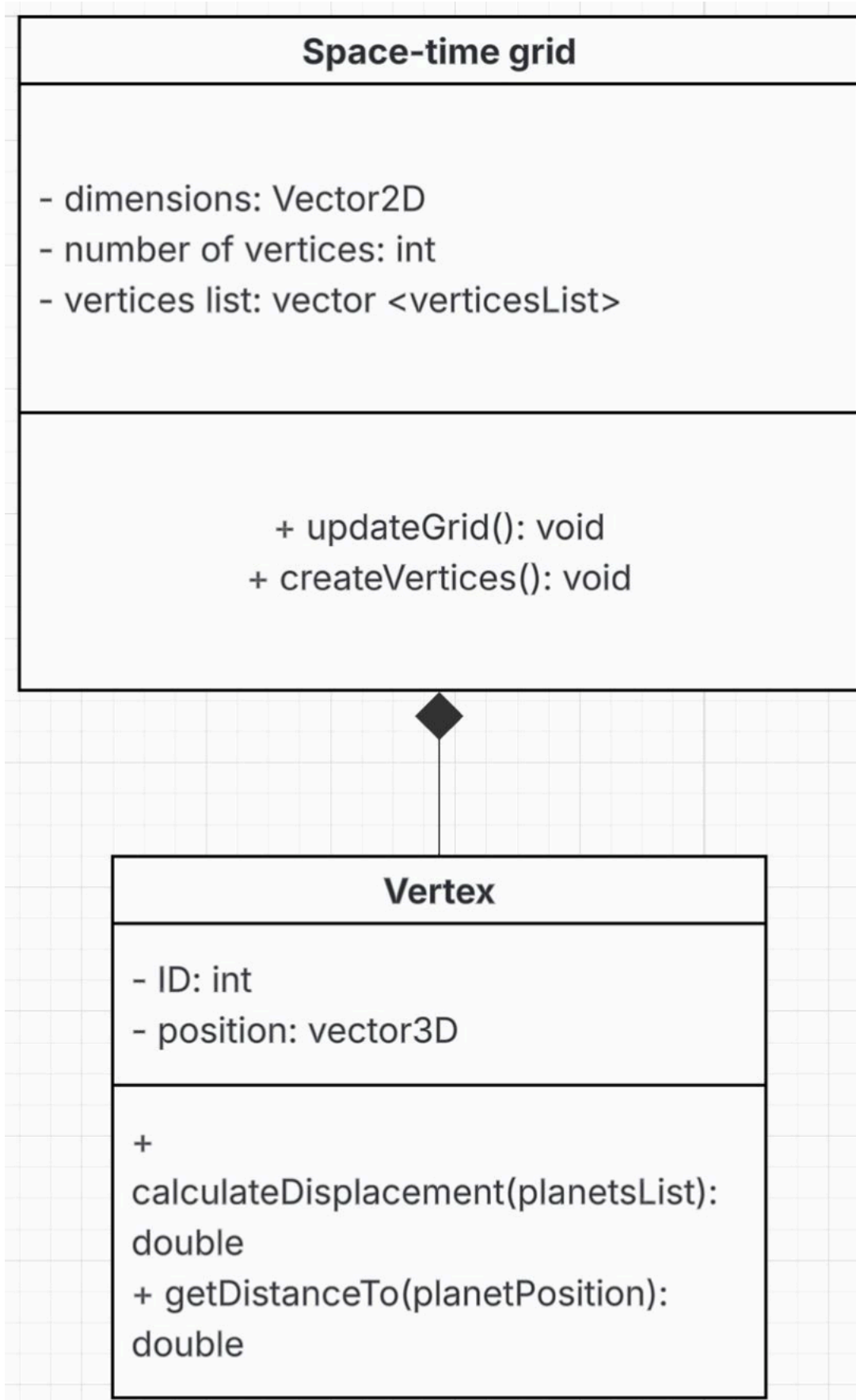
```

Each planet has a composite relationship with its label. Each planet only has a single label and a label cannot exist without the planet. The label has the attributes display and position. The display is stored as a string and contains the name of the planet which it should show.

The attribute position stores the position of the label as a 2D vector, which will be used to render it in the simulation.

updatePosition(): The label class has this method to update its positions every time step. The function will take the position of the planet it is associated with as an input and use this to update the position of the label. As mentioned in the pseudo code above, I may choose to offset the position of the label from the planet, as I don't want the label obscuring the planet and negatively affecting the user's experience.

Space-time grid



The SpaceTimeGrid class holds the array of vertices that will be involved in the space-time grid. The class will also be responsible for updating the position of each vertex every time-step.

```
1  class SpaceTimeGrid
2      private dimensions
3      private numberOfVertices
4      private verticesList
5
6      public procedure new(givenDimensions, givenNumberOfVertices)
7          dimensions = givenDimensions
8          numberOfVertices = givenNumberOfVertices
9          verticesList = createVertices()
10     end procedure
11
12     public createVertices()
13         for x = min to max
14             for y = min to max
15                 verticesList.add(new Vertex(x,y))
16             next y
17         next x
18     end procedure
19
20     public procedure updateGrid()
21         for vertexID in verticesList
22             verticesList[vertexID].calculateDisplacement
23     end procedure
```

Attributes: The attribute **dimensions**, will hold the dimensions of the space-time grid as a 2D vector. This will be a constant and will not change.

Similarly, the attribute **numberOfVertices** will not change. It holds the total number of vertices in the grid.

The attribute, **vertexList** holds every vertex, all of which will be instances of the vertex class shown below. In order to plot the grid, the front-end rendering will be able to iterate through the array of vertices and plot each one. This will occur every time step.

new(): The constructor method takes the dimensions of the number of vertices and uses it to instantiate the space-time grid. I don't create these as set values because I want this to be easy to edit if I need to.

createVertices(): In this method, I will use two loops that loop through the x and y direction creating the vertices of the grid and adding them to the list of vertices.

updateGrid(): In this method, I will use the vertex class's **calculateDisplacement** method to update the entire list of vertices.

Vertices

The vertex class will be used to instantiate vertices from. Each vertex will be stored in the SpaceTimeGrid array.

```
class Vertex
  private ID
  private position

  public procedure new(givenID, givenPosition)
    ID = givenID
    position = givenPosition
  end procedure

  public procedure getDistanceTo(planetPosition)
    distance = -1
    distance = sqrt((position.x - planetPosition.x)^2 + (position.y - planetPosition.y)^2)
    return distance
  end procedure

  public procedure calculateDisplacement(planetList)
    displacement = 0
    for planet in planetList
      displacement += -k * (mass of planet) / (distance to planet)^2
    next planet
    position.z = displacement
  end procedure
```

Attributes: Each vertex will need the attributes **ID** and **position**. The ID is an identifier for each vertex. The position of each vertex will be a 3D coordinate that can be used to identify its position in the grid.

The vertex will only change its position in one direction - up and down. I can therefore store the position of the vertex as a 3D coordinate and edit the z coordinate when I need to. The displacement is how far from its starting position that it should be rendered to give the effect that gravity is acting on it. This value will be used by the front-end to render the grid.

new(): The constructor method takes all the necessary parameters and uses it to instantiate a new vertex.

getDistanceTo(): This method will take a planet as a parameter and use the equation: $\text{distance} = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$ to find the distance to the given planet. This will be used in the next method to calculate the displacement.

calculateDisplacement(): This method will calculate the displacement of each vertex. This will use equations based on the proximity of planets and their masses.

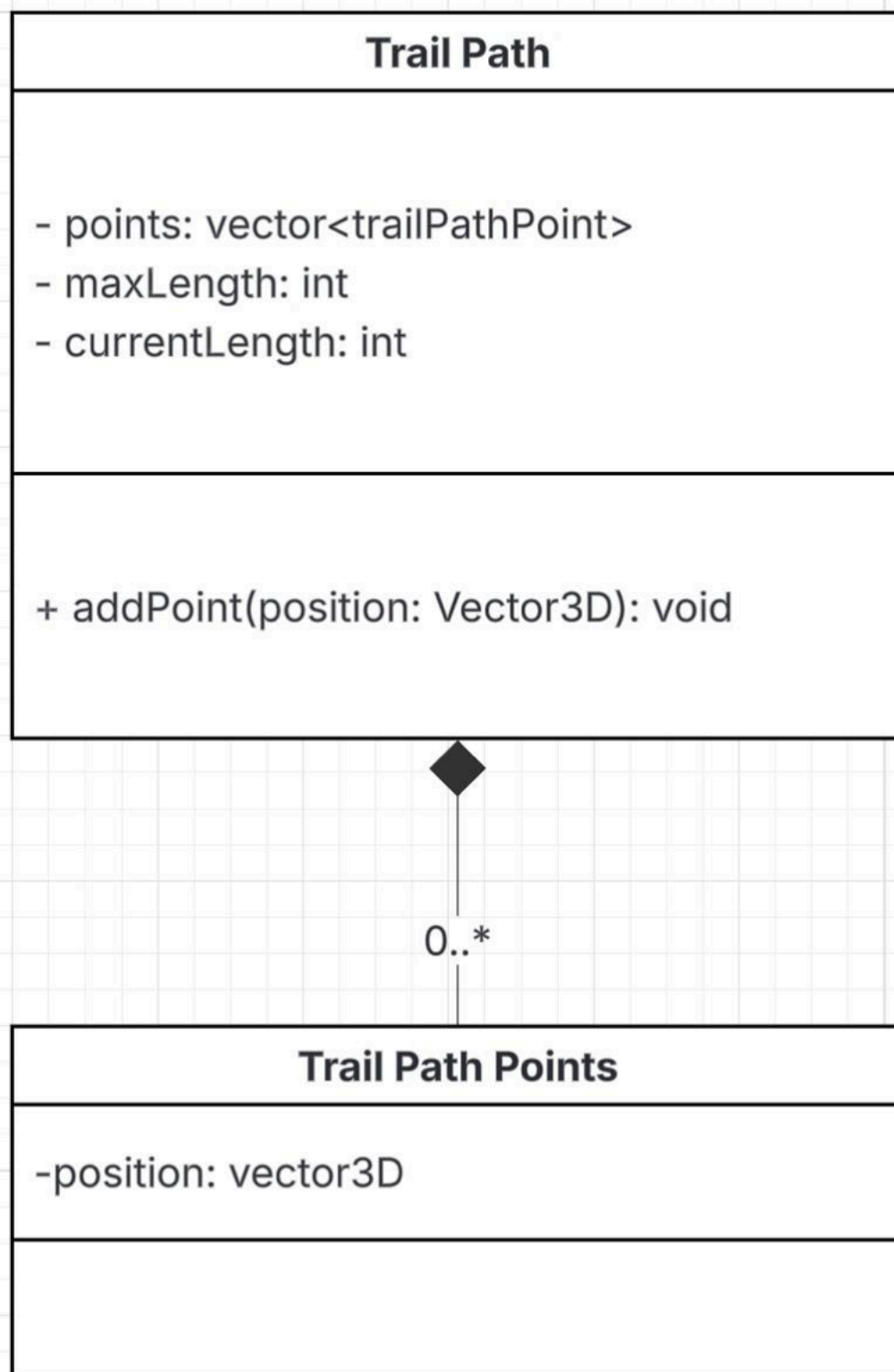
$$\text{Displacement} = -k * (\text{mass} / \text{distance}^2)$$

Where:

- k is a constant which I will calibrate in development
- mass is the mass of the current planet being considered
- distance is the distance to the current planet being considered

I will use this equation to calculate the displacement for each planet when considering a single vertex. I will add these displacements together to the variable **totalDisplacement**, which will feed into the vertex class as an attribute.

Trail Path



The trail path class will hold all the points in the trail path. The object created from this class will be used in rendering to create the trail paths of planets.

```
class TrailPath
    private points
    private maxLength
    private currentLength

    public procedure new(givenPoints)
        points = givenPoint
    end procedure

    public procedure addPoint(position)
        if currentLength < maxLength
            points[lastPoint] = new TrailPathPoint(position)
        else
            points[firstPoint].delete
            points[lastPoint] = new TrailPathPoint(position)
        endif
    end procedure
end class
```

Attributes: The trail path must have the attributes points, maxLength and currentLength. The points attribute is a vector which stores all the trail path points in it. This can be used by the rendering engine to plot the line as it can iterate through each point and plot a line from it to the adjacent point.

The maxLength attribute will be an integer value. This value will be constant and decided during the development stage of the project. I don't want this value to be too large as this might disrupt how easy the simulation is to observe. However, if this value is too small, this stops the user seeing where the planet was several time steps previously.

The currentLength attribute will store how many points the trail path has in at a given time-step. This can be compared with the maxLength value to limit the number of points that are rendered. When the maxLength value is reached, the last point in the points vector will be deleted and therefore not plotted.

addPoint(): Will use the current position of the planet to create a new point. This will use the **new()** method from the TrailPathPoint class to instantiate the point. This function will then add the point to the vector.

This function will maintain a fixed-length trail by discarding the oldest point before adding the new one.

Trail Path Points

```
class TrailPathPoints
  private position

  public procedure new(givenPosition)
    position = givenPosition
  end procedure
end class
```

The trail path points class will be used by the previous class to create points which can be added to the vector/list.

Attributes: Each point will have a position attribute.

The position attribute will be stored as a vector. It will tell the rendering algorithm where the line should start. The end point of this line will be the next point.

new(): The constructor method takes all the necessary parameters and uses it to instantiate a new point.

Design of the Main Loop Code

After creating all my classes, I can then move onto my main loop code. I will use the classes I've created as parent classes from which I can instantiate objects when I need them.

The simulation will start with the drawing of the space-time grid. The main loop code will therefore start by using the space-time grid class to create a single grid. This grid will be the only one instantiated from this class in each simulation. Creating this grid will involve creating many vertices which will be included in the space-time grid using a list or something similar.

After these objects have been instantiated, the main loop code can then use the rendering functions within them. For example, **Space_time_grid_1.draw**. This will render the space-time grid in its original position, before any planets are added. This allows the user to see what it will look like before any effect of gravity. Bare in mind I didn't include draw methods within my class definition above because of lack of pseudocode commands for this.

After this, the user will add planets using the display provided to them. This will be inputted into the main loop code. Using this data, the main loop code will instantiate planets as input by the user. When the user presses the add planet button, the main loop code will call the constructor method, creating a new planet with the attributes provided.

The next step is to add this planet to the container in which I will store all my planets. To do this I'm going to use a **std::vector<planet>** data type in which I push all the planets after they are created. This will be a very important part of the iterated code which is run every time step.

Every Time Step

The algorithm will loop through the **std::vector<planet>** list. For each planet in the list, a single method will be used that calls all the necessary methods (e.g: **updateVelocity**, **updatePosition**). This will update the position of each planet.

This is what this process might look like in pseudo code:

```
1   for planet in planetsList
2       planetList[planet].calculateResultantForce(planetsList)
3       planetList[planet].updateVelocity()
4       planetList[planet].updatePosition()
5   next planet
```

Other methods that perform less crucial aspects of the simulator will then be called. An example of this is calling a method to update the trail path by removing the last point and adding a new point at the current position of the planet.

After the calculations are complete and all variables have been updated, the algorithm will then move onto the drawing functions and methods. This will use various OpenGL functions to plot shapes in the window and display this to the user. Every time step, the position of the planets, trail path points and space-time grid on the screen will be updated if needed. If they do not need to be updated, their position will remain unchanged.

This main loop of code will run every time step. I'm hoping that my hardware at home is powerful enough for this to be a smooth process that allows the simulation to run at 24 frames per second. This will require looping the code 24 times per second.

User Interface

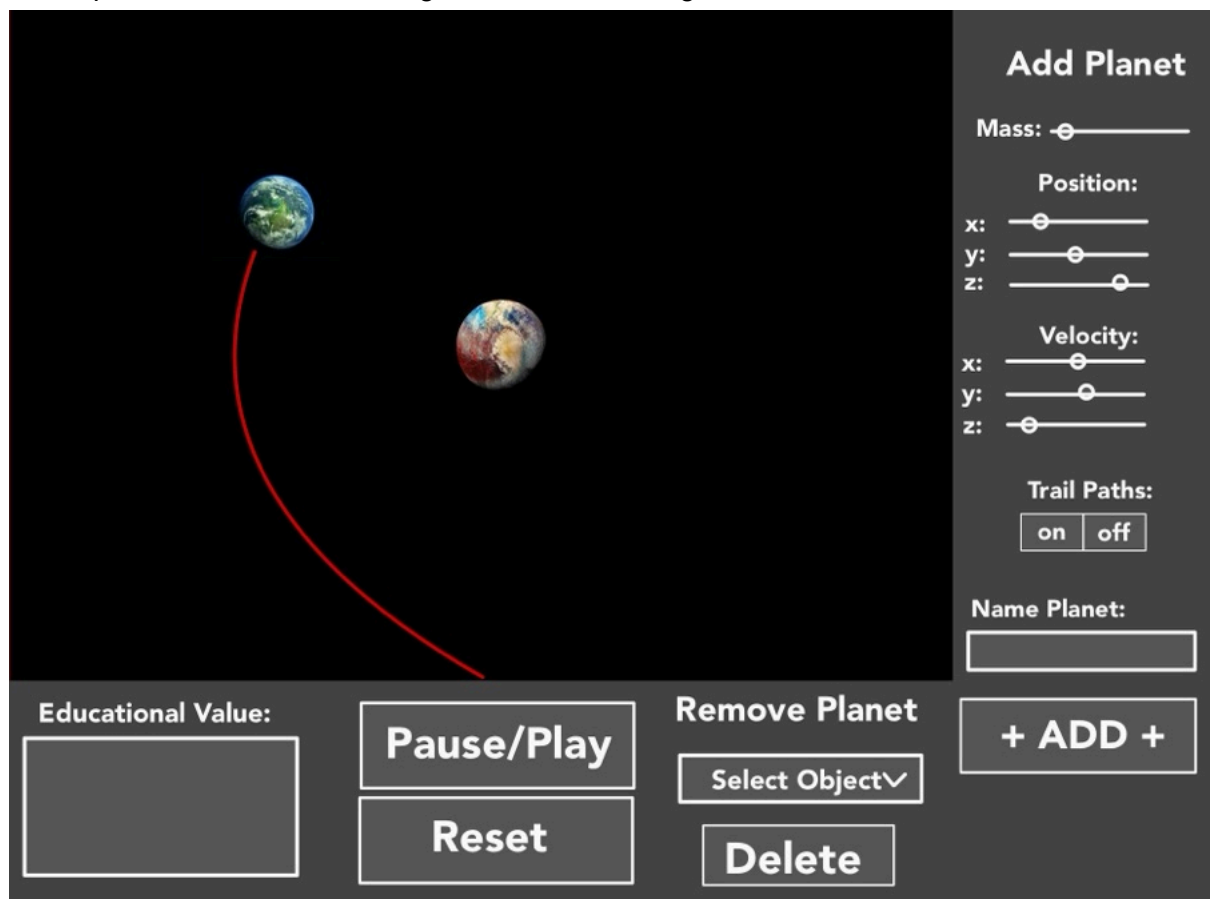
There were several ways I could have laid out the simulator. Each one would have a different user experience.

Firstly, I had to decide how I would lay out the simulation window, which is a crucial part of the simulation as it would allow the user to view it. I could either have the user enter all the

planets they wanted to and then play the simulation in full screen or have part of the screen covered by the simulation window and part of the screen taken up by controls. The benefit of using the entire screen for the simulation is that it is easier to see for the user. However, this reduces the interactiveness of the simulation as users cannot edit the simulation as it is running. This was not something I was willing to sacrifice. I therefore decided it was best if the simulator has a window in part of the screen that would run the simulation and controls outside this window which allow the user to interact with it.

The first one I considered was a more text-based approach which would involve the user entering the attributes of the planets using their keyboard. While this would allow the user to pick precise values for the planet's position, velocity and mass, it isn't necessarily user friendly for users who don't want to pick precise values. As this simulator is for students, who most likely only have a basic understanding of Astrophysics, I don't think this is the right solution.

I therefore pivoted to thinking about sliders. These would allow the user to much more easily navigate the controls as well as ensuring the user doesn't have to think about the scale of their input. I created the following wireframe including these sliders.



Usability Features

Starting on the right hand side, I have the controls to add a mass. The sliders allow the user to alter the features of a mass before it enters the simulation. I have chosen to use sliders because this means that I can select the upper and lower bound for each attribute.

- This is important for mass because it will stop the user from creating masses which are so big that they take up the entire screen, or objects which are so small that they will struggle to be seen. Masses that are too small will also have no effect on the simulation.
- It's important for position because the simulation will take place in a set 3D space with limits in all directions. I will model the space as a cube, with equal width, height and depth. It is therefore intuitive to use sliders which vary the distance I choose for the dimension of the cube, starting from 0.
- Similarly, varying velocity by changing the x,y and z components should be intuitive for a user, especially if they are doing the same process for the position. Using sliders means that the user cannot input values that are too large. This would firstly be unrealistic and would cross the 3D space in such a short amount of time that the user would be unable to observe it.

Below the sliders is a toggle that allows the user to select whether or not they want this planet to have a trail path. I have chosen to use a toggle because it is easy for the user to navigate and it will be clear what they have picked. This also allows the user to easily change this before the planet has been added to the system.

Just below this is the feature that allows the user to name the planet that they add. This is necessary because users will then be able to select this name when removing planets. It will also allow the user to keep track of the planet during the simulation as each planet will be labelled.

The large 'Add Mass' button below these is intentionally obvious as this is easy for the user. They can change the attributes of the mass, and press the 'Add Mass' button to add it to the simulation.

In the bottom right hand corner of the wireframe is the remove planet feature. This involves a drop-down menu, which will display the names of each planet. The user will be able to select the planet they wish to remove and once they have done so, they will use the 'delete' button to remove the planet from the simulation. The delete planet feature is important because it will allow the user to interact and edit the simulation as it is running.

In the bottom left the educational prompts will be displayed. When there aren't any relevant prompts, or when all the prompts have been displayed, this will be a blank box with nothing in it. While this feature won't disappear completely when all the prompts have been displayed, it does not get in the way of the simulation and therefore users can ignore it should they wish.

The feature to the right of this is the pause/play button. This will be a simple button that is easily accessible to the user at all times. This will enable the user to quickly pause the simulation if there is something they wish to observe.

The reset button will remove all the planets from the simulation, as well as reverting the space-time grid to its original state and reset the time to 0.

I haven't visualised the camera controls on the wireframe, as the user will be able to click and drag on the window and use the scroll wheel when on the simulation to be able to zoom in and out.

The simulation window itself will contain all the planets that the user decides to add. I have added to it two planets, one of which has its trail path mapped. I have not decided to include the space-time in this mock-up because I don't know how exactly I will implement it.

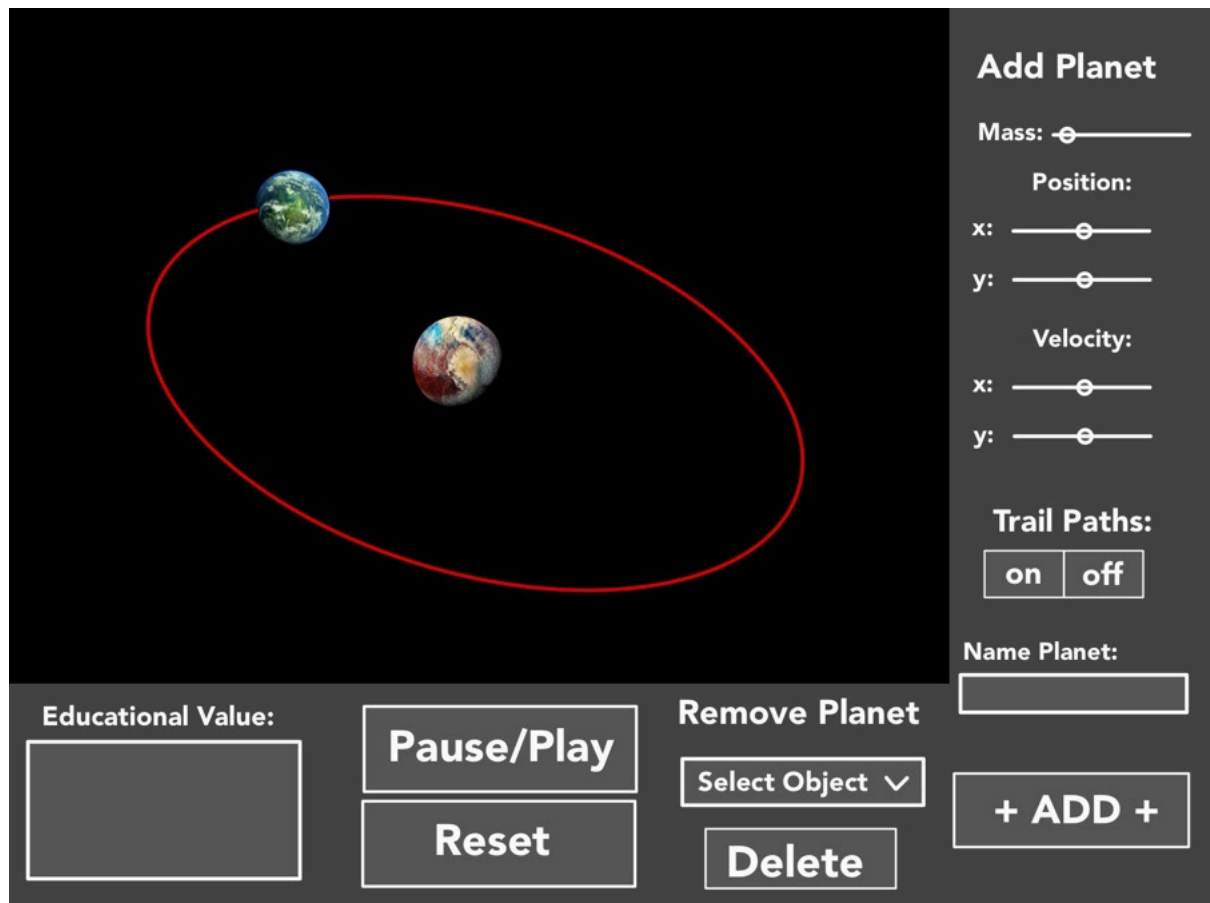
Stakeholder Feedback

Dr Mahoney: I showed this mock-up of the simulation to my physics teacher, one of my stakeholders. He brought to my attention that it might be more suitable to just run the simulation in 2D, while still rendering it in 3D. The 'camera' position would be above and off to the side, allowing the user to view the simulation in 3D.

The simulation will be rendered in 3D but all physical calculations will be constrained to two dimensions (x and y). The z direction will be used solely for camera movement and displacement of the space-time grid vertices. This simplifies my simulation while maintaining the 3D aspect of it and its 3D educational representation.

It will make it easier to implement the space-time grid, as the space-time grid is going to be 2D. If the planets cannot vary in one direction, this means they will constantly sit on the grid, making it very obvious to the user where space-time warps around other planets.

I therefore edited the wireframe to remove the z direction from both the position and velocity inputs. It then looked like this:



In this mock-up I have also slightly tweaked the layout of the features on the right hand side control panel. This is because taking out the sliders for the z direction created more space to work with. This wire-frame is just as user-friendly because it is easy to navigate and all the controls are easily accessible for a user.

Validation

Since I'm using sliders for the input of the data into the simulation, there is no need for me to run validation on these parts of the user input. I will however need to make sure that the sliders are correctly calibrated and allow for a good amount of customisation while also keeping the simulator simple and easy to use. To maintain ease of use, the sliders vary uniformly. This means that any change in position in the slider is directly proportional to the change in mass, velocity or position.

However, validation will be necessary when users have to input the name of the planet. The first thing I'll have to validate is the length of the name that the user has inputted. To stop users entering names which are too long, I will limit names to 20 characters. During the simulation, I don't want the planet's label to take up too much space and limiting the name to 20 characters should take care of this. I will also make sure that the user doesn't enter a string of 0 characters, as each planet will have to have a name. I won't have to validate anything else in this input, as special characters and numbers are allowed.

The remaining input methods are buttons and drop down menus which the user will navigate with their mouse. This will not require validation.

Testing to Inform Iterative Development

Validation Testing

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail
VT1	Normal	"Earth"	These check that the textbox correctly accepts and doesn't accept values.	Accepts value		SC1	
VT2	Normal	"Test 1"		Accepts value		SC1	
VT3	Erroneous	" " - no input from the user		Doesn't accept value, displays invalid message		SC1	
VT4	Erroneous	Planet 1: "Earth" Planet 2: "Earth"		Doesn't accept second value, displays invalid message		SC1	
VT5	Erroneous	"Twenty Characters\$\$\$\$"		Doesn't accept value, displays invalid message		SC1	
VT6	Boundary	"01234567890123456789" - a string of 20 characters		Accepts value		SC1	

I'm going to create my project in stages, starting with the basics and using prototypes which I will integrate into my code to build up complexity. Therefore, I have created tests associated with all the stages of my project.

Here is an outline of the stages, an explanation of what they will include and justification to why it is its own stage:

Stage	Explanation	Justification
1	In this stage I'll create a working window.	It will allow me to get used to opening a window and using GLFW and OpenGL before trying to manipulate the window.
2	In this stage I'll create the planet class.	I will use the planet class in the next stages so it is crucial that it is completed first before I try to implement the main loop code.
3	Prototype 1. My aim with the prototype is to create a fully-functioning 2D version of my simulator.	Once I have a working 2D simulation, I can add aspects to it to make it 3D.
4	In this stage I'm going to implement 3D aspects to make the simulation 3D.	Allows me to focus solely on implementing 3D graphics.
5	In this stage I'll add functionality - the trail paths and space-time grid.	Means that I can focus on these sections of the build without also being in the stage of developing other things.
6	In this stage I'll add the final usability features to my solution and carry out any necessary calibration.	Means that I can focus on creating an effective user interface without having to think about other parts of development.

I have different tests for each stage that I will use during development. These can be seen below.

Stage 1: Creating a Working Window

Test ID	Description	Expected result	Success criteria	Pass or Fail
TW1	Checks I can use an OpenGL	A window with a blank	SC14	

	windowing library to open a window.	background opens.		
TW2	I can use OpenGL libraries to render an image to the screen. I will use an orange triangle with a blue background to show this because a triangle is simple to code.	An orange triangle renders in the window with a blue background.	SC14	

Stage 2: The Planet Class

getRadius()

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail
TPR1:	Normal	45	This will test that the getRadius() method correctly calculates the radius of the planet given the mass.	90		SC18	
TPR2:	Normal	60		120		SC18	

distanceToPlanet()

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail
TPD1	Normal	Planet1	This will	5		SC15	

		position: 0,0 Planet2 position: 3,4	test that this method correctly calculates the distance between itself and another planet				
TPD2	Normal	Planet1 position: 3,5 Planet2 position: 7,9		5.65685		SC15	
TPD3	Boundary	Planet1 position: 0,0 Planet2 position: 0,0		0		SC15	

applyGravitationalForce()

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail
TPG1	Normal	Mass 1: 45 Mass 2: 35 Distance : 20	This will test that this method correctly calculates the force exerted on the planet. The distance cannot be 0 as this would lead to a calculation that cannot be solved.	5.25×10^{-9}		SC15	
TPG2	Normal	Mass 1: 0 Mass 2: 35 Distance : 20		0		SC15	

			For this reason, I will have to validate this which could involve deleting both planets if the distance is 0.				
--	--	--	---	--	--	--	--

calculateAngle()

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail
TPA1	Normal	Position of planet 1: (0,0) Position of planet 2: (4,3)	These tests ensure that the calculateAngle function correctly calculates the angle between two planets. If this is not correct, it could lead to wrong calculations in the calculateResultantForce method.	36.86... degrees		SC15	
TPA2	Normal	Position of planet 1: (0,0) Position of planet 2: (4,0)		0 degrees		SC15	
TPA3	Normal	Position of planet 1: (2,1) Position of planet 2: (4,3)		45 degrees		SC15	

calculateResultantForce()

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail
TPRF1	Normal	Angle: 45 Force: 30	Testing to check that the calculateResultantForce method correctly uses the angle and force to calculate the forces in each direction on a planet.	X direction: 21.21... Y direction: 21.21...		SC15	
TPRF1	Normal	Angle: 23 Force: 100		X direction: 39.07... Y direction: 92.05...		SC15	
TPRF3	Normal	Angle: 0 Force: 100		X direction: 0 Y direction: 100		SC15	

updateVelocity()

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail
TPV1	Normal	X Initial velocity: 0 Acceleration: 10 Y Initial velocity: 0 Acceleration: 10	These tests ensure that the velocity in each direction is calculated correctly. If it is not calculated correctly, this could lead to flaws in	X direction: 10 Y direction: 10		SC15	
TPV2	Normal	X Initial velocity: 13 Acceleration		X direction: 20 Y		SC15	

		tion: 7 Y Initial velocity: 7 Accelera tion: 5	the physics calculati ons later down the line and therefore wrong position of the planets on screen.	direction: 12			
TPV3	Normal	X Initial velocity: 10 Accelera tion: 0 Y Initial velocity: 10 Accelera tion: 0		X direction: 10 Y direction: 10		SC15	

updatePosition()

Test ID	Test Type	Test Data	Descripti on	Expecte d	Actual	Success Criteria	Pass/Fail
TPV1	Normal	X Initial Position: 0 Velocity: 10 Y Initial Position: 0 Velocity: 10	These tests ensure that the position of the planet is calculate d correctly. This is importan t because if the position is not calculate d correctly, it will be displaye d in the	X direction: 10 Y direction: 10		SC14 SC15	
TPV2	Normal	X Initial position: 13 Velocity: 7 Y Initial		X direction: 20 Y direction: 12		SC14 SC15	

		position: 7 Velocity: 5	wrong position on the screen and the simulator won't function correctly.				
TPV3	Normal	X Initial position: 10 Velocity: 0 Y Initial position: 10 Velocity: 0		X direction: 10 Y direction: 10		SC14 SC15	

Stage 3: Prototype 1

I aim to make **Prototype 1** a fully functional 2D version of my simulation with limited functionality. At this stage, there won't be any space-time grid because this requires the simulation to have a 3D aspect. Furthermore, there won't be a trail path option and it won't show the name of planets. This will use the planet class I've already created.

Prototype 2

Test ID	Description	Success criteria	Pass or Fail
PT1	The screen renders a circle	SC14	

Testing Rendering

These tests follow on from each other, so instead of carrying out every single test after a stage of development, I can carry out the first tests, see if these pass and then carry on testing if they pass.

Test ID	Description	Success criteria	Pass or Fail
PO1	The simulation successfully renders planets on the screen as circles.	SC14	

PO2	The simulation can successfully introduce velocity for planets in the simulation and render this as movement across the window.	SC16	
PO3	The simulation can successfully introduce acceleration for planets in the simulation and render this as increasing velocity for the planets on the screen.	SC16	
PO4	The simulation can move and accelerate two planets at once.	SC16	
PO5	The simulation successfully uses the planet class functions to give a visual representation of gravity between two planets in the window.	SC15	
PO6	The simulation allows the user to add planets.	SC6	
PO7	The simulation allows the user to customise planets.	SC1-5	
PO8	The simulation has a user interface that allows the user to change the initial position of a planet with sliders.	SC3	
PO9	The simulation has a user interface that allows the user to change the initial velocity of a planet with sliders.	SC4	
PO10	The simulation has a	SC2	

	user interface that allows the user to change the mass of a planet with sliders.		
PO11	The simulation has a user interface which adds the customised planet to the window with the press of a button.	SC6	
PO12	The simulation has a user interface with a text input box for the name of the planet.	SC1	

Web Tests

At this stage I will also attempt to run this on the web. Here are the tests I will use to decide whether or not this is successful:

Test ID	Description	Success Criteria	Expected	Actual	Pass/Fail
TWe1	Checks it runs in a browser, not as a native executable.	SC25	I can open a browser and access the program on a local host server.		
TWe2	Checks that my simulation draws frames within the browser canvas.	SC25	My simulation to run as expected on the web page.		
TWe3	Checks that user input is handled within the browser.	SC25	Any input into the browser behaves as it would in the native executable.		

Stage 4: Going 3D

Testing Rendering

Similarly to how I tested **Prototype 1** I will first do the visual tests that test its accuracy. This version will have the same limited functionality as **Prototype 1** but it will be in 3D instead of 2D. These tests will be almost identical to the 2D tests, but for the new updated version. These tests follow on from each other, so instead of carrying out every single test after a stage of development, I can carry out the first tests, see if these pass and then carry on testing if they pass.

Test ID	Description	Success criteria	Pass or Fail
MLRT1	The simulation successfully renders planets on the screen as spheres.	SC17	
MLRT2	The simulation can successfully introduce velocity for planets in the simulation and render this as movement across the window in 3D.	SC16 SC18	
MLRT3	The simulation can successfully introduce acceleration for planets in the simulation and render this as increasing velocity for the spheres on the screen.	SC16 SC18	
MLRT4	The simulation can move and accelerate two planets at once.	SC16	
MLRT5	The simulation successfully uses the planet class functions to give a visual 3D representation of gravity between two planets in the window.	SC15	
MLRT6	The simulation allows the user to add planets and renders	SC6	

	them as spheres.		
MLRT7	The simulation allows the user to customise planets.	SC1 SC2 SC3 SC4	
MLRT8	The simulation has a user interface that allows the user to change the initial position of a planet with sliders.	SC3	
MLRT9	The simulation has a user interface that allows the user to change the initial velocity of a planet with sliders.	SC4	
MLRT10	The simulation has a user interface that allows the user to change the mass of a planet with sliders.	SC2	
MLRT11	The simulation has a user interface which adds the customised planet to the window with the press of a button.	SC6	
MLRT12	The simulation has a user interface with a text input box for the name of the planet.	SC1	

Stage 5: Adding Functionality

Trail Path Class

Testing Accuracy

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail

TP1	Normal	Input these vertices: (0,1) (0,2) (0,3) (0,4) (0,5) (0,6) (1,0) (2,0) (3,0) (4,0) (5,0) (6,0)	Here I will input 12 values. As the maximum size of the trail path is 10, when I print the path, only the last 10 should be output.	(0,3,0) (0,4,0) (0,5,0) (0,6,0) (1,0,0) (2,0,0) (3,0,0) (4,0,0) (5,0,0) (6,0,0)		SC23	
TP2	Normal	(0,1) (0,2) (0,3) (0,4) (0,5)	I will input just 5 vertices. When I print the path, these 5 vertices should be output.	(0,1,0) (0,2,0) (0,3,0) (0,4,0) (0,5,0)		SC23	
TP3	Normal	(0,0) (0,0) (0,0) (0,0)	Testing with different data.	(0,0,0) (0,0,0) (0,0,0) (0,0,0)		SC23	

Testing the Rendering

Similarly to the other tests I've carried out, each of these follow on from the one before, meaning that if the first test fails, I will not carry on to the next and so on.

Test ID	Description	Success criteria	Pass or Fail
TPR1	The path of the planet can be rendered in the window successfully.	SC5	

TPR2	This works alongside the space-time grid successfully.	SC23	
TPR3	The path has a maximum length, and when it reaches this maximum length, the points at the end disappear.	SC24	
TPR4	The user can choose whether or not they want a trail path for each planet and this dictates whether or not the trail path is shown.	SC5	

Space-Time Grid Class

Testing Accuracy

createGrid()

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail
STG A1	Normal	Size = 800 Divisions = 25	I will get the method to output the position of each of the vertices.	I will look at the final 5 vertices. I'm expecting all the z values to be -3000 through the whole list. For the end vertices I expect all the x coordinates to be 10000, as this will be the largest x value. I'm expecting that the y coordinates will vary in steps of 800, as this is the step of the grid.		SC20	

UpdateGrid()

In order to test the **updateGrid()** method I'm going to add a planet to the **planets** list with the position (10000, 10000). I will then run the **UpdateGrid()** method and output the coordinates of all the space-time vertices in the list. As the vertices with the positions closest to (10000,10000) are output last, I can carry out my own calculations to determine the expected z displacement of the final 3 vertices. I will then compare these to the actual values that are outputted. The test passes if these values match.

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail
STG U1	Normal	Position of planet: (10000,10000) Mass of planet: 1000	This ensures that the displacement of the vertices is calculated correctly. If this is incorrect it could lead to an inaccurate depiction of space-time.	Vertex 1 coordinates: (10000, 10000, -240000) Vertex 2 coordinates: (10000, 9800, -5853.66) Vertex 3 coordinates: (10000, 9600, -1490.68)		SC21 SC22	

Testing Rendering

Test ID	Description	Success criteria	Pass or Fail
STGR1	The grid appears in the window successfully and with a clear grid shape.	SC20	
STGR2	The grid renders alongside the planets.	SC20	
STGR3	The grid renders alongside the trail paths.	SC20	
STGR4	The grid vertices change their displacement depending on the distance from massive planets.	SC21 SC22	

Stage 6: Usability Features

Label Class

Rendering Tests

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail
TL1	Normal	Left corner of my grid, coordinates (10000, -10000)	These tests are testing that the class correctly takes in the position it is given and updates the labels position accordingly, outputting this to the display.	Label renders on the left corner of the grid.		SC13	
TL2	Normal	Right corner of my grid, coordinates (-10000, 10000)		Label renders on the right corner of the grid.		SC13	
TL3	Normal	"Test 1"	These tests are testing that the class correctly stores and outputs the name of each planet.	"Test 1"		SC13	
TL4	Normal	"Mars"		"Mars"		SC13	

User Interface

Test ID	Description	Success criteria	Pass or Fail
TUI1	The user can select a	SC9	

	planet from a drop down menu and remove this from the simulation.		
TUI2	The user can use a button to pause the simulation, and with the same button resume the simulation.	SC7	
TUI3	The user can reset the simulation using a button on the interface.	SC8	
TUI4	Educational prompts are displayed to the user.	SC11	
TUI5	The educational prompts change depending on what is in the simulation.	SC12	
TUI6	The user can navigate the worldspace using the camera controls.	SC10	

Post Development Tests

Tests PD1-25

Test ID	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail
PD1	Input planet names into textbox, press 'Add Planet'	Checks that the name textbox correctly gives the planet names.	The inputted name is output in the label.		SC1, SC13, SC6	

PD2	Add one planet of mass 400 and one planet of mass 1200.	These tests check that the mass slider works correctly.	Noticeable difference in the size of these planets.		SC2, SC18, SC6	
PD3	Add a planet of mass 400 and a planet of mass 1200. Output the mass of each planet on the label.		The label's display 400 and 1200 for the mass respectively.		SC2, SC6	
PD4	Add a planet at the position (456,7890), output this to the label.	These tests check that the position sliders work correctly.	The label outputs (456, 7890)		SC3, SC6	
PD5	Add a planet at position (10000, 10000).		The planet is rendered in the --- corner of the grid.		SC3, SC14, SC6	
PD6	Add a planet with velocity (123, -467), output this to the label.	These tests check the velocity sliders work correctly.	The label outputs (123, -467).		SC4, SC6	
PD7	Add a planet with velocity (0,500)		The planet moves in the y direction with a fast speed.		SC4, SC16, SC6	
PD8	Tick the trail path checkbox	These tests check the trail path checkbox	A planet is rendered with a trail path.		SC5, SC6, SC23	

	and add the planet.	works correctly.				
PD9	Don't tick the trail path checkbox and add the planet.		A planet is rendered without a trail path.		SC5, SC6, SC23	
PD10	Customise the planet and press the 'Add Planet'	This test checks if the add planet button works. This will also be checked in all previous tests.	The planet is added to the system.		SC6	
PD11	Have several masses in the simulation moving around and press the pause button.	These tests check that the pause/play button works.	All movement in the simulation pauses.		SC7	
PD12	After the previous test, press the resume button.		All movement in the simulation resumes.		SC7	
PD13	Have several masses in the simulation and press the reset button.	This test checks whether the reset button works correctly.	All planets are removed and the space-time grid returns to its original state.		SC8	
PD14	Have several named planets in	These tests check that the delete drop-down menu and	The selected planet is removed.		SC9	

	the simulation. Use the drop down menu to select a planet and the button to remove it.	associated button work.				
PD15	Have a single mass in the simulation. Use the drop down menu to select a planet and the button to remove it.		The selected planet is removed and the space-time grid returns to its original state.		SC9	
PD16	Use the camera position sliders to navigate around the environment.	This test checks that the camera position sliders work correctly.	This test passes if these sliders can be used to view systems and the space-time grid in more detail. I will ask stakeholders to rank the effectiveness of the camera controls. If it ranked a 4 or better, this test passes.		SC10	
PD17	Educational messages	These tests check that the	An educational		SC11	

	are displayed to the user via the user interface.	educational messages work correctly.	message output to the user.			
PD18	Educational messages change depending on the number of masses in the simulation.		The message changes if there are 0, 1 or more masses in the simulation.		SC12	
PD19	Add two planets to the simulation.	This test checks that planets experience gravity due to other planets in the simulation.	The planets move towards each, visually representing gravity.		SC15	
PD20	Add a frame rate measurer to my user interface.	This test checks that my simulation gets a suitable frame-rate.	A frame rate of at least 30FPS.		SC19	
PD21	Reset the simulation.	These tests check the space-time grid functions correctly.	A flat grid.		SC20	
PD22	Add planets to the simulation.		The grid visibly and clearly warps around planets which are added to the simulation.		SC21	
PD23	Add a planet with a trail and velocity to	These tests check that the trail paths are rendered correctly.	The trail paths are rendered to the screen.		SC23	

	the simulation.					
PD24	Add a planet with a trail and velocity to the simulation.		When the trail path reaches a certain length, this length is maintained and it is deleted from the back.		SC24	

Test of Gravitational Accuracy - Test PDA1

In order to test the accuracy of the gravity calculation, I'm going to add two planets to the system. Firstly, I will calculate the force I expect each planet to experience. I will then modify the program to output the force that each planet experiences from the other one. I will compare the expected values with the actual values. If they match, this test will pass and if they don't match it will fail.

I want to calculate:

- The force between the planets
- The distance between the planets
- The x and y components of the velocities of each planet

I choose to compare these values because if these 3 values all matched up I could be very confident that all the physics calculations were being carried out correctly.

In order to make this process easier for myself, I will use a chat bot to carry out these calculations and output them in a table which I can use to compare to the values from my simulation.

Development

Stage 1: Creating a Working Window

The first stage in my development is to create a working window using OpenGL. I did this using GLFW which is an open-source library for c and c++ that provides a simple API to create windows and manage OpenGL.

I've decided to use GLFW for several reasons. Firstly, GLFW is a cross platform library, allowing my code to be run consistently on all operating systems. This lets me focus on OpenGL rendering and physics rather than system details.

In addition to this, GLFW handles user input and provides callbacks for these events, which integrate neatly with an object-oriented program. This allows for easy keyboard and mouse input handling, which is essential for interacting with my gravity simulator.

Build

To get this working, I downloaded this using homebrew via the terminal as I am using MacOS. I then used the pre-written code on the GLFW website to initialise a window:

```
C/C++
#include <GLFW/glfw3.h>

int main(void)
{
    GLFWwindow* window;

    /* Initialize the library */
    if (!glfwInit())
        return -1;

    /* Create a windowed mode window and its OpenGL context */
    window = glfwCreateWindow(640, 480, "Hello World", NULL, NULL);
    if (!window)
    {
        glfwTerminate();
        return -1;
    }

    /* Make the window's context current */
    glfwMakeContextCurrent(window);

    /* Loop until the user closes the window */
    while (!glfwWindowShouldClose(window))
```

```

{
    /* Render here */
    glClear(GL_COLOR_BUFFER_BIT);

    /* Swap front and back buffers */
    glfwSwapBuffers(window);

    /* Poll for and process events */
    glfwPollEvents();
}

glfwTerminate();
return 0;
}

```

I then linked the VScode file to the GLFW documentation I had downloaded by using the following compiling command:

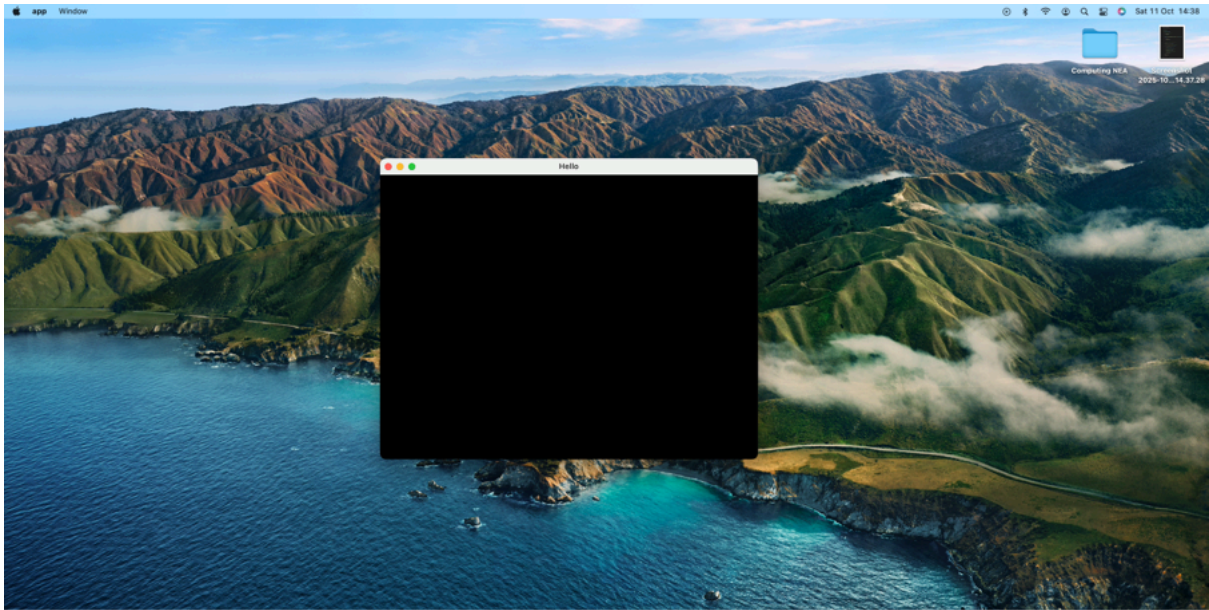
g++ -arch x86_64 main.cpp -o app -I/usr/local/include -L/usr/local/lib -lglfw -framework Cocoa -framework OpenGL -framework IOKit.

I then ran the code using **./app**, which successfully created the window.

Testing

Test ID	Description	Expected result	Success criteria	Pass or Fail	EVIDENCE
TW1	Checks I can use an OpenGL windowing library to open a window.	A window with a blank background opens.	SC14	Pass - the expected result (a blank window) was output to the screen.	

Evidence in more detail:



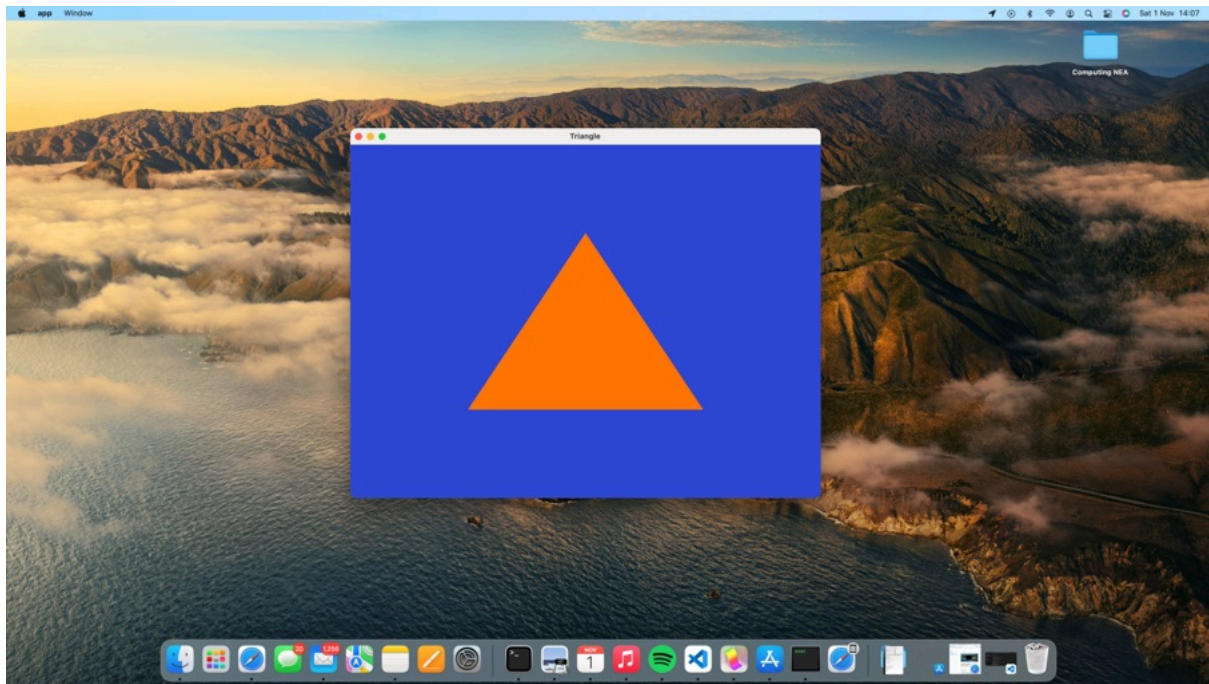
The GLAD problem:

The next step in setting up the simulation window was including libraries that would allow me to manipulate the window I'd just created. From initial research I concluded that I would use GLAD, a library that allows the user to access modern OpenGL functions.

In order to include this in my c++ files, I had to download the relevant software from the website and arrange this into my project directory in a way that would allow the compiler to access the necessary header files. After following the recommendations of this website, <https://rpxomi.github.io>, I tried compiling my file but the include command at the top of my program showed an error. To try and fix this, I asked GitHub copilot this prompt: 'How do I fix a include error in c++?'. I went on to follow the advice of GitHub copilot, giving it further details such as my operating system and CPU details. However, none of the solutions it gave me managed to solve my problem.

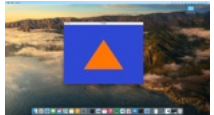
I therefore looked for another solution that didn't involve GLAD and during my research I stumbled upon a GitHub repository called OpenGLOnMac (<https://github.com/zamirmf/OpenGLOnMac>). This repository had the include command: **#include <OpenGL/gl3.h>**, which after research I found out was a header file available to macOS users. MacOS provides OpenGL headers as part of development tools and this requires no external downloads. This was therefore the solution I chose to fix the problem of not being able to include GLAD into my c++ files in order to use OpenGL.

After choosing this, I created this window as proof of concept, I found the code for creating a triangle in a youtube tutorial (<https://youtu.be/45MlykWJ-C4?si=el0wR9vgdT-YQ1Gz>) as at this stage I am not focusing on the actual OpenGL code, but rather being able to link libraries in order to open and manipulate windows:



I used this command to compile: **clang++ main.cpp -std=c++11 -I/usr/local/include -L/usr/local/lib -lglfw -framework OpenGL -framework Cocoa -framework IOKit -framework CoreVideo -Wno-deprecated-declarations -o app**

I could then carry out the following tests for stage 1 of my development:

Test ID	Description	Expected result	Success criteria	Pass or Fail	EVIDENCE
TW2	I can use OpenGL libraries to render an image to the screen. I will use an orange triangle with a blue background to show this because a triangle is simple to code.	An orange triangle renders in the window with a blue background.	SC14	Pass - the actual result was how I expected it.	

Review

After stage 1 I can successfully open and modify a window using my chosen languages. This allows me to move onto stage 2 of my development and gives me a platform to build on when I started using the planet class to render circles and spheres to the screen.

The purpose of this stage was to make sure I could open and manipulate this window, I have succeeded in this.

Stage 2: The Planet Class

Planet Class Version 1

Build

I started by using my design to create an initial class for planets, which contained the attributes and methods which I outlined in the design stage of the project. I also added the method **calculateAngle()** which I need in the method **calculateResultantForce()** in order to resolve the forces in the x direction and y direction, which makes it easier to add these forces together and calculate a resultant force.

Here's the code for the planet class. I have used structs to define the 2D variables (position, velocity, acceleration, resultant force). I have chosen to do this because it is simpler to work with these than other options.

I'm using the library `cmath` for trigonometric functions, powers and square root functions.

```
C/C++
#include <iostream>
#include <cmath>

struct Position{
    double x;
    double y;
};

struct Velocity{
    double x;
    double y;
};

struct Acceleration{
    double x;
    double y;
};
```

```

struct ResultantForce{
    double x;
    double y;
};

class Planet{
public:
    std::string name;
    double mass;
    Position position;
    Velocity velocity;
    Acceleration acceleration;
    bool trailPath;
    double radius;

    Planet(std::string name, double mass, Position position, Velocity
velocity, Acceleration acceleration, bool trailPath){
        this->name = name;
        this->mass = mass;
        this->position = position;
        this->velocity = velocity;
        this->acceleration = acceleration;
        this->trailPath = trailPath;
    }

    int getRadius(int mass){
        radius = mass * 1; // as mentioned in design, a constant, k,
will decide the value for the radius. For now I am using 1 for simplicity.
//feel like this could be in the constructor, for version 2
maybe
        return 0;
    }

    double distanceToPlanet(Position planet2){
        double distance;
        distance = sqrt(pow((planet2.x-position.x),2) +
pow((planet2.y-position.y),2));
        return distance;
        // this function calculates the distance to a given planet. To
be used in the force calculations
    }

    double applyGravitationalForce(double mass, double mass2, double
distance){
        double Force;
        Force = (6.67 * pow(10,-11)) * ((mass*mass2) / pow(distance,2));
        return Force;
    }
}

```

```

    }

    double calculateAngle(Position position, Position planet2){
        double Angle;
        Angle = atan(abs(position.x - planet2.x)/abs(position.y -
planet2.y));
        return Angle;
    }

    ResultantForce calculateResultantForce(double Force, double angle){
        ResultantForce resultantForce;
        resultantForce.x = Force * sin(angle);
        resultantForce.y = Force * cos(angle);
        return resultantForce;
    }

};

```

Testing

getRadius()

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail
TPR1:	Normal	45	This will test that the getRadius() method correctly calculates the radius of the planet given the mass.	90	90	SC18	Pass
TPR2:	Normal	60		120	120	SC18	Pass

distanceToPlanet()

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail
TPD1	Normal	Planet1 position: 0,0 Planet2 position: 3,4	This will test that this method correctly calculates the distance between itself and another planet	5	5	SC15	Pass
TPD2	Normal	Planet1 position: 3,5 Planet2 position: 7,9		5.65685	5.65685	SC15	Pass
TPD3	Boundary	Planet1 position: 0,0 Planet2 position: 0,0		0	0	SC15	Pass

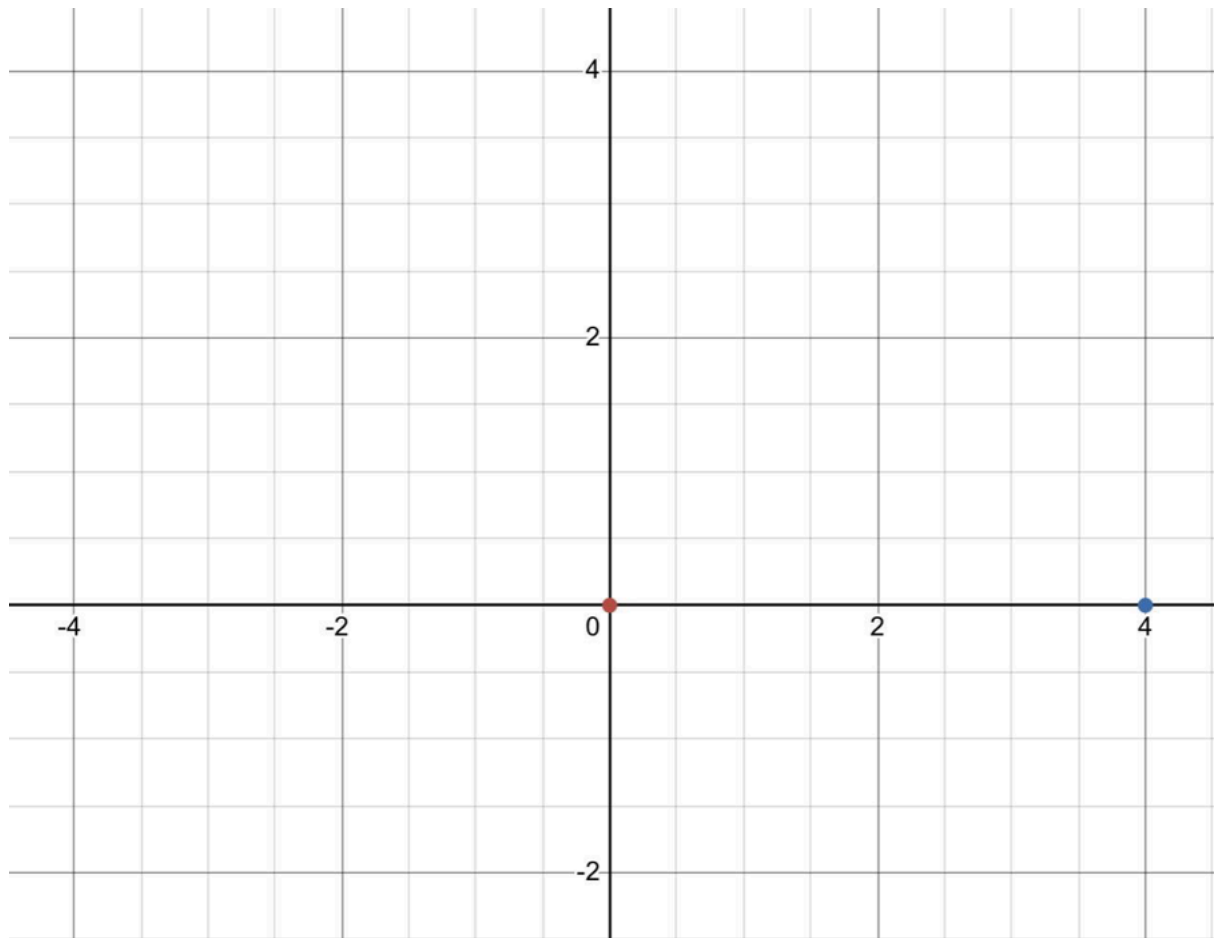
applyGravitationalForce()

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail
TPG1	Normal	Mass 1: 45 Mass 2: 35 Distance : 20	This will test that this method correctly calculates the force exerted on the planet.	2.62×10^{-10}	$2.62631 e^{-10}$	SC15	Pass
TPG2	Normal	Mass 1: 0 Mass 2: 35 Distance : 20		0	0	SC15	Pass

calculateAngle()

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail
TPA1	Normal	Position of planet 1: (0,0) Position of planet 2: (4,3)	These tests will ensure that the calculateAngle() method correctly uses the arctan function to calculate the angle between two planets.	36.86... degrees / 0.927...rad	0.927297	SC15	Pass
TPA2	Normal	Position of planet 1: (0,0) Position of planet 2: (4,0)		0 degrees / 0 rad	1.52707	SC15	Fail
TPA3	Normal	Position of planet 1: (2,1) Position of planet 2: (4,3)		45 degrees / 0.78539 ...	0.785398	SC15	Pass

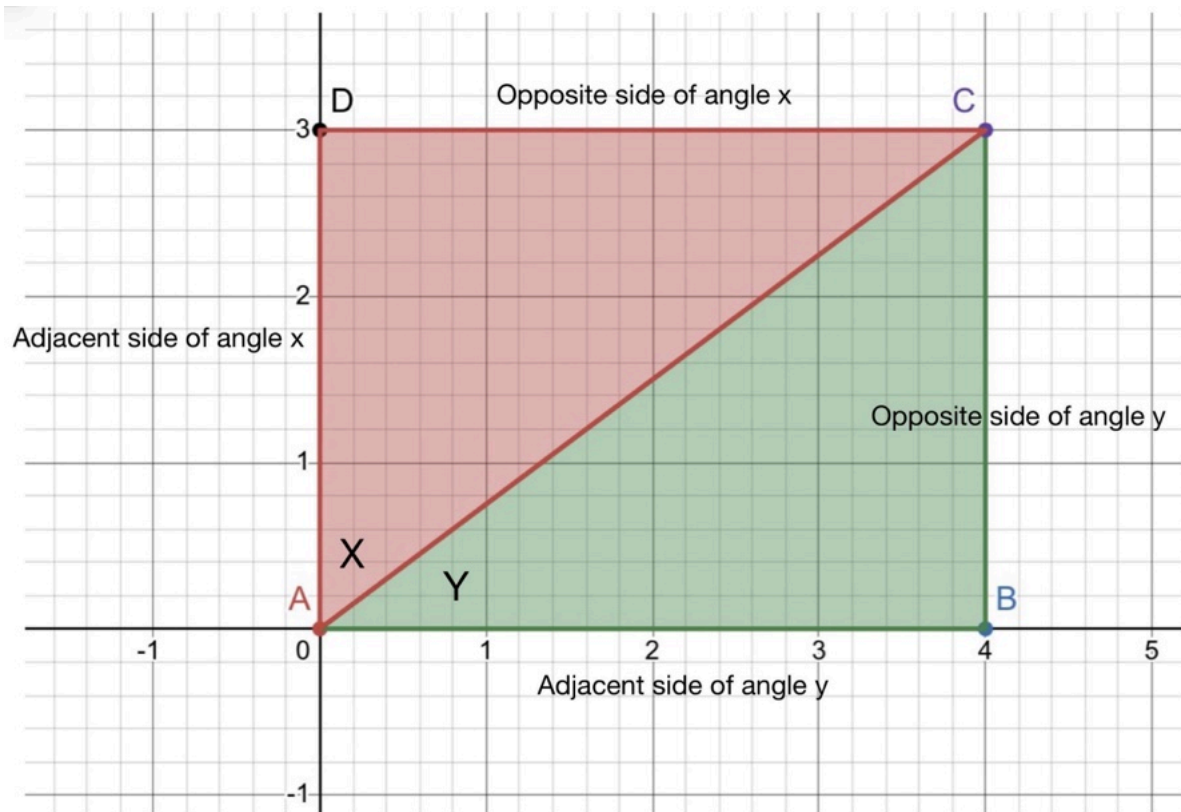
The TPA2 test here failed which was unexpected at first because I was thinking about it as a graph that looks like this:



In the above graph the angle between the two points is 0 from the x-axis, but 90 from the y-axis, which is what we are measuring from, since we are using the difference in x coordinates as the opposite side of the triangle rather than the adjacent side.

This didn't happen for 45 degrees ($\pi/4$ radians) because X and Y in this scenario because $90 - 45 = 45$.

Here is a graph to demonstrate:



As you can see, the coordinate you choose to be the opposite and adjacent sides affect the size of the angle, as angle X and Y are different, but the only difference between them is which side is the close side and which side is the far side.

I therefore updated my test plan to this:

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail
TPA1	Normal	Position of planet 1: (0,0) Position of planet 2: (3,4)	These tests will ensure that the calculateAngle() method correctly uses the arctan function to calculate the angle between two planets.	0.927295 rad	0.927295	SC15	Pass
TPA2	Normal	Position of planet 1: (0,0) Position of planet 2: (0,4)		1.52707 rad	1.52707 ...	SC15	Pass
TPA3	Normal	Position of planet 1: (2,1) Position of planet 2: (4,3)		0.785398 rad	0.785398	SC15	Pass

		2: (4,3)					
--	--	-----------	--	--	--	--	--

Another thing I learned from this testing is that the cmath trigonometric functions work in radians rather than degrees. I have therefore edited any further testing to use radians instead of degrees.

calculateResultantForce()

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail
TPRF1	Normal	Angle: $\pi/4$ rad Force: 30	Testing to check that the calculateResultantForce method correctly uses the angle and force to calculate the forces in each direction on a planet.	X direction: 21.21... Y direction: 21.21...	X: 21.2132 Y: 21.2132	SC15	Pass
TPRF1	Normal	Angle: 0.40142 ... rad Force: 100		X direction: 92.05... Y direction: 39.07...	X: 92.0507 Y: 39.0726	SC15	Pass
TPRF3	Normal	Angle: 0 Force: 100		X direction: 100 Y direction: 0	X: 100 Y: 0	SC15	Pass

updateVelocity() and updatePosition()

I was unable to test these two methods because when I created the initial class I didn't include the initial position and initial velocity of the planet within the parameters of these methods. This is something I fixed in post testing improvements.

Review

After the first test this is a list of improvements and edits I have to make to the planet class:

1. Add a position input into the parameters of the **updateVelocity()** and **updatePosition()** methods.
2. I need to add a **calculateAcceleration()** method within the class which will take the resultant force 2D vector and use this in the equation:

$$\text{Resultant Force} = \text{Mass} * \text{Acceleration}$$

in order to calculate the acceleration in each direction.

After the first test this is a list of general improvements I can make:

1. Use a single struct with 2 dimensions instead of separate structs for each 2D vector that I have. I'm calling this struct `vec2`. This is the code for it:

```
C/C++
struct vec2{
    double x;
    double y;
};
```

Plant Class Version 2

Build

Firstly added the struct **vec2** which will hold all my 2D vectors in one, rather than having separate structs for each variable. This makes my code a lot more maintainable and easy to code. I then went through the class, updating data types to **vec2** where this was necessary.

In addition to this I added the additional methods that I would need (**updateVelocity()**, **updatePosition()** and **calculateAcceleration()**).

```
C/C++
#include <iostream>
#include <cmath>

/*=====Error Message Definition=====*/

std::string Error_Message = "An error has occurred";

/*=====Struct for 2D vectors=====*/

struct vec2{
    double x;
    double y;
};

/*=====Planet Class Definition=====*/

class Planet{
private:
    std::string name;
    double mass;
```

```

    vec2 position;
    vec2 velocity;
    vec2 acceleration;
    bool trailPath;
    double radius;

public:

    Planet(std::string name, double mass, vec2 position, vec2 velocity,
    vec2 acceleration, bool trailPath){
        this->name = name;
        this->mass = mass;
        this->position = position;
        this->velocity = velocity;
        this->acceleration = acceleration;
        this->trailPath = trailPath;
    };

    int getRadius(){
        int radius = mass * 2;
        return radius;
    };

    double distanceToPlanet(vec2 thisPlanet, vec2 otherPlanet){
        double distance;
        distance = sqrt(pow((thisPlanet.x - otherPlanet.x),2) +
        pow((thisPlanet.y - otherPlanet.y),2));
        return distance;
    };

    double applyGravitationalForce(double mass, double mass2, double
    distance){
        double Force;
        Force = (6.67 * pow(10,-11)) * ((mass*mass2) /
        pow(distance,2));
        return Force;
    };

    double calculateAngle(vec2 thisPlanet, vec2 otherPlanet){
        double Angle;
        Angle = atan(abs(thisPlanet.x - otherPlanet.x)/abs(thisPlanet.y
        - otherPlanet.y));
    };

```

```

        return Angle;
    }

    vec2 calculateResultantForce(double Force, double angle){
        vec2 resultantForce;
        resultantForce.x = Force * sin(angle);
        resultantForce.y = Force * cos(angle);
        return resultantForce;
    }

    vec2 calculateAcceleration(vec2 resultantForce, double mass){
        vec2 acceleration;
        acceleration.x = resultantForce.x / mass;
        acceleration.y = resultantForce.y / mass;
        return acceleration;
    }

    vec2 updateVelocity(vec2 acceleration){
        velocity.x = velocity.x + acceleration.x;
        velocity.y = velocity.y + acceleration.y;
        return velocity;
    }

    vec2 updatePosition(vec2 velocity){
        position.x = position.x + velocity.x;
        position.y = position.y + velocity.y;
        return position;
    }

};

```

Testing

updateVelocity()

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail
TPV1	Normal	X Initial velocity: 0 Acceleration: 10	Testing to ensure that the updateVelocity() function	X direction: 10 Y direction: 10	X: 10 Y: 10	SC15	Pass

		Y Initial velocity: 0 Accelera tion: 10	operates as it should.				
TPV2	Normal	X Initial velocity: 13 Accelera tion: 7 Y Initial velocity: 7 Accelera tion: 5		X direction: 20 Y direction: 12	X: 20 Y: 12	SC15	Pass
TPV3	Normal	X Initial velocity: 10 Accelera tion: 0 Y Initial velocity: 10 Accelera tion: 0		X direction: 10 Y direction: 10	X: 10 Y: 10	SC15	Pass

updatePosition()

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail
TPV1	Normal	X Initial Position: 0 Velocity: 10 Y Initial Position: 0	Testing to ensure that the updatePosition() function operates as it should.	X direction: 10 Y direction: 10	X: 10 Y: 10	SC14 SC15	Pass

		Velocity: 10					
TPV2	Normal	X Initial position: 13 Velocity: 7 Y Initial position: 7 Velocity: 5		X direction: 20 Y direction: 12	X: 20 Y: 12	SC14 SC15	Pass
TPV3	Normal	X Initial position: 10 Velocity: 0 Y Initial position: 10 Velocity: 0		X direction: 10 Y direction: 10	X: 10 Y: 10	SC14 SC15	Pass

Review

The aim of this stage was to create a planet I class that I can use in later development for my program.

At this stage I am very happy with the planet class. All the necessary tests for the planet class now pass. It also aligns closely with my design, only having a few minor changes. I am therefore confident that I can move onto creating **Prototype 1** using the planet class I have just created.

Stage 3: Prototype 1

Explanation and Justification

For stage 3 in my development, I'm going to create a prototype of the overall solution. The main difference between this prototype and my final program is that the prototype will be entirely in 2 dimensions. This is much easier to work with as I don't have to think about the complications that come with 3D rendering such as shading and matrices.

This prototype will also not have trail paths or the space-time grid for simplicity. I will add these after I have a working 3D simulation. This is because a space-time grid doesn't work in 2D as it requires displacement in the z direction to function. As 2D only has x and y directions, a space-time grid would not function in 2D. I don't plan to add trail paths to this prototype because it will take focus away from the problem I'm trying to solve by developing this prototype; to simulate gravity calculations.

My justification for this is that it will allow me to introduce the physics calculations I need without worrying about rendering complications. As the main program will only run calculations in 2D, I can carry over the code from this prototype into the main program and focus on the rendering side of it when developing the main program.

I will gradually build up the complexity of my simulation starting from the very basics.

Prototype 2: Circle

Explanation and Justification

Within prototype 1, I want to create a circle prototype using OpenGL. The aim for this prototype is to output a circle to the window using OpenGL.

My justification for this is that making working code which creates a circle on the screen will make it much easier in the development of prototype 1 to move circles around the screen according to calculations of gravity.

Build

I started by using OpenGL to render a circle onto the window. In OpenGL, there is no defined function for creating a circle. Instead, a circle is created by iterating around the edge of a circle using a varying angle and creating vertices around the edge at regular intervals. These vertices are then used with the centre to create many triangles which OpenGL is able to render. To implement this I used a for loop which iterates through the entire circumference of the circle. The angle is calculated in radians (which I learnt from implementing the planet class). The constant **M_PI** is a cmath constant for the value of pi. As 2.0π radians is equal to 360 degrees, this means I'm looping through the entire circle.

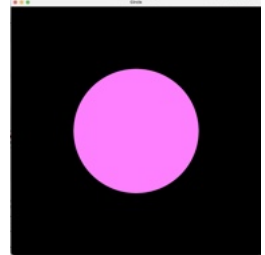
The variable **numSegments** dictates how many segments I create on the circumference of the circle. More segments = greater resolution. I divide through by this so that as i increases - getting closer to the number of segments - a greater proportion of 2π is used as the angle.

At each iteration in the loop, I use the **radius** variable with the angle to calculate the coordinates of each point on the circumference.

I then use the OpenGL function **glVertex2f()** to tell OpenGL that I want to include this point in my shape. As I've used the OpenGL command **glBegin(GL_TRIANGLE_FAN)**, this tells OpenGL that I want to draw triangles with these points. Therefore, OpenGL draws triangles, forming the circle.

```
C/C++
glBegin(GL_TRIANGLE_FAN);
glColor3f(1.0f, 0.5f, 1.0f);
glVertex2f(0.0f, 0.0f);
int numSegments = 100;
float radius = 0.5f;
for (int i = 0; i <= numSegments; i++) {
    float angle = i * 2.0f * M_PI / numSegments;
    float x = radius * cosf(angle);
    float y = radius * sinf(angle);
    glVertex2f(x, y);
}
glEnd();
```

Testing

Test ID	Description	Success criteria	Pass or Fail	EVIDENCE
PT1	The screen renders a circle	SC14	Pass	

Review

Since I have successfully created a circle using OpenGL commands and test PT1 has passed, I am confident that I can move onto implementing Prototype 1 Version 1 using this. I am also closer to succeeding in implementing SC14.

Prototype 1 Version 1 - Using Planet Class

Build

The first step in creating prototype 1 is to merge the planet class with the draw circle function which I had just created in prototype 2. Here is the updated code for drawing a circle using the position of **circle1**, an instance of the planet class:

```
C/C++
glBegin(GL_TRIANGLE_FAN);
```

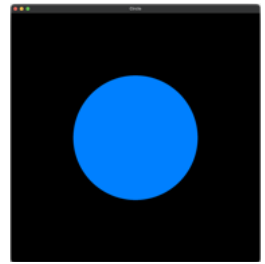
```

glColor3f(0.0f, 0.5f, 1.0f); // Planet color
glVertex2f(static_cast<float>(circle1.position.x),
static_cast<float>(circle1.position.y)); // Center
    for (int i = 0; i <= 100; i++) {
        float angle = i * 2.0f * static_cast<float>(M_PI) / 100.0f;
        glVertex2f(static_cast<float>(circle1.position.x) + cosf(angle)
* 40.0f, static_cast<float>(circle1.position.y) + sinf(angle) * 40.0f);
    }
glEnd();

```

Testing

I can now carry out test PO1

Test ID	Description	Success criteria	Pass or Fail	EVIDENCE
PO1	The simulation successfully renders planets on the screen as circles.	SC14	Pass	

Review

Now that I can render a circle using the planet class, I can move onto implementing movement to these circles, which is required for the simulation to work effectively.

Prototype 1 Version 2 - Movement

Build

Next, I created the c++ code to move a single planet with a constant acceleration and output the position in the terminal, using the planet class I had already created.

```

C/C++
int main(){

    Planet circle1("CircleOne", 5000, vec2{-100,200}, vec2{20,0},
vec2{9.8,0}, true);

```

```

    dt = 1;

    for(int i = 0; i < 11; i++){

        // Update physics using the time step:
        // apply acceleration scaled by dt to velocity, and move position by
        // velocity * dt
        circle1.updateVelocity(vec2{circle1.acceleration.x * dt,
circle1.acceleration.y * dt});
        circle1.updatePosition(vec2{circle1.velocity.x * dt,
circle1.velocity.y * dt});

        std::cout << "Circle Position" << circle1.position;
    }

}

```

I then combined the two processes into a single program that used the planet class to render planets on the window.

```

C/C++
int main(){

    Planet circle1("CircleOne", 5000, vec2{-100,200}, vec2{20,0},
vec2{9.8,0}, true);

    if (!glfwInit()) {
        std::cerr << "Failed to initialize GLFW\n";
        return -1;
    }

    // Request an older/compatibility context so immediate mode
    (glBegin/glEnd) works on platforms
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 2);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 1);

    const int width = 1000;
    const int height = 700;
    GLFWwindow* window = glfwCreateWindow(width, height, "Planet
Simulation", NULL, NULL);
    if (!window){

```

```

        std::cerr << "Failed to create GLFW window\n";
        glfwTerminate();
        return -1;
    }
    glfwMakeContextCurrent(window);

    // Set up viewport and coordinate system
    glViewport(0, 0, width, height);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(-width/2, width/2, -height/2, height/2, -1, 1);
    glMatrixMode(GL_MODELVIEW);

    // Set a clear color for the background
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);

    // Use a time step so movement is frame-rate independent
    double lastTime = glfwGetTime();

    while (!glfwWindowShouldClose(window)) {

        double currentTime = glfwGetTime();
        double dt = currentTime - lastTime;
        if (dt <= 0.0) dt = 0.0001;
        lastTime = currentTime;

        // Update physics using the time step:
        // apply acceleration scaled by dt to velocity, and move
        position by velocity * dt
        circle1.updateVelocity(vec2{circle1.acceleration.x * dt,
circle1.acceleration.y * dt});
        circle1.updatePosition(vec2{circle1.velocity.x * dt,
circle1.velocity.y * dt});

        glClear(GL_COLOR_BUFFER_BIT);

        // Reset modelview so object coordinates are drawn in world
space
        glLoadIdentity();

        // Draw planet (simple circle)
        glBegin(GL_TRIANGLE_FAN);
        glColor3f(0.0f, 0.5f, 1.0f); // Planet color
        glVertex2f(static_cast<float>(circle1.position.x),
static_cast<float>(circle1.position.y)); // Center
        for (int i = 0; i <= 100; i++) {
            float angle = i * 2.0f * static_cast<float>(M_PI) / 100.0f;

```

```

        glVertex2f(static_cast<float>(circle1.position.x) +
cosf(angle) * 40.0f, static_cast<float>(circle1.position.y) + sinf(angle) *
40.0f);
    }
    glEnd();

    //swaps the invisible back buffer with the buffer currently being
    rendered on the screen
    glfwSwapBuffers(window);

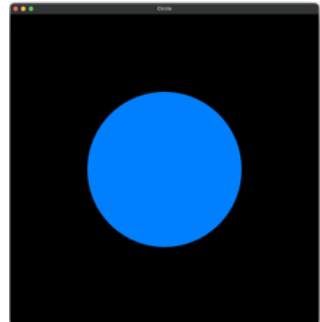
    //checks for inputs such as keyboard presses, mouse movements, or
    window resizing that have occurred since the last frame
    glfwPollEvents();
}

glfwTerminate();
return 0;
}

```

Testing

I then carried out these tests to see if my program was working correctly at this stage:

Test ID	Description	Success criteria	Pass or Fail	Evidence
PO1	The simulation successfully renders planets on the screen as circles.	SC14	Pass	 As seen above
PO2	The simulation can successfully introduce velocity for planets in the simulation and render this as movement across the window.	SC16	Fail - the circle would disappear halfway across the screen	
PO3	The simulation can successfully introduce acceleration for planets in	SC16	Fail - the circle would disappear	

	the simulation and render this as increasing velocity for the planets on the screen.		halfway across the screen	
--	--	--	---------------------------	--

During these tests the circle would disappear halfway across the screen and therefore failed tests MLR2 and MLR3

Review and Debugging

I wasn't sure how to fix this, so I asked GitHub copilot. This is fully referenced below with its solution. I accessed GitHub copilot for this fix on the 8th November 2025.

Prompt	EVIDENCE
'I'm using OpenGL to render a circle moving across the window. However, the circle disappears halfway across the window, how can I fix this.'	I'm using OpenGL to render a circle moving across the window. However, the circle disappears halfway across the window, how can I fix this? I'd be happy to help you fix the issue with your OpenGL circle disappearing halfway across the window! This is typically caused by one of a few common issues: Most Likely Causes: 1. Clipping/Culling Issues: The circle might be getting clipped because its center goes off-screen but the rendering doesn't account for the radius 2. Coordinate System Problems: Your viewport or projection matrix might not match your coordinate calculations 3. Depth Buffer Issues: If using 3D coordinates, depth testing might be culling the circle 4. Window Bounds Check: Logic that stops rendering when the center crosses a threshold Quick Fixes to Try: 1. Adjust Your Boundary Check If you're checking if the circle is out of bounds, make sure to account for the radius: C++ // TODO: add

The first suggestion from GitHub copilot was this:

Response	EVIDENCE
Check boundary conditions to see if this was causing the problem.	Quick Fixes to Try: 1. Adjust Your Boundary Check If you're checking if the circle is out of bounds, make sure to account for the radius: C++ // Instead of: if (circle_x > window_width) // Wrong - disappears when center reaches edge // Use: if (circle_x - radius > window_width) // Correct - accounts for radius

Since I didn't have any boundary conditions at this stage, I started by implementing some that would reposition the circle to the opposite side if it left the screen. . The boundary conditions looked like this:

C/C++

```
if (circle1.position.x > width/2) circle1.position.x = -width/2;
if (circle1.position.x < -width/2) circle1.position.x = width/2;
if (circle1.position.y > height/2) circle1.position.y = -height/2;
if (circle1.position.y < -height/2) circle1
```

This didn't solve the problem, but did make it a lot easier to see what was happening now. The circles were only moving across a certain section of the window. When they reached the other side of this window, they were being repositioned to the opposite as my boundary conditions told them to. However, I was sure that these new boundary conditions did not create this problem because the problem existed before implementing these.

The second suggestion from GitHub copilot was this:

Response	EVIDENCE
Check my projection and viewport code	2. Check Your Projection/Viewport <pre>C++ // Make sure your projection matches your coordinate system glViewport(0, 0, window_width, window_height); glMatrixMode(GL_PROJECTION); glLoadIdentity(); glOrtho(0, window_width, window_height, 0, -1, 1); // Adjust as needed</pre>

I therefore looked to this code, as this might be the source of my problem:

C/C++

```
glViewport(0, 0, width, height);
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
glOrtho(-width/2, width/2, -height/2, height/2, -1, 1);
glMatrixMode(GL_MODELVIEW);
```

The **glViewport()** function defines the rectangular region of the window where OpenGL will draw. Everything outside this will not be drawn.

The **glOrtho()** function sets up the coordinate system of the space. In this example I'm using it to set up 2D projection.

I therefore thought that inconsistency between these two functions was causing my problem.

The **glOrtho()** function was not covering the right side of the window.

I therefore updated the parameters of the **glOrtho()** function to match up with those of the **glViewport()** function, hoping this would make a difference but I still had the same problem.

C/C++

```
glViewport(0, 0, width, height);
glMatrixMode(GL_PROJECTION);
```

```
glLoadIdentity();
glOrtho(0, width, 0, height, -1, 1);
glMatrixMode(GL_MODELVIEW);
```

I ran the code a few more times with different dimensions but I still couldn't fix the problem.

This was until I clicked on the window to move it around my screen. When I clicked on the window, the viewpoint immediately changed, making the circle bigger and changing its position. The circle no longer disappeared halfway. I was very intrigued at how this could happen, so I asked GitHub copilot:

This was also accessed on the 8th November

Prompt	EVIDENCE
'Why, when I clicked on the window, did this problem fix itself?'	"Why, when I clicked on the window, did this problem fix itself?" Ah, that's a really interesting clue! The fact that clicking on the window fixes the problem suggests it's not actually a boundary condition issue, but rather a rendering/update issue. Here are the most likely causes: Most Probable Causes: 1. Window Focus/Event Processing Issue The window might not be processing events or refreshing properly until you interact with it: <pre>C++ // Problem: Only redrawing when events occur while (! glfwWindowShouldClose(window)) { glfwPollEvents(); // Only processes when there ARE events render(); } // Solution: Force continuous rendering while (!glfwWindowShouldClose(window)) { glfwPollEvents(); render(); glfwSwapBuffers(window); // Make sure this is called every frame }</pre>

Response	EVIDENCE
When I click the window, GLFW triggers a redraw event. It also suggested that the cause for this was using the wrong command word, but since I wasn't, I ignored this. In the future, I would make sure GLFW triggers a redraw event before testing.	When you click on the window, GLFW triggers a window event (like a mouse click or focus event), which causes your event loop to wake up and process a redraw. This is a classic symptom of an event-driven render loop rather than a continuous render loop. The Problem: You're likely using <code>glfwWaitEvents()</code> instead of <code>glfwPollEvents()</code>: <pre>C++ // ✗ WRONG - Only redraws when events occur (clicks, mouse moves, etc.) while (!glfwWindowShouldClose(window)) { glfwWaitEvents(); // * Blocks until an event happens! render(); }</pre> <code>glfwWaitEvents()</code> blocks your program until a user interaction occurs, which is why: - The circle appears frozen/disappears - Clicking "fixes" it temporarily (triggers one redraw) - The animation doesn't run smoothly

Testing

Having fixed this problem, I was left with a window that rendered an accelerating circle, moving from left to right, which would leave one side of the window and enter back on the opposite side. This can be seen on **EVIDENCE VIDEO 1**.

In the video you can see the shift in view of the window.

I then repeated the tests that had failed:

Test ID	Description	Success criteria	Pass or Fail	Evidence
PO2	The simulation can successfully introduce velocity for planets in the simulation and render this as movement across the window.	SC16	Fail - the circle would disappear halfway across the screen	EVIDENCE VIDEO 1
PO3	The simulation can successfully introduce acceleration for planets in the simulation and render this as increasing velocity for the planets on the screen.	SC16	Fail - the circle would disappear halfway across the screen	EVIDENCE VIDEO 1

Prototype 1 Version 3 - Multiple Circles

Build

Next I wanted to add another circle, which would move from right to left, in order to see how the simulation would work with several bodies.

I first instantiated another planet:

```
C/C++  
Planet circle2("CircleTwo", 3000, vec2{100,-200}, vec2{-30,0}, vec2{-9.8,0},  
true);
```

Next I included the if statements to wrap the circle to the opposite side. I've used 80.0 here instead of 40.0 for the previous circle because as I will show later, I increased the radius of this circle.

C/C++

```
if (circle2.position.x > width/2 + 80.0) circle2.position.x = -width/2 - 80.0;
if (circle2.position.x < -width/2 - 80.0) circle2.position.x = width/2 + 80.0;
if (circle2.position.y > height/2 + 80.0) circle2.position.y = -height/2 - 80.0;
if (circle2.position.y < -height/2 - 80.0) circle2.position.y = height/2 + 80.0;
```

My next step was to repeat the block of code that draws the circle, but this time I will double the radius:

C/C++

```
glBegin(GL_TRIANGLE_FAN);
glColor3f(0.0f, 1.0f, 0.0f); // Planet color
glVertex2f(static_cast<float>(circle2.position.x), static_cast<float>(circle2.position.y)); // Center
for (int i = 0; i <= 100; i++) {
    float angle = i * 2.0f * static_cast<float>(M_PI) / 100.0f;
    glVertex2f(static_cast<float>(circle2.position.x) + cosf(angle) * 80.0f, static_cast<float>(circle2.position.y) + sinf(angle) * 80.0f);
}
glEnd();
```

Testing

This is shown in **EVIDENCE VIDEO 2**.

I could then carry out test PO4 to see if it would pass:

Test ID	Description	Success criteria	Pass or Fail	Evidence
PO4	The simulation can move and accelerate two planets at once.	SC16	Pass	EVIDENCE VIDEO 2

Review

As I have now implemented moving multiple planets/circles - necessary for SC16 - I am moving onto implementing a list that will store all my planets. I think this was a successful

build, however it was frustrating that after carrying out debugging I realised that simply clicking on the window would solve my problem.

Prototype 1 Version 4 - Planets List

Build

In version 4 I wanted to introduce a vector which would store all the planets that I instantiate. I started by including the vector library into my file. I then created the **planets** vector which will store the planets I instantiate. I added my planets to this:

```
C/C++  
std::vector<Planet> planets;  
planets.push_back(circle1);  
planets.push_back(circle2);
```

I could then replace the manual process of updating the velocity and the position of the planets with this for loop:

```
C/C++  
for(int i = 0; i < planets.size(); i++){  
    planets[i].updateVelocity(vec2{planets[i].acceleration.x * dt,  
    planets[i].acceleration.y * dt});  
    planets[i].updatePosition(vec2{planets[i].velocity.x * dt,  
    planets[i].velocity.y * dt});  
};
```

I can then replace the if statements which wrap the circles around the window with this for loop:

```
C/C++  
for (int i = 0; i < planets.size(); i++){  
    if (planets[i].position.x > width/2 + planets[i].radius)  
planets[i].position.x = -width/2 - planets[i].radius;  
    if (planets[i].position.x < -width/2 - planets[i].radius)  
planets[i].position.x = width/2 + planets[i].radius;  
    if (planets[i].position.y > height/2 + planets[i].radius)  
planets[i].position.y = -height/2 - planets[i].radius;  
    if (planets[i].position.y < -height/2 - planets[i].radius)  
planets[i].position.y = height/2 + planets[i].radius;  
}
```

However, this requires using the radius of the planet rather than me manually inputting it for each planet. Due to this I looked back at my planet class to see how I would implement this and realised I could make some changes to the way I dealt with the radius of my planets.

I want my radius to depend on the mass of the object. I had previously planned to do this with the **getRadius()** method which I have already created. Since creating this, however, I have realised that a much better way of doing this is to use the constructor method to instantiate the radius as a multiple of the mass. This is how I have done that:

```
C/C++
Planet(std::string name, double mass, vec2 position, vec2 velocity, vec2
acceleration, bool trailPath){
    this->name = name;
    this->mass = mass;
    this->position = position;
    this->velocity = velocity;
    this->acceleration = acceleration;
    this->trailPath = trailPath;
    this->radius = mass/100;
}
```

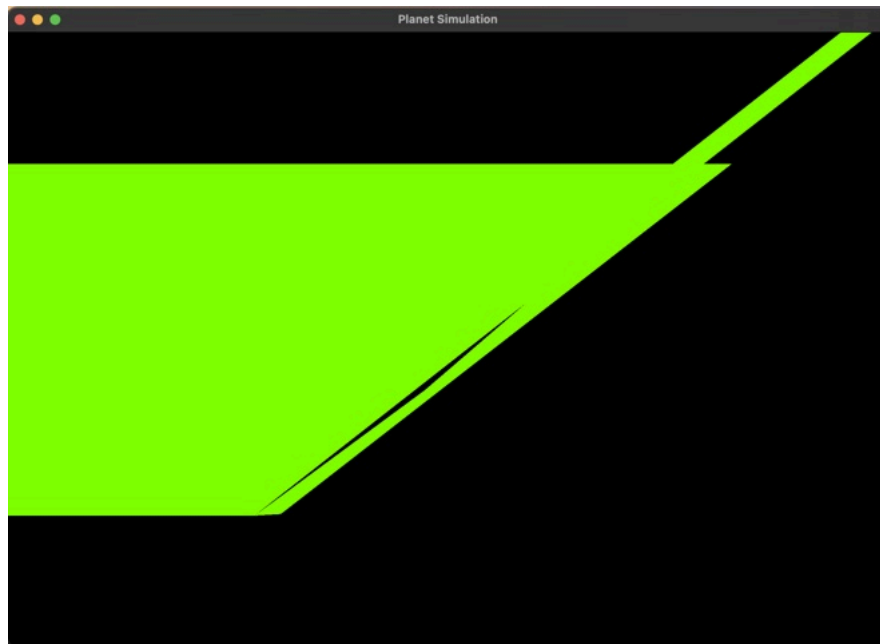
I then deleted the **getRadius()** method as I no longer needed it.

The next stage to implementing the vector was to replace the lines of code that draw circles onto the window with a for loop that loops through the vector and draws each circle:

```
C/C++
for (int i = 0; i < planets.size(); i++){
    glBegin(GL_TRIANGLE_FAN);
    glColor3f(0.5f, 1.0f, 0.0f);
    glVertex2f(static_cast<float>(planets[i].position.x),
static_cast<float>(planets[i].position.y));
    for (int i = 0; i <= 100; i++) {
        float angle = i * 2.0f * static_cast<float>(M_PI) /
100.0f;
        glVertex2f(static_cast<float>(planets[i].position.x) +
cosf(angle) * planets[i].radius, static_cast<float>(planets[i].position.y) +
sinf(angle) * planets[i].radius);
    }
    glEnd();
};
```

Testing and Debugging

Running this did not give me what I expected. The window showed a big green shape that moved around the window. Here is a screenshot of what it looked like:



After looking at the lines of code that draw the circles again, the error was immediately obvious. I had used the same loop variable for both for loops. This was an easy fix as I replaced the `i` loop variable in the inner loop with a `j`, this fixed the problem.

To check that this vector would work with planets, I added another planet to the vector. I decided to have this one moving in the other direction, to show that planets in both directions could be rendered.

```
C/C++
Planet circle3("CircleThree", 4000, vec2{100,0}, vec2{0,40}, vec2{0,9.8},
true);
planets.push_back(circle3);
```

I then repeated test PO4:

Test ID	Description	Success criteria	Pass or Fail	Evidence
PO4	The simulation can move and accelerate two planets at once.	SC16	Pass	EVIDENCE VIDEO 3

This is shown in **EVIDENCE VIDEO 3**.

Review

As I now have a working list of planets which I can iterate through to update the positions of each, I am now happy that I can move onto implementing gravity and start to move towards fulfilling SC15 and SC16.

Prototype 1 Version 5 - Gravity

Build

In version 5 I wanted to introduce gravity to the simulation prototype. The first step of introducing gravity is using the planet methods I have already created to calculate the effect of gravity on each planet. I therefore created a for loop which used these to calculate the acceleration of gravity on each planet.

C/C++

```
for(int i = 0; i < planets.size(); i++){
    planets[i].calculateAcceleration(planets[i].calculateResultantForce(planets[
    i].applyGravitationalForce(planets[i].mass, planets[i+1].mass,
    planets[i].distanceToPlanet(planets[i].position, planets[i+1].position)),
    planets[i].calculateAngle(planets[i].position, planets[i+1].position)),
    planets[i].mass);
}
```

However, this revealed that my planet class could be much better designed. There are several functions all as parameters of each other. This is not only confusing, but difficult to maintain and update if I need to later down the line. I therefore shifted my attention to modifying my planet class to avoid this. Here are the updated methods of my planet class

C/C++

```
double distanceToPlanet(vec2 otherPlanet){
    double distance;
    distance = sqrt(pow((position.x - otherPlanet.x),2) +
    pow((position.y - otherPlanet.y),2));
    return distance;
}

double applyGravitationalForce(double otherMass, vec2
otherPosition){
    double Force;
    Force = (100) * ((mass*otherMass) /
    pow((distanceToPlanet(otherPosition)),2));
    return Force;
}
```

```

};

double calculateAngle(vec2 otherPlanet){
    double Angle;
    Angle = atan(abs(position.x - otherPlanet.x)/abs(position.y -
otherPlanet.y));
    return Angle;
}

vec2 calculateResultantForce(double Force, double angle){
    vec2 resultantForce;
    resultantForce.x = resultantForce.x + (Force * sin(angle));
    resultantForce.y = resultantForce.y + (Force * cos(angle));
    return resultantForce;
}

void updateAcceleration(vec2 otherPosition, double otherMass){
    vec2 resultantForce;
    resultantForce =
calculateResultantForce(std::get<double>(applyGravitationalForce(otherMass,
otherPosition)), calculateAngle(otherPosition) );
    acceleration.x = acceleration.x + resultantForce.x / mass;
    acceleration.y = acceleration.y + resultantForce.y / mass;
}

vec2 updateVelocity(vec2 acceleration){
    velocity.x = velocity.x + acceleration.x;
    velocity.y = velocity.y + acceleration.y;
    return velocity;
}

vec2 updatePosition(vec2 velocity){
    position.x = position.x + velocity.x;
    position.y = position.y + velocity.y;
    return position;
}

```

This meant that I could use this much more organised for loop which is much easier to maintain:

```

C/C++
for(int i = 0; i <planets.size(); i++){
    for(int j = 0; j <planets.size(); j++){

```

```

        if (i != j){
            planets[i].updateAcceleration(planets[j].position, planets[j].mass);
        }
    }
}

```

This loop has an outer loop which iterates through all the planets in the vector and uses an inner loop to calculate the gravitational force due to every other mass in the simulation. This is then used to calculate the acceleration due to this force.

Testing

I carried out test PO5 at this stage, as I was expecting to get a visual representation of gravity between two planets:

Test ID	Description	Success criteria	Pass or Fail	Evidence
PO5	The simulation successfully uses the planet class functions to give a visual representation of gravity between two planets in the window.	SC15	Fail - the planets both accelerated towards the right	EVIDENCE VIDEO 4

Review and Debugging

Testing revealed that I did not get what I expected. I expected to see two circles on either side of the window moving towards each other, but instead the two circles both accelerated towards the right. This is because currently there is no way of either planet being able to have a negative acceleration and therefore they both move to the left.

This can be seen in **EVIDENCE VIDEO 4**

I looked back at the planet class methods to try and fix this error. I realised that it was the `abs()` function in my **calculateAngle()** method that was causing the error, as this stopped any negative angles from being returned. I updated the **calculateAngle()** method to this:

```
C/C++
double calculateAngle(vec2 otherPlanet){
    double dx = otherPlanet.x - position.x;
    double dy = otherPlanet.y - position.y;
    return atan2(dy, dx);
}
```

Testing

After debugging I could repeat the PO5 test as I hoped my changes had fixed my problem.

Test ID	Description	Success criteria	Pass or Fail	Evidence
PO5	The simulation successfully uses the planet class functions to give a visual representation of gravity between two planets in the window.	SC15	Pass	EVIDENCE VIDEO 5

This solved my problem. I now have a running 2D gravity simulator which I tested by running the simulation with two planets at first. I then tested it with 3 planets, which started with initial velocities. This can be seen in **EVIDENCE VIDEO 5**.

Review

This version allowed me to effectively create a simulation where two planets are under the gravitational influence of each other, necessary for SC15. As I have this working I will move onto implementing a user interface.

Prototype 1 Version 6 - Front End

Iteration 1

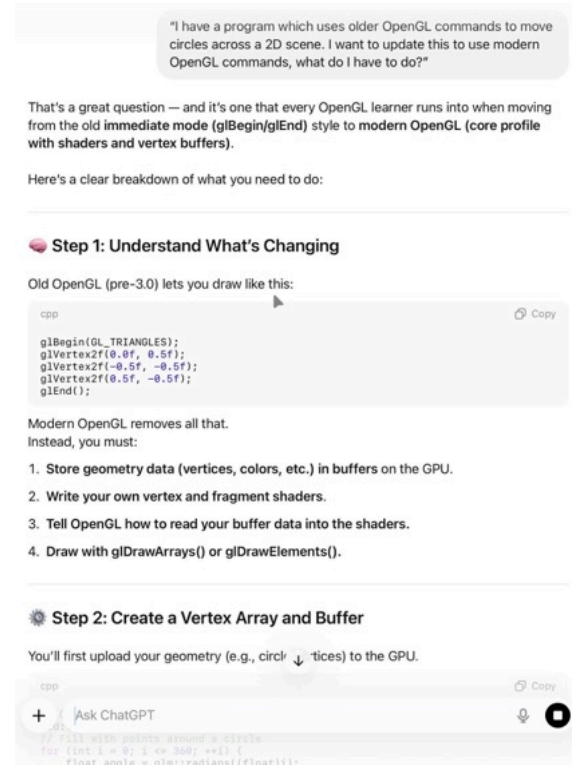
Build

As I have got to the point where I have a working 2D simulation, I now want to implement a front end before continuing with my back-end development. In order to implement my front-end, I'm using Emscripten, which is a compiler that translates C++ into output that runs in web browsers. It will allow me to run my code on the web and provide integration with WebGL. What I didn't realise about Emscripten though was that it doesn't support older

OpenGL commands which I had been using. I therefore had to switch to modern OpenGL, which Emscripten does support.

I used chatGPT to support me through this process. This is because I wanted a solid list of changes that I would have to make in order to switch from one version to another. I could go through this list and implement the necessary changes to my program.

For this ChatGPT was accessed on the 15th November 2025.

Prompt	EVIDENCE
"I have a program which uses older OpenGL commands to move circles across a 2D scene. I want to update this to use modern OpenGL commands, what do I have to do?"	 The screenshot shows a ChatGPT interface. At the top, the user's prompt is: "I have a program which uses older OpenGL commands to move circles across a 2D scene. I want to update this to use modern OpenGL commands, what do I have to do?". The response starts with: "That's a great question — and it's one that every OpenGL learner runs into when moving from the old immediate mode (glBegin/glEnd) style to modern OpenGL (core profile with shaders and vertex buffers). Here's a clear breakdown of what you need to do:". It then lists two steps. Step 1: "Understand What's Changing". It explains that Old OpenGL (pre-3.0) lets you draw like this, showing a code block: <pre>cpp glBegin(GL_TRIANGLES); glVertex2f(0.8f, 0.5f); glVertex2f(-0.5f, -0.5f); glVertex2f(0.5f, -0.5f); glEnd();</pre> It then states that Modern OpenGL removes all that and lists four tasks: 1. Store geometry data (vertices, colors, etc.) in buffers on the GPU. 2. Write your own vertex and fragment shaders. 3. Tell OpenGL how to read your buffer data into the shaders. 4. Draw with glDrawArrays() or glDrawElements(). Step 2: "Create a Vertex Array and Buffer". It says: "You'll first upload your geometry (e.g., circles) to the GPU." and shows a partial code block for a circle.

Response	EVIDENCE																																	
ChatGPT gave me an in depth broken down list of what I had to do. Here is the summary of those tasks that it gave me.	<table><tr><th>Step</th><th>Task</th><th>Purpose</th></tr><tr><td>1</td><td>Setup GLFW & GLAD</td><td>Create OpenGL 3.3+ context</td></tr><tr><td>2</td><td>Generate circle vertices</td><td>Replace glBegin/glVertex</td></tr><tr><td>3</td><td>Create VAO/VBO</td><td>Store geometry on GPU</td></tr><tr><td>4</td><td>Write shaders</td><td>Replace fixed pipeline</td></tr><tr><td>5</td><td>Compile & link shader program</td><td>Send to GPU</td></tr><tr><td>6</td><td>Use GLM</td><td>Handle transforms (no glTranslatef)</td></tr><tr><td>7</td><td>Send matrices as uniforms</td><td>Apply camera + model movement</td></tr><tr><td>8</td><td>Draw with glDrawArrays()</td><td>Render circle</td></tr><tr><td>9</td><td>Update per-frame transforms</td><td>Move circle across screen</td></tr><tr><td>10</td><td>Clean up resources</td><td>Proper memory management</td></tr></table>	Step	Task	Purpose	1	Setup GLFW & GLAD	Create OpenGL 3.3+ context	2	Generate circle vertices	Replace glBegin/glVertex	3	Create VAO/VBO	Store geometry on GPU	4	Write shaders	Replace fixed pipeline	5	Compile & link shader program	Send to GPU	6	Use GLM	Handle transforms (no glTranslatef)	7	Send matrices as uniforms	Apply camera + model movement	8	Draw with glDrawArrays()	Render circle	9	Update per-frame transforms	Move circle across screen	10	Clean up resources	Proper memory management
Step	Task	Purpose																																
1	Setup GLFW & GLAD	Create OpenGL 3.3+ context																																
2	Generate circle vertices	Replace glBegin/glVertex																																
3	Create VAO/VBO	Store geometry on GPU																																
4	Write shaders	Replace fixed pipeline																																
5	Compile & link shader program	Send to GPU																																
6	Use GLM	Handle transforms (no glTranslatef)																																
7	Send matrices as uniforms	Apply camera + model movement																																
8	Draw with glDrawArrays()	Render circle																																
9	Update per-frame transforms	Move circle across screen																																
10	Clean up resources	Proper memory management																																

I started working through this list. As I had already set up GLFW, I could ignore step 1. I also wanted to start with implementing GLM because this involved editing my headers.

As many modern OpenGL functions use GLM for maths and matrices, I included this library into my file:

```
C/C++
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>
```

After this I started updating my program to use modern OpenGL functions. To start with I replaced this code which set up the viewport and coordinate system:

```
C/C++
glViewport(0, 0, width, height);
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
glOrtho(-width/2, width/2, -height/2, height/2, -1, 1);
glMatrixMode(GL_MODELVIEW);
```

I could no longer use the `glOrtho` function, which is used to set the coordinate system. Instead I have to tell OpenGL to create the projection matrix using orthographic projection as I'm rendering in 2D. This is what the **`glm::ortho`** command does. This also involves inputting the dimensions which can be seen in the parameters of the `glm::ortho` command.

Next, I have to get the uniform location of the shader program and upload this from the c++ variable to the location I've obtained:

```
C/C++
glViewport(0, 0, width, height);
glm::mat4 projection = glm::ortho(
    -width / 2.0f, width / 2.0f,
    -height / 2.0f, height / 2.0f,
    -1.0f, 1.0f
);
GLuint projectionLoc = glGetUniformLocation(shaderProgram, "uProjection");
glUniformMatrix4fv(projectionLoc, 1, GL_FALSE, glm::value_ptr(projection));
```

Replacing the draw circle loop is where this process gets a lot more complicated. In modern OpenGL, I need to use a vertex shader and fragment shader. These shaders are crucial parts of the graphics pipeline, with the vertex shader being used to position vertices within the window and the fragment shader is responsible for colouring each pixel on the window. Here is the code for these, they are both written in GLSL, which is OpenGL's shader language:

C/C++

//Vertex Shader

```
const char* vertexShaderSource = R"(
    #version 330 core
    layout (location = 0) in vec2 aPos;
    uniform mat4 projection;
    uniform vec2 offset;
    uniform float radius;
    void main() {
        vec2 pos = aPos * radius + offset;
        gl_Position = projection * vec4(pos, 0.0, 1.0);
    }
);
```

//Fragment Shader

```
const char* fragmentShaderSource = R"(
    #version 330 core
    out vec4 FragColor;
    uniform vec3 color;
    void main() {
        FragColor = vec4(color, 1.0);
    }
);
```

The next step was to set up these shaders, which involved implementing this code outside the main render loop. This is required to compile and link the shaders which I have already defined. I have added comments which describe the purpose of the line above:

C/C++

// Create and compile shaders

```
unsigned int vertexShader = glCreateShader(GL_VERTEX_SHADER);
//Creates an empty vertex shader object on the GPU
glShaderSource(vertexShader, 1, &vertexShaderSource, NULL);
//Uploads the GLSL source code (which is shown above) into the shader object
glCompileShader(vertexShader);
//Compiles the source code into code the GPU can use
```

//The comments above apply to the fragment shader as well

```
unsigned int fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
glShaderSource(fragmentShader, 1, &fragmentShaderSource, NULL);
glCompileShader(fragmentShader);
```

```
unsigned int shaderProgram = glCreateProgram();
//Creates a new shader program object, used to link multiple shaders
```

```

    glAttachShader(shaderProgram, vertexShader);
    glAttachShader(shaderProgram, fragmentShader);
    //Attaches the compiled shaders to the program
    glLinkProgram(shaderProgram);
    //Links the shaders into a single GPU program

    glDeleteShader(vertexShader);
    glDeleteShader(fragmentShader);
    //These delete the shader objects as they are linked into the program and
    are not longer needed

    //This is the code for creating a circle, it loops through the outside of
    the circle making vertices and adding these to the vector unitCircleVertices
    const int numSegments = 100;
    std::vector<float> unitCircleVertices;

    // Precompute unit circle positions (radius 1.0, centered at origin)
    unitCircleVertices.push_back(0.0f); // center
    unitCircleVertices.push_back(0.0f);
    for (int j = 0; j <= numSegments; ++j) {
        float angle = j * 2.0f * M_PI / numSegments;
        unitCircleVertices.push_back(cosf(angle));
        unitCircleVertices.push_back(sinf(angle));
    }

    glGenVertexArrays(1, &planetVAO);
    glGenBuffers(1, &planetVBO);
    //Creates a Vertex Array Object (VAO) and Vertex Buffer Object (VBO), I have
    explained the need for these below outside the code block

    glBindVertexArray(planetVAO);
    glBindBuffer(GL_ARRAY_BUFFER, planetVBO);
    //Activates the VAO and VBO
    glBufferData(GL_ARRAY_BUFFER, unitCircleVertices.size() * sizeof(float),
    unitCircleVertices.data(), GL_STATIC_DRAW);
    //Uploads vertex data inside unitCircleVertices from the CPU to the GPU

    glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, 2 * sizeof(float),
    (void*)0);
    //This tells the GPU how to interpret the data in the VBO. The first 0 is
    the attribute location, the 2 tells the GPU that each vertex has 2
    components (x,y)
    glEnableVertexAttribArray(0);
    //Enables this attribute so the shader can access it

    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindVertexArray(0);
    //Unbinds the buffer and VAO

```

The VAO and VBO:

The VBO stores vertex data such as positions and colour on the GPU.

The VAO stores how the vertex data is handled and which buffers and attributes to use when drawing.

In short the VBO stores vertex data and the VAO stores vertex layout.

Then, inside my render loop I updated the old for loop which rendered each planet as a circle to this:

```
C/C++
glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
glClear(GL_COLOR_BUFFER_BIT);

glUseProgram(shaderProgram);
glBindVertexArray(planetVAO);

for (const auto& planet : planets) {
    // Build a model matrix per planet
    glm::mat4 model = glm::mat4(1.0f);
    model = glm::translate(model, glm::vec3(planet.position.x,
planet.position.y, 0.0f));
    model = glm::scale(model, glm::vec3(planet.radius, planet.radius,
1.0f));

    // Send uniforms
    glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "uModel"), 1,
GL_FALSE, glm::value_ptr(model));
    glUniform3f(glGetUniformLocation(shaderProgram, "uColor"), 1.0f, 0.5f,
0.5f);

    // Draw
    glDrawArrays(GL_TRIANGLE_FAN, 0, numSegments + 2);
}

glBindVertexArray(0);
```

I repeated tests PO1-5 after carrying out these changes and they all passed, suggesting that this process of updating my program to use modern OpenGL commands was a success.

After updating my program to use modern OpenGL functions, I can now use Emscripten to run it on the web.

Testing

Compiling with Emscripten is different from compiling a normal c++ file. The first thing I have to do to compile with Emscripten is to include the relevant header files in my c++ file:

```
C/C++
```

```
#include <emscripten/emscripten.h>
```

I then have to compile the code I've written with this command:

emcc Emscripten1.cpp -o index.html

I could then run it using: **emrun index.html**

This ran my code on a localhost browser, which I accessed via my default browser, safari. However, after doing this, instead of seeing my application run in the browser, a preparing message was displayed indefinitely.

At this stage all of my web-based tests failed:

Test ID	Description	Success criteria	Expected	Actual	Pass/Fail
TWe 1	Checks it runs in a browser, not as a native executable.	SC25	I can open a browser and access the program on a local host server.	Indefinite 'preparing' message	Fail
TWe 2	Checks that my simulation draws frames within the browser canvas.	SC25	My simulation to run as expected on the web page	Indefinite 'preparing' message	Fail
TWe 3	Checks that user input is handled within the browser.	SC25	Any input into the browser behaves as it would in the native executable.	Indefinite 'preparing' message	Fail

I tried several things to fix this error, including switching browsers and using different commands to run this code. There was not a lot of information online about this problem. In addition to this, the web browser that ran this code, which displayed the indefinite 'preparing' message, did not look as I had imagined. This is what it looked like, I found this picture on the web:



☐ Resize canvas ☒ Lock/hide mouse pointer

Fullscreen



I did not like the look of this, and thought it would ruin the look of my simulator. I therefore decided to look for another solution to the problem of creating a working front end for my simulation.

Review

After discussing with peers, and doing research on the best way to create a control panel for OpenGL, I decided to move away from Emscripten on the web and use Dear ImGui instead. From the Dear ImGui GitHub repository: “**Dear ImGui** is a bloat-free graphical user interface library for C++.”

I therefore went into prototype 1 version 7 iteration 2 with the aim of using Dear ImGui (ImGui for short) to create a front-end for my prototype.

Iteration 2

Build

The first step in setting this up was using the terminal to download Dear ImGui using the command: `git clone https://github.com/ocornut/imgui.git`.

I then added the relevant files to my project, and included these headers:

```
C/C++
#include "imgui.h"
#include "backends/imgui_impl_glfw.h"
#include "backends/imgui_impl_opengl3.h"
```

The following code was needed to initialize ImGui in my program:

```
C/C++
ImGui_CHECKVERSION();
ImGui::CreateContext();
ImGuiIO& io = ImGui::GetIO();

// Setup ImGui bindings
ImGui_ImplGlfw_InitForOpenGL(window, true);
ImGui_ImplOpenGL3_Init("#version 330");
```

This is the code I have to add inside my rendering loop. It displays the number of planets within the **planets** vector. This is just testing ImGui:

```
C/C++
ImGui_ImplOpenGL3_NewFrame();
ImGui_ImplGlfw_NewFrame();
ImGui::NewFrame();

ImGui::Begin("Planet Simulation Controls");
ImGui::Text("Number of Planets: %d", static_cast<int>(planets.size()));
ImGui::End();

ImGui::Render();
ImGui_ImplOpenGL3_RenderDrawData(ImGui::GetDrawData());
```

I also needed to add this cleanup before destroying my GLFW window:

```
C/C++
ImGui_ImplOpenGL3_Shutdown();
ImGui_ImplGlfw_Shutdown();
ImGui::DestroyContext();
```

I now have to update my compile command to include the necessary ImGui files: **g++ -std=c++17 main.cpp imgui/imgui.cpp imgui/imgui_draw.cpp imgui/imgui_tables.cpp imgui/imgui_widgets.cpp imgui/backends/imgui_impl_glfw.cpp imgui/backends/imgui_impl_opengl3.cpp -limgui -limgui/backends -I/usr/local/include -L/usr/local/lib -lglfw -framework OpenGL -framework Cocoa -framework IOKit -framework CoreVideo -Wno-deprecated-declarations -o app**

Testing

There was no testing to be carried out at this stage, as PO1-5 already passed and PO6 would not pass because I have not implemented this functionality. However, that being said, ImGui was successfully rendered in the window.

This can be seen in **EVIDENCE VIDEO 6**.

Iteration 3

Build

My next step was to set up the front end so that I could add planets of custom attributes. I therefore had to link the ImGui input to the c++. It was at this point that I realised that ImGui doesn't have a built-in function for using sliders with a double. I therefore had to change all the data types of my 2D struct and planet to floats. The only difference between floats and doubles in the memory they use, with doubles using 8 bytes and floats using 4 bytes. This won't make any difference to my program as floats still give me the precision I need.

Therefore, after changing all the doubles in my program, I then added the following code to the ImGui render loop to edit the mass (and therefore radius) of one of the planets in my simulation. This can be seen in **EVIDENCE VIDEO 7**.

```
C/C++  
ImGui::SliderFloat("Mass", circle1.mass, 100.0f, 10000.0f);
```

Testing

I then carried out tests PO6 and PO7:

Test ID	Description	Success criteria	Pass or Fail	Evidence
PO6	The simulation allows the user to add planets.	SC6	Fail - there is no interface control that allows that user to add planets.	EVIDENCE VIDEO 7
PO7	The simulation allows the user to customise planets.	SC1-5	Pass - the user can edit the mass of the planet that the slider controls.	EVIDENCE VIDEO 7

Iteration 4

Build

I added the following code inside my render loop. I've defined several variables as defaults to allow me to work with them. I've then created sliders and text boxes to edit these variables. These variables are then used to instantiate an instance of the class planet.

```
C/C++
ImGui::Begin("Planet Creator");

char name[128] = "Placeholder";
float newPositionX[1] = {0.0f};
float newPositionY[1] = {0.0f};
float newVelocityX[1] = {0.0f};
float newVelocityY[1] = {0.0f};
float newMass = 1.0f;
float newColor[3] = {1.0f, 1.0f, 1.0f};

ImGui::Text("Create a new planet:");

ImGui::InputText("Planet Name", name, 128);
ImGui::Text("Position:");
ImGui::SliderFloat("X: ", newPositionX, -10.0f, 10.0f);
ImGui::SliderFloat("Y: ", newPositionY, -10.0f, 10.0f);

ImGui::Text("Velocity:");
ImGui::SliderFloat("Velocity", newVelocityX, -5.0f, 5.0f);
ImGui::SliderFloat("Velocity", newVelocityY, -5.0f, 5.0f);

ImGui::SliderFloat("Mass", &newMass, 0.1f, 100.0f);

if (ImGui::Button("Add Planet")) {
    // Create planet using your class constructor

    vec2 newPosition = {newPositionX[0], newPositionY[0]};
    vec2 newVelocity = {newVelocityX[0], newVelocityY[0]};
    vec2 newAcceleration = {0.0f, 0.0f};
    Planet newPlanet(name, newMass, newPosition, newVelocity,
newAcceleration, true);

    planets.push_back(newPlanet);
}

ImGui::End();
```

Testing

I could then repeat test PO6 and PO7, as these didn't pass I could not move onto the next tests:

Test ID	Description	Success criteria	Pass or Fail	Evidence
PO6	The simulation allows the user to add planets.	SC6	Fail - it is impossible to add planets when the sliders immediately return to their default positions.	EVIDENCE VIDEO 8
PO7	The simulation allows the user to customise planets.	SC1-5	Fail - it is impossible to customise planets when the sliders immediately return to their default positions.	EVIDENCE VIDEO 8

Review

Although this successfully rendered an ImGui window with the user interface controls, whenever I tried to use the sliders, they would instantly return to their default after releasing from the mouse. This was because I had defined the placeholder variables inside the render loop. This meant that every frame, the sliders were told to return to their default values. I aimed to fix this by including the placeholder values outside the main render loop.

Iteration 5

Build

In iteration 5 also wanted to calibrate the sliders so that they covered a suitable range of values. This is the code for the calibrated sliders:

```
C/C++  
ImGui::InputText("Planet Name", name, 128);  
ImGui::Text("Position:");
```

```

ImGui::SliderFloat("Position X: ", newPositionX, -500.0f, 500.0f);
ImGui::SliderFloat("Position Y: ", newPositionY, -350.0f, 350.0f);

ImGui::Text("Velocity:");
ImGui::SliderFloat("Velocity X: ", newVelocityX, -100.0f, 100.0f);
ImGui::SliderFloat("Velocity Y: ", newVelocityY, -100.0f, 100.0f);

ImGui::SliderFloat("Mass", &newMass, 600.0f, 3000.0f);

```

Testing

I then tested PO6 and PO7 again:

Test ID	Description	Success criteria	Pass or Fail	Evidence
PO6	The simulation allows the user to add planets.	SC6	Pass	EVIDENCE VIDEO 9
PO7	The simulation allows the user to customise planets.	SC1-5	Pass	EVIDENCE VIDEO 9

This meant I could move onto the other tests for the main loop code:

Test ID	Description	Success criteria	Pass or Fail	Evidence
PO8	The simulation has a user interface that allows the user to change the initial position of a planet with sliders.	SC3	Pass	EVIDENCE VIDEO 9
PO9	The simulation has a user interface that allows the user to change the initial velocity of a planet with sliders.	SC4	Pass	EVIDENCE VIDEO 9
PO10	The simulation has a user interface that allows the user to change the	SC2	Pass	EVIDENCE VIDEO 9

	mass of a planet with sliders.			
PO11	The simulation has a user interface which adds the customised planet to the window with the press of a button.	SC6	Pass	EVIDENCE VIDEO 9
PO12	The simulation has a user interface with a text input box for the name of the planet.	SC1	Pass	EVIDENCE VIDEO 9

Review and Adding Functionality

From these tests it is clear that putting these placeholder variables outside the main render loop made sure that the user interface worked successfully to add planets to the simulation. This can be seen in **EVIDENCE VIDEO 9**.

Despite all these tests passing, I wanted to fit this user interface control panel to the right side of the window. This is because in my design I set out to create the user interface like this and I didn't want the user being able to move the user interface window as they are using the simulator. To do this I set the ImGui window position using:

```
C/C++
ImGui::SetWindowPos(ImVec2(width - 300, 10));
ImGui::SetWindowSize(ImVec2(300, height));
```

This is included inside the ImGui render loop, before the text and sliders. The next step was to adjust the OpenGL view to take up only the left side. This involved modifying the OpenGL viewport to not include the right-most 300 pixels. I achieved this by using the ImGui `SetNextWindowPos`:

```
C/C++
ImGui::SetNextWindowPos(ImVec2(width - 300, 0), ImGuiCond_Always);
ImGui::SetNextWindowSize(ImVec2(300, height), ImGuiCond_Always);
```

However, even after implementing this, the window could still be dragged by the user, but it would just return to its original position on the release of the mouse. I therefore had to disable any movement from the ImGui window in order to achieve this. This involved adding a parameter to my `ImGui::Begin` function, which told it to disable window movement:

C/C++

```
ImGuiWindowFlags window_flags = ImGuiWindowFlags_NoMove;  
ImGui::Begin("Planet Creator", nullptr, window_flags);
```

I also wanted to increase the size of my window. I did this by editing the width and height constants which I had already defined. This can be seen in **EVIDENCE VIDEO 9/10**.

Testing

At this point I repeated all the necessary tests in my test plan again:

Test ID	Description	Success criteria	Pass or Fail	Evidence
PO1	The simulation successfully renders planets on the screen as circles.	SC14	Pass	EVIDENCE VIDEO 10
PO2	The simulation can successfully introduce velocity for planets in the simulation and render this as movement across the window.	SC16	Pass	EVIDENCE VIDEO 10
PO3	The simulation can successfully introduce acceleration for planets in the simulation and render this as increasing velocity for the planets on the screen.	SC16	Pass	EVIDENCE VIDEO 10
PO4	The simulation can move and accelerate two planets at once.	SC16	Pass	EVIDENCE VIDEO 10
PO5	The simulation successfully uses the planet class functions to give a visual representation of gravity between two planets in the window.	SC15	Pass	EVIDENCE VIDEO 10

PO6	The simulation allows the user to add planets.	SC6	Pass	EVIDENCE VIDEO 10
PO7	The simulation allows the user to customise planets.	SC1-5	Pass	EVIDENCE VIDEO 10
PO8	The simulation has a user interface that allows the user to change the initial position of a planet with sliders.	SC3	Pass	EVIDENCE VIDEO 10
PO9	The simulation has a user interface that allows the user to change the initial velocity of a planet with sliders.	SC4	Pass	EVIDENCE VIDEO 10
PO10	The simulation has a user interface that allows the user to change the mass of a planet with sliders.	SC2	Pass	EVIDENCE VIDEO 10
PO11	The simulation has a user interface which adds the customised planet to the window with the press of a button.	SC6	Pass	EVIDENCE VIDEO 10
PO12	The simulation has a user interface with a text input box for the name of the planet.	SC1	Pass	EVIDENCE VIDEO 10

Review

All the tests in my test plan for prototype 1 have passed. This means I have successfully created a working prototype that simulates 2D gravity and allows the user to add planets to the system. Something that I should have realised in my design, however, was the fact that I needed to fix the positioning of the user interface. This would have been helpful as I didn't have a test for this step in my development.

Adding Validation

Build

In order to add validation to the text box that is used for the planet name, I'm using if statements with a variable **ValidationError**. If the name inputted was longer than 20 characters, empty or a repeat of another name, instead of creating a planet with this name and the other selected features, this variable will be set to true and a message will be displayed.

I added this if statement into the add planet button:

```
C/C++
if (std::string(name).size() >= 20){ //if statement to check for input
validation errors
    ValidationError = true;
}
else if (nameRepeat){
    ValidationError = true;
}
else if (std::string(name).empty()){
    ValidationError = true;
}
else{
    //code for instantiating a planet....
}
```

Along with this I added an ImGui text beneath the input box code, which would display the message "Invalid input detected, name must be less than 20 characters, can't be empty and cannot be a repeat of another planet's name!"

Here is what that looked like:

```
C/C++
if (ValidationError){
    ImGui::Text("Invalid input detected, name must be less than 20
characters, can't be empty and cannot be a repeat of another planet's
name!");
}
```

Testing

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail
---------	-----------	-----------	-------------	----------	--------	------------------	-----------

VT1	Normal	"Earth"	These check that the textbox correctly accepts and doesn't accept values.	Accepts value	Accepts value	SC1	Pass
VT2	Normal	"Test 1"		Accepts value	Accepts value	SC1	Pass
VT3	Erroneous	" " - no input from the user		Doesn't accept value, displays invalid message	Doesn't accept value, no planet is created, but no error message is displayed.	SC1	Fail
VT4	Erroneous	Planet 1: "Earth" Planet 2: "Earth"		Doesn't accept second value, displays invalid message	Doesn't accept value, no planet is created, but no error message is displayed.	SC1	Fail
VT5	Erroneous	"Twenty Characters\$\$\$\$"		Doesn't accept value, displays invalid message	Doesn't accept value, no planet is created, but no error message is displayed.	SC1	Fail
VT6	Boundary	"01234567890123456789" - a string of 20 characters		Accepts value	Doesn't accept value, no planet is created, but no error message is displayed.	SC1	Fail

Review and Debugging

At this stage, I knew that the if statement within the add planet button was working because the planets were not being added to the environment if they didn't pass the validation checks. However, the message was not appearing. I found that it was because I had positioned the **ValidationError** variable inside my render loop. This meant that every frame it was being reset to 'true'. This meant that when I pressed the add planet button, the message was appearing for just a single frame, which was not enough time for me to see it. I fixed this and repeated tests VT3-6.

VT3	Erroneous	"" - no input from the user	These check that the textbox correctly accepts and doesn't accept values.	Doesn't accept value, displays invalid message	Doesn't accept value, displays invalid message	SC1	Pass
VT4	Erroneous	Planet 1: "Earth" Planet 2: "Earth"		Doesn't accept second value, displays invalid message	Doesn't accept value, displays invalid message	SC1	Pass
VT5	Erroneous	"Twenty Characters\$\$\$\$\$"		Doesn't accept value, displays invalid message	Doesn't accept value, displays invalid message	SC1	Pass
VT6	Boundary	"01234567890123456789" - a string of 20 characters		Accepts value	Doesn't accept value, displays invalid message	SC1	Pass

Review

The purpose of this stage was to create a working 2D simulation using the planet class I have already created. Since I was successful in doing this, I can move onto stage 4 of my development and integrate 3D elements into my simulation.

Stage 4: Going 3D

Prototype 3: Sphere

Explanation and Justification

Similarly to how I gradually built up the 2D simulation, I am going to first render a sphere in 3D without the effects of gravity, then build in the code I've already created such as the planet class.

I've got two reasons for this:

1. I can create the code for rendering a sphere in this prototype then bring this into the main program I create after.
2. Instead of trying to balance physics calculations and rendering in 3D, I can instead focus on the rendering in 3D now and the physics calculations in 3D later (in the main program).

This prototype will be successful if it passes test MLRT1.

Prototype 3 Version 1

Build

Rendering a sphere in OpenGL uses the same concept in 2D but with added aspects. In order to render a sphere, we must first make it into a list of vertices which can then be used to render triangles. This is achieved by using two angles (theta and phi), which iterate in different directions across the sphere and create points on it which can be rendered into triangles.

Firstly I coded a function that would generate a sphere. This firstly requires creating a circle with a 3D spherical coordinate system for each vertex and then converting these coordinates into a 3D cartesian coordinate system.

The **divisionsLat** and **divisionsLong** variables tell the function how many longitudinal and latitudinal divisions to make, with **divisionLat** representing latitudinal divisions and **divisionsLong** representing longitudinal divisions.

A 3D spherical coordinate system works by using 3 values. Radius, theta and phi. The radius defines the distance away from the origin. In this case the origin is the centre of the sphere. Theta is the longitudinal angle and phi is the latitudinal angle.

The **GenerateSphere()** function will create a list of the vertices of the sphere. This is achieved by using a for loop to iterate through the angle theta, while using another for loop within this to iterate through the angle phi.

Within each loop, two angles are iterated through to create the squares which then can be used to create two triangles, which can be rendered.

Therefore, each iteration of the loop uses 4 angles (theta1, theta2, phi1, phi2). These angles are then used with the radius to create 4 vertices in cartesian form. This is done by using the **sphericalToCartesian()** function which I have created to convert spherical coordinates to cartesian coordinates.

The last stage in the **GenerateSphere()** function is to create the two triangles and add them to the vertices vector.

```
C/C++
std::vector<float> GenerateSphere(float radius, int divisionsLat, int
divisionsLong)
{
    std::vector<float> vertices;
    vertices.reserve(divisionsLat * divisionsLong * 6 * 3); // 2 triangles
    per quad, 3 verts each

    for (int i = 0; i < divisionsLat; ++i) {
        float theta1 = (float)i / (float)divisionsLat * glm::pi<float>();
        float theta2 = (float)(i + 1) / (float)divisionsLat *
glm::pi<float>();

        for (int j = 0; j < divisionsLong; ++j) {
            float phi1 = (float)j / (float)divisionsLong *
glm::two_pi<float>();
            float phi2 = (float)(j + 1) / (float)divisionsLong *
glm::two_pi<float>();

            glm::vec3 v1 = sphericalToCartesian(radius, theta1, phi1);
            glm::vec3 v2 = sphericalToCartesian(radius, theta1, phi2);
            glm::vec3 v3 = sphericalToCartesian(radius, theta2, phi1);
            glm::vec3 v4 = sphericalToCartesian(radius, theta2, phi2);

            // Triangle 1: v1, v2, v3
            vertices.insert(vertices.end(), { v1.x, v1.y, v1.z,
                                              v2.x, v2.y, v2.z,
                                              v3.x, v3.y, v3.z });

            // Triangle 2: v2, v4, v3
            vertices.insert(vertices.end(), { v2.x, v2.y, v2.z,
                                              v4.x, v4.y, v4.z,
                                              v3.x, v3.y, v3.z });
        }
    }
    return vertices;
}
```

To create the **sphericalToCartesian()** function, I used the trigonometric functions cos and sin with the radius to determine the distance in each direction from the origin to the point:

```
C/C++
glm::vec3 sphericalToCartesian(float r, float theta, float phi)
{
    float x = r * sinf(theta) * cosf(phi);
    float y = r * cosf(theta);
    float z = r * sinf(theta) * sinf(phi);
    return glm::vec3(x, y, z);
}
```

After initialising GLFW as normal, I moved onto creating and compiling my shaders. I then have to link the two shaders into a single shader program which can be used in the rendering pipeline by the GPU. This is done using this code:

```
C/C++
unsigned int vertexShaderObj = glCreateShader(GL_VERTEX_SHADER);
glShaderSource(vertexShaderObj, 1, &vertexShader, NULL);
glCompileShader(vertexShaderObj);

unsigned int fragmentShaderObj = glCreateShader(GL_FRAGMENT_SHADER);
glShaderSource(fragmentShaderObj, 1, &fragmentShader, NULL);
glCompileShader(fragmentShaderObj);

unsigned int shaderProgram = glCreateProgram();
glAttachShader(shaderProgram, vertexShaderObj);
glAttachShader(shaderProgram, fragmentShaderObj);
glLinkProgram(shaderProgram);

glDeleteShader(vertexShaderObj);
glDeleteShader(fragmentShaderObj);
```

I then used my **GenerateSphere()** function to create the vertices of a sphere with radius 100.0f with 64 latitudinal divisions and 64 longitudinal divisions.

I then had to calculate the amount of vertices in the vector **sphere**. Using the .size() function gives the total amount of floats in this vector. Therefore, in order to obtain the amount of vertices, I need to divide this by 3, as this is the amount of components each vector has. As OpenGL expects this data as the GLsizei data type I have to convert the number I get from the previous calculation to this data type using **static_cast<GLsizei>**.

C/C++

```
float sphereRadius = 100.0f;
int divisionsLat = 64;
int divisionsLong = 64;
std::vector<float> sphere = GenerateSphere(sphereRadius, divisionsLat,
divisionsLong);
GLsizei sphereVertexCount = static_cast<GLsizei>(sphere.size() / 3);
```

I then had to create and activate the VBO and VAO; upload vertex data from the CPU to the GPU; tell the GPU how to interpret the data within the VBO and enable it so the shader program could access it.

C/C++

```
// Create VAO/VBO
GLuint vao = 0, vbo = 0;
glGenVertexArrays(1, &vao);
glGenBuffers(1, &vbo);
glBindVertexArray(vao);
glBindBuffer(GL_ARRAY_BUFFER, vbo);
glBufferData(GL_ARRAY_BUFFER, sphere.size() * sizeof(float), sphere.data(),
GL_STATIC_DRAW);
glEnableVertexAttribArray(0); // aPos
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float),
(void*)0);
glBindVertexArray(0);
```

Next I had to set the OpenGL state to deal with 3 dimensions by using the following code to allow OpenGL to render closer objects in 3D space in front of objects which are further away.

C/C++

```
glEnable(GL_DEPTH_TEST)
```

After this I had to define the geometric transformations of the 3D scene. Involved using the **glGetUniformLocation()** function to obtain the memory address of the uniform variable within the GPU's memory space for that shader program. The **glGetUniformLocation()** function takes two arguments:

1. ID of the shader program
2. A name containing the exact name of the uniform variable within the GPU's memory space.

The code retrieves the location of these 3 matrices that define the geometric transformation of the 3D space:

1. The Model Matrix - transforms the vertices of a 3D model from its local Object Space into the scene's global World Space

2. The View Matrix - transforms all vertices from World Space into the viewer's Camera Space
3. The Projection Matrix. Transforms vertices from Camera Space into the normalized 2D clipping volume (Clip Space), handling perspective or orthographic distortion.

C/C++

```
GLuint modelLoc = glGetUniformLocation(shaderProgram, "model");
GLuint viewLoc = glGetUniformLocation(shaderProgram, "view");
GLuint projLoc = glGetUniformLocation(shaderProgram, "projection");
```

I then had to set the perspective projection matrix. The first step to this was to calculate the ratio between the width and height of the window, to avoid geometric distortion of the window. The ratio is stored in the variable **aspect**.

I then used the GLM library's perspective function to create the 4x4 projection matrix. This function takes 4 arguments:

1. The vertical angle of the viewing cone. A smaller angle = a smaller field of view and vice versa. I've chosen 45 as this is the standard intermediate choice.
2. The aspect ratio - calculated in the previous line.
3. The near plane - anything less than 0.1f from the camera will be clipped and not rendered.
4. The far plane - anything farther from the camera than 1000.0f will be clipped and not rendered.

C/C++

```
float aspect = (float)width / (float)height;
glm::mat4 projection = glm::perspective(glm::radians(45.0f), aspect, 0.1f,
1000.0f);
```

Next was to set the camera position. This is achieved by using the **glm::lookAt()** function. This function takes 3 vectors as arguments:

1. The camera position - defines the position of the camera in world space.
2. Look at point - defines the target point the camera is looking at.
3. Up vector - defines which direction is considered up relative to the camera's view.

I've chosen to position the camera at (0, 0, 500); 500.0f, looking at the origin with the y-direction as the up vector.

This means that the camera will be looking down at the y-axis and x-axis perpendicular to each other. This is because the movement of my planets will take place in this 2D plane.

C/C++

```
glm::mat4 view = glm::lookAt(
    glm::vec3(0.0f, 0.0f, 500.0f), // Camera position
```

```

    glm::vec3(0.0f, 0.0f, 0.0f),    // Look at point
    glm::vec3(0.0f, 1.0f, 0.0f)    // Up vector
);

```

After this I created the render loop as I would normally and used what I had already created to render the sphere.

I had to set up shaders, camera, light position, view matrix and projection matrix first:

```

C/C++
glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
glUseProgram(shaderProgram);
glUniform3f(lightPosLoc, 200.0f, 200.0f, 200.0f);

glm::vec3 camPos(0.0f, 0.0f, 500.0f);
glUniform3f(viewPosLoc, camPos.x, camPos.y, camPos.z);
glUniform3f(lightColorLoc, 1.0f, 1.0f, 1.0f);

glUniformMatrix4fv(projLoc, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));

```

The first step of creating the sphere is to create a 4x4 Model Matrix and initialise it to the Identity Matrix which places the object at the world origin with no rotation or scaling before any transformations are applied. I then use the function **glUniformMatrix4fv()** to send the current model matrix data from the CPU to the GPU.

The next step is to set the color of the planet by accessing its location in memory and inputting RGB values. I've chosen a bright green, RGB: 0.5f, 0.8f, 0.2f.

I then had to bind the VAO to the active OpenGL state which loads the data for the sphere's vertices. This is done using the **glBindVertexArray()** function.

I had to then draw the sphere data which I had bound to the active OpenGL state. I also had to tell the program how to draw the vertices using **GL_TRIANGLES** and how many vertices there are (sphereVertexCount).

I then had to unbind the VAO using **glBindVertexArray(0)**.

The final two lines swap the back buffer with the current buffer on the window (**glfwSwapBuffers()**) and check for events that have occurred since the last frame (**glfwPollEvents()**).

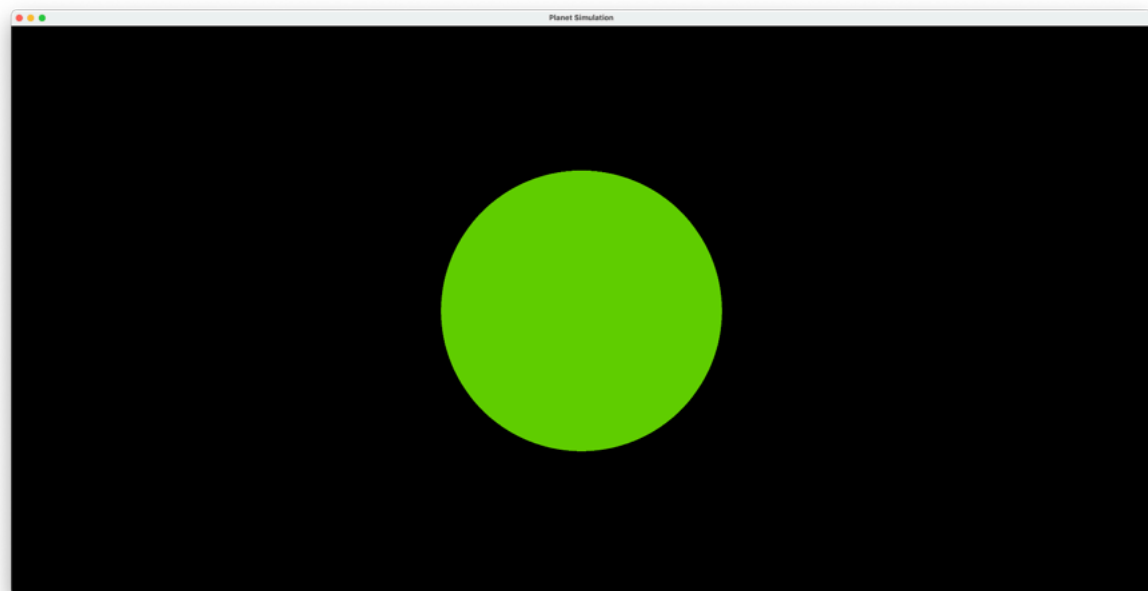
```
C/C++
glm::mat4 model = glm::mat4(1.0f);
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glUniform3f(glGetUniformLocation(shaderProgram, "color"), 0.5f, 0.8f, 0.2f);

glBindVertexArray(vao);
glDrawArrays(GL_TRIANGLES, 0, sphereVertexCount);
glBindVertexArray(0);

glfwSwapBuffers(window);
glfwPollEvents();
```

Testing

This is what it looks like in the window:



Here is the testing I carried out at this stage.

Test ID	Description	Success criteria	Pass or Fail	Evidence
MLRT 1	The simulation successfully renders planets on the screen as spheres.	SC17	Fail - this looks exactly how the 2D circle looked. Therefore, it does not successfully render a sphere.	

As it failed MLRT1, I did not carry on with the tests.

Review

As these tests did not pass I will have to carry out debugging. I realised at this stage that I had not changed my shaders, so that all pixels on the sphere were being rendered as a single colour.

To fix this I will have to introduce some lighting aspects that would make this look more spherical. To do this I will have to consider normals, the perpendicular vectors to a surface. These are used to tell the lighting system which direction a surface is facing, which determines how much light hits that surface. I therefore had to update my shaders to add normals.

Prototype 3 Version 2

Build

First, I had to update the vertex shader. I had to create the variable **FragPos** to hold the exact world coordinates of the vertex. This will be used by the fragment shader to determine the lighting of the triangle it is part of.

Second, I had to calculate the normal vector, storing it in the variable **Normal**. This uses the normalize function, which uses the fact that a sphere is centred at the origin to calculate the normal vector of the vertex.

Thirdly, I had to calculate the transformed position of the vertex in clip space (the 2D viewing window). The vertex position - given by Fragpos, which is transformed to a vec4 - is multiplied by the view and projection matrix to transform its coordinates into clip space. This is stored in the variable **gl_Position**.

```
C/C++
const char* vertexShader = R"(
    #version 330 core
    layout (location = 0) in vec3 aPos;

    out vec3 FragPos; //outputs the Fragpos to the fragment shader
    out vec3 Normal; //outputs the Normal to the fragment shader

    uniform mat4 model;
    uniform mat4 view;
    uniform mat4 projection;
    void main() {
        FragPos = vec3(model * vec4(aPos, 1.0));
        Normal = normalize(aPos);
        gl_Position = projection * view * vec4(FragPos, 1.0);
    }
);
```

Next was to update my fragment shader to deal with inputs from the vertex shader and use these to shade the sphere.

This starts by defining the outputs and inputs:

- **FragColor** - an output that declares the final output variable output for the fragment/pixel.
- **Normal** - input from the vertex shader, contains the normal vector.
- **FragPos** - input from vertex shader, contains position of current fragment.
- **lightPos** - input that contains position of light source.
- **viewPos** - input that contains position of camera.
- **lightColor** - input that contains color / intensity of the light source.
- **objectColor** - input that contains color of the object.

In the main function I had to calculate 3 lighting components:

1. Ambient lighting. This sets a minimum strength of light for the entire world space so that triangles facing away from the light source are not entirely black.
2. Diffuse shading. This calculates the directional light by calculating the direction from the fragment position to the light source. It then takes the dot product of the normal vector and light direction vector. Finally it calculates the final light color.
3. Specular shading. This starts by setting the intensity of the specular light. It then calculates the direction from the fragment to the camera and stores it in the variable **viewDir**. Next, it calculates the reflection vector (the direction light reflects off the surface) and stores it in the variable **reflectDir**. I then used the dot product to calculate the specular intensity and then calculated the final specular color.

The final step was to combine the light intensities I had calculated and use this to calculate the final color of the fragment, stored in the variable **FragColor**.

C/C++

```
const char* fragmentShader = R"(
    #version 330 core
    out vec4 FragColor;
    in vec3 Normal;
    in vec3 FragPos;

    uniform vec3 lightPos;
    uniform vec3 viewPos;
    uniform vec3 lightColor;
    uniform vec3 objectColor;

    void main() {
        // Ambient shading
        float ambientStrength = 0.2;
        vec3 ambient = ambientStrength * lightColor;

        // Diffuse shading
        vec3 norm = normalize(Normal);
        vec3 lightDir = normalize(lightPos - FragPos);
        float diff = max(dot(norm, lightDir), 0.0);
        vec3 diffuse = diff * lightColor;

        // Specular shading
```

```

        float specularStrength = 0.5;
        vec3 viewDir = normalize(viewPos - FragPos);
        vec3 reflectDir = reflect(-lightDir, norm);
        float spec = pow(max(dot(viewDir, reflectDir), 0.0), 32);
        vec3 specular = specularStrength * spec * lightColor;

        //Combine results
        vec3 result = (ambient + diffuse + specular) * objectColor;
        FragColor = vec4(result, 1.0);
    }
}";

```

After updating my shaders, I also had to get the ID of these variables by using the **glGetUniformLocation()** function. This is so they can be used later in the program:

```

C/C++
GLuint lightPosLoc = glGetUniformLocation(shaderProgram, "lightPos");
GLuint viewPosLoc = glGetUniformLocation(shaderProgram, "viewPos");
GLuint lightColorLoc = glGetUniformLocation(shaderProgram, "lightColor");
GLuint objectColorLoc = glGetUniformLocation(shaderProgram, "objectColor");

```

In addition to this I had to upload the position of my light source to the uniform variable in the active OpenGL shader program, in order to use it for lighting calculations. This was positioned inside the render loop:

```

C/C++
glUniform3f(lightPosLoc, 200.0f, 200.0f, 200.0f);

```

In the render loop I also had to draw the sphere. This used exactly the same code as the previous program, but I can now use the variable **objectColorLoc** which I have already defined instead of accessing this using the **glGetUniformLocation()** function:

```

C/C++
glm::mat4 model = glm::mat4(1.0f);
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));

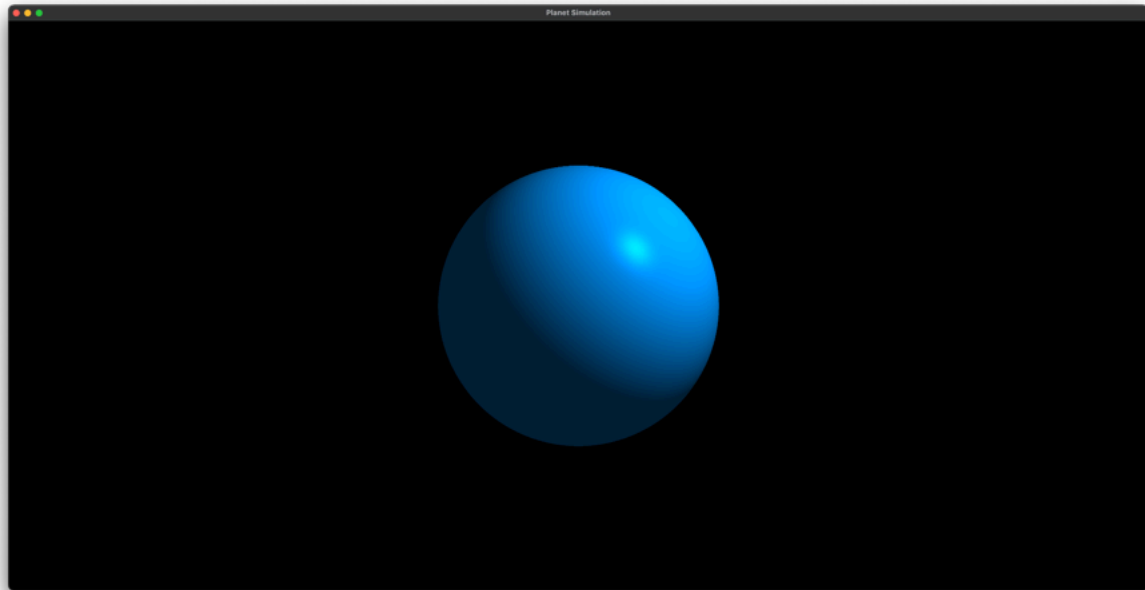
glUniform3f(objectColorLoc, 0.1f, 0.6f, 1.0f);

glBindVertexArray(vao);
glDrawArrays(GL_TRIANGLES, 0, sphereVertexCount);
glBindVertexArray(0);

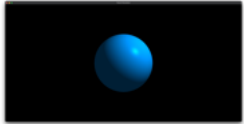
```

Testing

After adding all of this, I had a shaded sphere:



Here is the testing I carried out after this stage. I only carried out test MLRT1 because no movement has been introduced yet, so it would fail the tests after MLRT1:

Test ID	Description	Success criteria	Pass or Fail	Evidence
MLRT 1	The simulation successfully renders planets on the screen as spheres.	SC17	Pass	

Review

At this stage, I had a functioning prototype of a sphere which passed MLRT1. Therefore this prototype is a success and now I'm ready to implement this into a main program.

Main Program

Build

I then wanted to integrate the 3D aspect I had already created with my simulation.

I also had to change the way that my program dealt with projection. In order to render 3D graphics I would have to switch to using perspective projection. This is implemented by replacing my old Orthographic coordinate system with this:

C/C++

```
float aspect = (float)width / (float)height;
glm::mat4 projection = glm::perspective(glm::radians(45.0f), aspect, 0.1f,
1000.0f);
```

When rendering in 3D, an important aspect is the camera. This is where the simulation is viewed from in the 3D space. I set up the camera with the following code, I will calibrate its position after the simulation is working and I can adjust and test it:

C/C++

```
glm::mat4 view = glm::lookAt(
    glm::vec3(0.0f, 0.0f, 500.0f), // Camera position
    glm::vec3(0.0f, 0.0f, 0.0f),  // Look at point
    glm::vec3(0.0f, 1.0f, 0.0f)   // Up vector
);
```

After this, I had to edit all the coordinate data in my program. As planets would not be moving in 3D, I could set their z coordinate as 0.0f, which is a constant as I don't want it to change. This first involved me changing the vec2 struct to include a z component that is 0. Despite this struct now containing 3 components, I will still call it vec2 for now, to avoid me having to change this everywhere in the program. Because I had used this struct, I only had to make this change:

C/C++

```
struct vec2{
    float x;
    float y;
    float z = 0.0f;
};
```

I also had to edit the planet class. I added two methods to the planet class. These were the **sphericalToCartesian()** and **GenerateSphere()** functions. I also added two attributes to the class:

1. **sphereVertices** - stored the vertices of the planet in a `std::vector<float>` data type.
2. **sphereVertexCount** - stored the amount of vertices in the list of vertices. This is to be used when drawing the planets.

I added this code to the constructor method in order to initialise a planet with these two attributes:

C/C++

```
this->sphereVertices = GenerateSphere(64, 64);
```

```
this->sphereVertexCount = static_cast<int>(sphereVertices.size() / 3);
```

The sphereVertexCount constructor divides the size of sphereVertices by 3 because each vertex has 3 floats associated with it.

I then had to update my rendering code to render in 3D. The first step of this was adding the code that I had used to create the sphere with lighting. This meant adding the updated shaders to my program and defining these variables as I did when creating the sphere. I have also added model the list of things I need to send to my shaders, this is needed to transform my objects coordinates from worlds model space (local) into world space (3D scene):

C/C++

```
GLuint modelLoc = glGetUniformLocation(shaderProgram, "model");
GLuint viewLoc = glGetUniformLocation(shaderProgram, "view");
GLuint projLoc = glGetUniformLocation(shaderProgram, "projection");
GLuint viewPosLoc = glGetUniformLocation(shaderProgram, "viewPos");
GLuint lightColorLoc = glGetUniformLocation(shaderProgram, "lightColor");
GLuint objectColorLoc = glGetUniformLocation(shaderProgram, "objectColor");
```

I set up the render loop as I had done previously, this doesn't change apart from including my ImGui initialisation into it.

However, what does change is how I draw the spheres. Similarly to how I had drawn the 2D version I wanted to loop through the planets list and draw each one. Each line of code is as follows:

- Create the for loop, which iterates through every planet in the list
- Initialise the identity matrix
- Apply the scaling transformation to the planet
- Apply the position transformation to the planet
- Upload this calculated matrix from the CPU to the GPU
- Set the color
- Bind the VAO
- Draw the sphere
- Unbind the VAO

This code is exactly how I created the singular sphere, however I added the individual planets sphereVertexCount into the glDrawArrays function.

C/C++

```
for(int i = 0; i < planets.size(); i++){
    glm::mat4 model = glm::mat4(1.0f);
    model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
    model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
```

```

glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));

glUniform3f(objectColorLoc, 0.1f, 0.6f, 1.0f);

glBindVertexArray(VAO);
glDrawArrays(GL_TRIANGLES, 0, planets[i].sphereVertexCount);
glBindVertexArray(0);
}

```

Testing

Test ID	Description	Success criteria	Pass or Fail	Evidence
MLRT 1	The simulation successfully renders planets on the screen as spheres.	SC17	Pass - a 3D was rendered on the screen	EVIDENCE VIDEO 11
MLRT 2	The simulation can successfully introduce velocity for the planets in the simulation and render this as movement across the window in 3D.	SC16 SC18	Fail - the simulation allows the user to add a planet, but this planet stays put in the middle of the window and its size does not change.	EVIDENCE VIDEO 11

As test MLRT2 failed, I stopped testing here to fix this problem.

Review and Debugging

At first I thought this could be a problem with the list of planets because no matter how many times I pressed the add planet button, only the planet in the middle of the screen would show. However, after carrying this test out several times, using a different mass each time, I noticed that changing the mass did not change the radius of the rendered sphere. An error in the planets list would not cause this problem.

I therefore looked at my translation and scale matrices. If these did not take the correct inputs, the position and size of the planets would not be able to change. Looking at these revealed I was correct. Instead of using the size and position of the planets to create the

matrices, I was instead using the identity matrices, which meant that no scaling or translations would take place and the planet would have a fixed size positioned at the origin.

I updated these matrices to take in the current inputs:

```
C/C++  
model = glm::scale(model, glm::vec3(planets[i].radius));  
model = glm::translate(model, glm::vec3(planets[i].position.x,  
planets[i].position.y, planets[i].position.z));
```

Testing

This fixed the problem I was having, so I carried on testing:

Test ID	Description	Success criteria	Pass or Fail	Evidence
MLRT 2	The simulation can successfully introduce velocity for the planets in the simulation and render this as movement across the window in 3D.	SC16 SC18	Pass - the gravitation between the planets causes movement of the planets in 3D	EVIDENCE VIDEO 12
MLRT 3	The simulation can successfully introduce acceleration for planets in the simulation and render this as increasing velocity for the spheres on the screen.	SC16 SC18	Pass - the gravitation between the planets causes acceleration.	EVIDENCE VIDEO 12
MLRT 4	The simulation can move and accelerate two planets at once.	SC16	Pass	EVIDENCE VIDEO 12

MLRT 5	The simulation successfully uses the planet class functions to give a visual 3D representation of gravity between two planets in the window.	SC16	Pass	EVIDENCE VIDEO 12
MLRT 6	The simulation allows the user to add planets and renders them as spheres.	SC6	Pass	EVIDENCE VIDEO 12
MLRT 7	The simulation allows the user to customise planets.	SC1-4	Pass - all the ImGui sliders are working successfully.	EVIDENCE VIDEO 12
MLRT 8	The simulation has a user interface that allows the user to change the initial position of a planet with sliders.	SC3	Pass	EVIDENCE VIDEO 12
MLRT 9	The simulation has a user interface that allows the user to change the initial velocity of a planet with sliders.	SC4	Pass	EVIDENCE VIDEO 12
MLRT 10	The simulation has a user interface that allows the user	SC2	Pass	EVIDENCE VIDEO 12

	to change the mass of a planet with sliders.			
MLRT 11	The simulation has a user interface which adds the customised planet to the window with the press of a button.	SC6	Pass	EVIDENCE VIDEO 12
MLRT 12	The simulation has a user interface with a text input box for the name of the planet.	SC1	Pass	EVIDENCE VIDEO 12

Review

All of the tests for rendering the main loop code passed, as seen in **EVIDENCE VIDEO 12**. The aim of stage 4 was to create a working 3D simulation that I can then add features to. I have succeeded in creating a 3D simulator and can therefore move onto stage 5. The next step is to add functionality to my program such as trail paths and the space-time grid.

Stage 5: Adding Functionality

Trail Paths

Trail Paths Version 1

Build

I'm going to stray away from my design here slightly. In my design, I've included 3 methods: **addPoint()**, **getLastPoint()** and **deletePoint()**. However, I will merge these into a single method, using an if statement to compare the max length with current length. If the current length is less than the max length, no point will be deleted. If the current length is equal to the maximum length, the last point in the vector will be deleted. I've also realised that I don't need a class for each point in the path, but instead I can store each point as a float with each component and just use OpenGL to draw lines between adjacent points in the list.

Here is the TrailPath class:

```
C/C++
class TrailPath{
public:
    std::vector<float> TrailVertices;
    int maxLength;
    int currentLength;

    TrailPath(){
        this->maxLength = 10;
        this->currentLength = 0;
    }

    void AddPoint(float x, float y){
        if (currentLength < maxLength){
            TrailVertices.push_back(x);
            TrailVertices.push_back(y);
            TrailVertices.push_back(0.0f); // z-coordinate
            currentLength++;
        }
        else{
            //Remove oldest point
            TrailVertices.erase(TrailVertices.begin(), TrailVertices.begin()
+ 3);

            //Add new point
            TrailVertices.push_back(x);
            TrailVertices.push_back(y);
            TrailVertices.push_back(0.0f); // z-coordinate
        }
    }
}
```

Testing

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail

TP1	Normal	Input these vertices: (0,1) (0,2) (0,3) (0,4) (0,5) (0,6) (1,0) (2,0) (3,0) (4,0) (5,0) (6,0)	Here I will input 12 values. As the maximum size of the trail path is 10, when I print the path, only the last 10 should be output.	(0,3,0) (0,4,0) (0,5,0) (0,6,0) (1,0,0) (2,0,0) (3,0,0) (4,0,0) (5,0,0) (6,0,0)	(0,3,0) (0,4,0) (0,5,0) (0,6,0) (1,0,0) (2,0,0) (3,0,0) (4,0,0) (5,0,0) (6,0,0)	SC23	Pass
TP2	Normal	(0,1) (0,2) (0,3) (0,4) (0,5)	I will input just 5 vertices. When I print the path, these 5 vertices should be output.	(0,1,0) (0,2,0) (0,3,0) (0,4,0) (0,5,0)	(0,1,0) (0,2,0) (0,3,0) (0,4,0) (0,5,0)	SC23	Pass
TP3	Normal	(0,0) (0,0) (0,0) (0,0)	Testing with different data.	(0,0,0) (0,0,0) (0,0,0) (0,0,0)	(0,0,0) (0,0,0) (0,0,0) (0,0,0)	SC23	Pass

Build

As all these tests passed I could move onto implementing this class into my program and using OpenGL to render these lines.

After adding my class to the program, I then created a `std::vector` to store all my trail paths:

```
C/C++
std::vector<TrailPath> trails;
```

At this point, every planet will have a trail path, so I will instantiate a trail path for each planet within the ImGui if statement which is linked to the 'Add Planet' button. Each trail will be added to the **trails** list:

```
C/C++
if (ImGui::Button("Add Planet")) {
    vec2 newPosition = {newPositionX[0], newPositionY[0]};
    vec2 newVelocity = {newVelocityX[0], newVelocityY[0]};
    vec2 newAcceleration = {0.0f, 0.0f};
    Planet newPlanet(name, newMass, newPosition, newVelocity,
newAcceleration, true);
    planets.push_back(newPlanet);
    planets.back().sphereVertices = planets.back().GenerateSphere(64, 64);
    TrailPath trail;
    trails.push_back(trail);
}
```

Now I have to draw each of the trails that are in the list. Unlike planets which are all the same shape (spheres), each trail path will have a different VAO and VBO because they will be different shapes. I therefore had to create the VAO and VBO for each path before rendering them. Inside the for loop that draws the planets, I added this code:

```
C/C++
glGenVertexArrays(1, &VAO);
glGenBuffers(1, &VBO);
glBindVertexArray(VAO);

glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, trails[i].TrailVertices.size() *
sizeof(float), trails[i].TrailVertices.data(), GL_DYNAMIC_DRAW);

glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float),
(void*)0);
glEnableVertexAttribArray(0);

glUniform3f(objectColorLoc, 1.0f, 1.0f, 0.0f); // Yellow color for trail

glDrawArrays(GL_LINE_STRIP, 0, trails[i].currentLength);
glBindBuffer(GL_ARRAY_BUFFER, 0);
glBindVertexArray(0)
```

Testing

Test	Description	Success criteria	Pass or Fail	Evidence
------	-------------	------------------	--------------	----------

ID				
TPR1	The path of the planet can be rendered in the window successfully.	SC5	Fail - although the trail paths are rendered, the planets are not, therefore they are not rendered 'successfully'.	EVIDENCE VIDEO 13 - you can see the trail paths working but this stops the planets from being rendered.

Trail Paths Version 2

Build

To try and fix this error I looked at the VAO and VBO, as there must be a mismatch between when it is used for planets and when it is used for the trail path. The problem was that when I created a new VAO and VBO for the planet class, when I binded this to the active OpenGL state, the sphere VAO and VBO were deleted from this state and could no longer be used. Therefore, I created an entirely separate VAO and VBO for rendering the trail paths. I used the variables **vao** and **vbo** for simplicity. I then updated the code I had previously to this:

```
C/C++
unsigned int vao, vbo;
glGenVertexArrays(1, &vao);
glGenBuffers(1, &vbo);
glBindVertexArray(vao);

glBindBuffer(GL_ARRAY_BUFFER, vbo);
glBufferData(GL_ARRAY_BUFFER, trails[i].TrailVertices.size() *
sizeof(float), trails[i].TrailVertices.data(), GL_DYNAMIC_DRAW);

glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float),
(void*)0);
glEnableVertexAttribArray(0);

glUniform3f(objectColorLoc, 1.0f, 1.0f, 0.0f); // Yellow color for trail

glDrawArrays(GL_LINE_STRIP, 0, trails[i].currentLength);
glBindBuffer(GL_ARRAY_BUFFER, 0);
glBindVertexArray(0)
```

Testing

Test ID	Description	Success criteria	Pass or Fail	Evidence
---------	-------------	------------------	--------------	----------

TPR1	The path of the planet can be rendered in the window successfully.	SC5	Fail - both the planets and trail path are rendered, but again not successfully as the trail path is not in the correct place.	EVIDENCE VIDEO 14
------	--	-----	--	--------------------------

In this video you can see that the window renders the sphere and the trail path. However, the trail path does not correctly link with the planet's movement, but instead follows its own trajectory above the planet.

Trail Paths Version 3

Build

Despite looking several times at the for loop which updates the position of the planets and adds necessary points to the trail path, I could not solve this problem. I've therefore decided to take a new approach which I hoped would help. I'm going to include the vao and vbo, along with their initialisation into the trail path class. I also increased the max length so it is easier to see the trail path when testing.

This is what the updated trail path class looked like:

```
C/C++
class TrailPath{
public:
    std::vector<float> TrailVertices;
    int maxLength;
    int currentLength;
    unsigned int vao, vbo;

    TrailPath(){
        this->maxLength = 10000000;
        this->currentLength = 0;
    }

    void AddPoint(float x, float y){
        if (currentLength < maxLength){
            TrailVertices.push_back(x);
            TrailVertices.push_back(y);
            TrailVertices.push_back(0.0f); // z-coordinate
            currentLength++;
        }
        else{
            //Remove oldest point
        }
    }
}
```

```

        TrailVertices.erase(TrailVertices.begin(), TrailVertices.begin()
+ 3);
        //Add new point
        TrailVertices.push_back(x);
        TrailVertices.push_back(y);
        TrailVertices.push_back(0.0f); // z-coordinate
    }
}

void Init() {
    glGenVertexArrays(1, &vao);
    glGenBuffers(1, &vbo);
    glBindVertexArray(vao);
    glBindBuffer(GL_ARRAY_BUFFER, vbo);

    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float),
(void*)0);
    glEnableVertexAttribArray(0);

    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindVertexArray(0);
}

};

```

I then updated the rendering code inside the for loop:

```

C/C++
trails[i].Init(); // Initialize VAO and VBO for the trail
glBindVertexArray(trails[i].vao);
glBindBuffer(GL_ARRAY_BUFFER, trails[i].vbo);

// Update the buffer with new trail data
glBufferData(GL_ARRAY_BUFFER, trails[i].TrailVertices.size() *
sizeof(float), trails[i].TrailVertices.data(), GL_DYNAMIC_DRAW);

glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float),
(void*)0);
glEnableVertexAttribArray(0);

glUniform3f(objectColorLoc, 1.0f, 1.0f, 0.0f);
glDrawArrays(GL_LINE_STRIP, 0, trails[i].currentLength);

glBindVertexArray(0);

```

Testing

I carried out testing again at this stage, hoping that the changes I made would have fixed the problem.

Test ID	Description	Success criteria	Pass or Fail	Evidence
TPR1	The path of the planet can be rendered in the window successfully.	SC5	Fail - the trail path is still not rendering in the correct position.	EVIDENCE VIDEO 15

This works somewhat more successfully. It looks better but I suspect that is because I increased the max length of the trail path.

I tried several fixes to try and get this code to work including reversing the list of vertices so that they are added to the front and taken from the back. However, nothing seemed to be working.

The Space-Time Grid

The first step of creating the space-time grid was to create the class that would contain the space-time grid. It was in the development of this class that I realised I would not need a separate class for each vertex, as I can just store every vertex in a **std::vector**. I therefore decided that the attributes I need for the space-time grid are the **vertices** vector, the VAO and the VBO.

I also realised that I needed 4 methods, rather than the one I had thought I would need in the design stage. These are the methods I need:

- **MakeVBOVAO()** - this method will create the VAO and VBO of the grid which will have to be separate from the VAO and VBO used for the sphere, trail paths, etc.
- **DrawGrid()** - this is the method that will use the VAO and VBO I've created to render the grid in the window.
- **CreateGrid()** - this method will create the initial grid before any planets are added. At this point there will be no curvature in the grid.
- **UpdateGrid()** - this method will calculate the displacement of each vertex in the grid and update the list/vector of vertices accordingly.

Space-Time Grid Version 1

Build

The first version of the space-time grid will have no **UpdateGrid** method, as I simply want to render the grid in a window.

These are the attributes:

```
C/C++
class SpaceTimeGrid{
public:
    std::vector<float> vertices;
    GLuint VAO, VBO;
    size_t vertexCount;
```

The Create Grid method uses 4 constants which cannot change during the running of the program. These set the size and number of divisions of the space-time grid. Although these cannot change while running the program, I will change them in a later stage to calibrate the grid correctly.

Next, I use 2 for loops, 1 for each coordinate. They use the variable **halfsize** to loop from one side of the window to the other. My window centres the origin as the centre of the coordinate system. Therefore, in order to centre the grid, I have to loop from negative **halfsize** to position **halfsize**.

The variables **xStart**, **xEnd**, **yStart**, **yEnd** give the coordinates of each side of a grid cell. Each for loop creates a series of parallel lines. Combining the two for loops, which render the parallel lines perpendicular to each other gives me a grid.

The loops then add the coordinates to the back of the vertices list.

```
C/C++
void CreateGrid() {
    float size = 20000.0f;
    float divisions = 25;
    float step = size / divisions;
    float halfSize = 20000.0f/2.0f;

    //x axis
    for (int yStep = 0; yStep <= divisions; ++yStep) {
        float y = -halfSize + yStep * step; //Start at -halfSize and
        increment by step size as grid is centred at origin
        for (int xStep = 0; xStep < divisions; ++xStep) {
            float xStart = -halfSize + xStep * step; //Start at
            -halfSize and increment by step size as grid is centred at origin
            float xEnd = xStart + step;
            vertices.push_back(xStart); vertices.push_back(y);
            vertices.push_back(z);
            vertices.push_back(xEnd);    vertices.push_back(y);
            vertices.push_back(z);
        }
    }
```

```

    }

    //y axis
    for (int xStep = 0; xStep <= divisions; ++xStep) {
        float x = -halfSize + xStep * step; //Start at -halfSize and
increment by step size as grid is centred at origin
        for (int yStep = 0; yStep < divisions; ++yStep) {
            float yStart = -halfSize + yStep * step; //Start at
-halfSize and increment by step size as grid is centred at origin
            float yEnd = yStart + step;
            vertices.push_back(x); vertices.push_back(yStart);
vertices.push_back(z);
            vertices.push_back(x); vertices.push_back(yEnd);
vertices.push_back(z);
        }
    }

    vertexCount = vertices.size() / 3; //Divide by 3 because each
vertex has 3 components (x,y,z)

    baseVertices = vertices; //Store the original grid vertices for
resetting purposes

}

}

```

The next method is the **MakeVBOVAO** method, to create a VBO and VAO so the vertex data can be sent to the GPU.

It starts by generating a VAO and VBO and storing them in the variables **VAO** and **VBO**.

It then binds the VAO, making it active in the OpenGL environment.

Then I bind the VBO using the **GL_ARRAY_BUFFER** target. This tells OpenGL that this buffer will hold position coordinates.

The **glBufferData** function allocates GPU memory and uploads the vertex data to the VBO. I've used the hint **GL_DYNAMIC_DRAW** to tell OpenGL that these positions will be changing. This is because when planets are introduced, the vertices will constantly be moving in response to the position of the planets.

I've used **glVertexAttribPointer** to tell OpenGL how to interpret the data I've uploaded to the GPU. Here is an explanation of each of the parameters:

- 0 - the attribute index in the shader is at location 0.
- 3 - the number of components for each vertex. I'm working in 3D, so this is 3.

- **GL_FLOAT** - the components are floats.
- **GL_FALSE** - we don't want to normalize the values given.
- **3*sizeof(float)** - tells the function the distance in bytes between the vertices.
- **(void*)0** - tells OpenGL to start at the beginning position in the VBO.

C/C++

```
void MakeVBOVAO(){
    glGenVertexArrays(1, &VAO);
    glGenBuffers(1, &VBO);

    glBindVertexArray(VAO);
    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    glBufferData(GL_ARRAY_BUFFER, vertices.size() * sizeof(float),
vertices.data(), GL_DYNAMIC_DRAW);

    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float),
(void*)0);
    glEnableVertexAttribArray(0);
    glBindVertexArray(0);
}
```

The **DrawGrid** method uses the same principles as drawing planets, but uses the active VBO and VAO in the OpenGL environment. It is important to remember that inside my rendering loop, I put the **DrawGrid** function after creating the VAO and VBO for the grid. I've used comments to describe what each line of code does:

C/C++

```
void DrawGrid(GLuint shaderProgram) {
    glUseProgram(shaderProgram); // Tells OpenGL to use the shader program
    glm::mat4 model = glm::mat4(1.0f); // Identity matrix for the grid
    // Get the location of the 'model' uniform variable in the shader
    GLint modelLoc = glGetUniformLocation(shaderProgram, "model");
    // Uploads the model matrix to the shader's 'model' uniform
    glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
    // Binds the VAO
    glBindVertexArray(VAO);
    glLineWidth(1.0f); // Maximum line length - 1
    glDrawArrays(GL_LINES, 0, vertices.size() / 3); // vertexCount is
number of floats, divide by 3 for vertices
    // Unbinds the VAO
    glBindVertexArray(0);
}
```

I then added this code to the main rendering loop code, in order to draw the grid:

```
C/C++
grid.CreateGrid();
grid.MakeVB0VA0();
grid.DrawGrid(shaderProgram);
```

Testing

I started by carrying out test STGA1 to test if the class was generating the vertices correctly.

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail
STGA1	Normal	Step = 800 Divisions = 25	I will get the method to output the position of each of the vertices.	I will look at the final 5 vertices. I'm expecting all the z values to be -3000 through the whole list. For the end vertices I expect all the x coordinates to be 10000, as this will be the largest x value. I'm expecting that the y coordinates will vary in steps of 800, as this is the step of the grid. I'm expecting that there will be two of each vertex because I'm rendering in squares, which means there will be shared corners.	As expected, evidenced below	SC20	Pass - all the necessary vertices were there as expected.

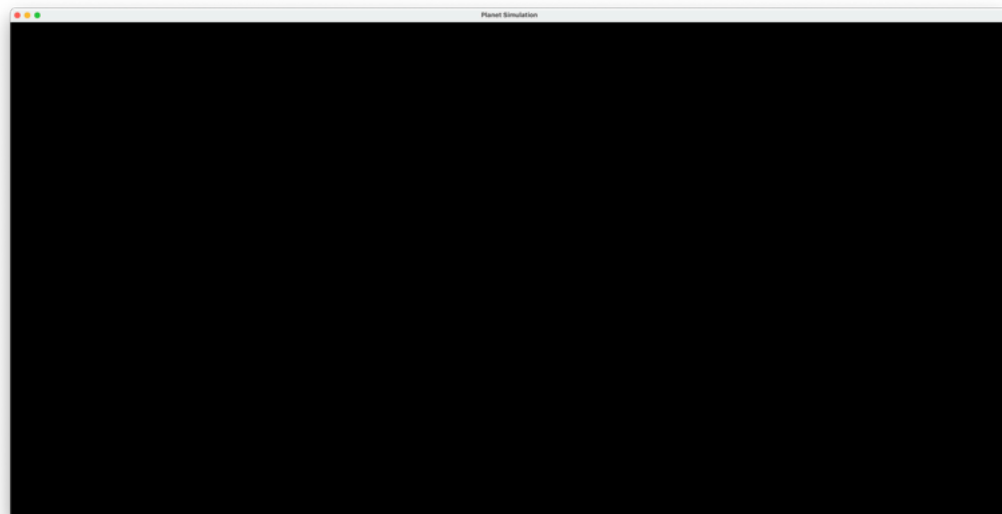
Here is the evidence of the vertices being printed in my terminal for STGA1.

```
Vertex 2588: (10000, 5200, -3000)
Vertex 2589: (10000, 6000, -3000)
Vertex 2590: (10000, 6000, -3000)
Vertex 2591: (10000, 6800, -3000)
Vertex 2592: (10000, 6800, -3000)
Vertex 2593: (10000, 7600, -3000)
Vertex 2594: (10000, 7600, -3000)
Vertex 2595: (10000, 8400, -3000)
Vertex 2596: (10000, 8400, -3000)
Vertex 2597: (10000, 9200, -3000)
Vertex 2598: (10000, 9200, -3000)
Vertex 2599: (10000, 10000, -3000)
Dauids-iMac:NEA_Draft_1 macuser$
```

I then carried out test STGR1:

Test ID	Description	Success criteria	Pass or Fail	Evidence
STGR 1	The grid appears in the window successfully and with a clear grid shape.	SC20	Fail - the grid was not rendered, all I saw was a black screen.	Evidence in the screenshot below.

EVIDENCE:



As the accuracy tests passed but the rendering tests didn't, this suggested that something was wrong with the functions I've been using for the rendering of the grid.

I therefore moved onto version 2 of the grid.

Space-Time Grid Version 2

Build

For version 2, I changed the **DrawGrid** method, to take in **objectColorLoc** as a parameter. I needed this so the method can set the shader's object color. I then used this within the

method to set a color for the grid. This is because I thought the problem could have been that the color of the grid was set to black.

Here is the updated **DrawGrid** method:

```
C/C++
void DrawGrid(GLuint shaderProgram, GLuint objectColorLoc) {
    glUseProgram(shaderProgram);

    glm::mat4 model = glm::mat4(1.0f);
    GLint modelLoc = glGetUniformLocation(shaderProgram, "model");
    glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));

    //I added this line of code:
    glUniform4f(objectColorLoc, 1.0f, 1.0f, 1.0f, 0.25f);

    glBindVertexArray(VA0);
    glLineWidth(10.0f);
    glDrawArrays(GL_LINES, 0, vertexCount);
    glBindVertexArray(0);
}
```

Testing

I repeated STGR1 at this point to test if these changes had fixed my problem.

Test ID	Description	Success criteria	Pass or Fail	Evidence
STGR 1	The grid appears in the window successfully and with a clear grid shape.	SC20	Fail - the grid was not rendered, all I saw was a black screen.	Same blank window as before: 

Space-Time Grid Version 3

Build

I next looked towards the fragment shader because I thought that the fragment shader could be shading the grid in a way that stopped it from being visible. This is because I have created the fragment shader to shade spheres, not lines. I therefore updated the fragment shader so that the color of any objects are not altered by the shader. It is important to know that this is not a permanent change, but just a temporary one while I'm creating the grid.

Here is the updated fragment shader, you can see that I've removed all code in the main function and assigned the **FragColor** as the color of the object:

```
C/C++
const char* fragmentShader = R"(
    #version 330 core
    out vec4 FragColor;
    in vec3 Normal;
    in vec3 FragPos;

    uniform vec3 lightPos;
    uniform vec3 viewPos;
    uniform vec3 lightColor;
    uniform vec4 objectColor;

    void main() {
        FragColor = objectColor;
    }
);
```

Testing

After making these changes, I repeated test STGR1:

Test ID	Description	Success criteria	Pass or Fail	Evidence
STGR 1	The grid appears in the window successfully and with a clear grid shape.	SC20	Pass - the grid was successfully rendered.	EVIDENCE SCREENSHOT 1

This worked and after this I had a successfully rendered grid. When I tried to upload a screenshot of this onto google docs, however, it wasn't very clear. This can therefore be seen in **EVIDENCE SCREENSHOT 1**.

I didn't move onto the next test because I hadn't introduced planets to the space-time grid program and therefore knew that it would fail any later tests.

Space-Time Grid Version 4

Build

Now I had a working grid, I wanted to introduce planets to it. The first step to this was to instantiate planets while also rendering the grid.

This involved adding the space-time class to the main program; instantiating the grid inside the **main()** function and drawing the grid inside the rendering loop.

Testing

After doing this, I tested how this functioned:

Test ID	Description	Success criteria	Pass or Fail	Evidence
STGR 1	The grid appears in the window successfully and with a clear grid shape.	SC20	Pass	EVIDENCE VIDEO 16
STGR 2	The grid renders alongside the planets.	SC20	Fail - no planets were rendered.	EVIDENCE VIDEO 16

The space-time grid worked successfully, however, the planets would not render. Therefore, STGR2 failed and I could not move onto the next tests.

Build

At this point, I decided to change my VAO and VBO variable names. When I was just rendering one type of object, using the variable names VAO and VBO was suitable. Now, however, it is slightly confusing. To clear this up, I will now call the planet VAO and VBO **planetVAO** and **planetVBO** respectively.

In doing so, I hoped the solution to my problem would become more clear. As the grid VAO and VBO are stored with the class, I will not change this as it is clear which VAO and VBO are being talked about.

This didn't reveal the problem, but nevertheless I found the problem in my for loop which was drawing the planets. In this I was applying my scaling matrix before the translation matrix. This could potentially be off-screen. I therefore reversed the order to try and fix this. I came to the conclusion that this didn't matter before the space-time grid before there was no static object for the planets to be compared to. Therefore it didn't matter what order I applied these matrices, as the only consequences would have minor visual differences.

After applying these changes I repeated test STGR2. The simulation worked successfully with the grid in the background, this can be seen in **EVIDENCE VIDEO 17**. In this video, the planets are rendered as 2D circles because of the temporary changes to my fragment shader.

Testing

Test ID	Description	Success criteria	Pass or Fail	Evidence
STGR 2	The grid renders alongside the planets.	SC20	Pass - planets and space-time grid both rendered.	EVIDENCE VIDEO 17

Space-Time Grid Version 5

Build

However, because of the changes I made to the fragment shader in the development of the space-time grid, my spheres now appeared as circles again. In order to change this I need to make changes to the fragment shader so that it treats the grid and planets differently.

This involved introducing the uniform variable **objectType** into my fragment shader. If this variable is given the value 0, then the fragment shader doesn't apply any shading to it. If the value is 1, the object is treated as a sphere and the fragment shader applies the necessary shading.

I achieved this by using an if statement in the main function of the fragment shader and using the code I had used in my previous 3D fragment shader for the sphere shading. This is the final code:

```
C/C++
const char* fragmentShader = R"(
    #version 330 core
    out vec4 FragColor;
    in vec3 Normal;
    in vec3 FragPos;

    uniform vec3 lightPos;
    uniform vec3 viewPos;
    uniform vec3 lightColor;
    uniform vec4 objectColor;
```

```

uniform int objectType; // 0 for grid, 1 for planet

void main() {
    if (objectType == 0){
        FragColor = objectColor;
    }

    else if (objectType == 1){
        // Ambient shading
        float ambientStrength = 0.2;
        vec3 ambient = ambientStrength * lightColor;

        // Diffuse shading
        vec3 norm = normalize(Normal);
        vec3 lightDir = normalize(lightPos - FragPos);
        float diff = max(dot(norm, lightDir), 0.0);
        vec3 diffuse = diff * lightColor;

        // Specular shading
        float specularStrength = 0.5;
        vec3 viewDir = normalize(viewPos - FragPos);
        vec3 reflectDir = reflect(-lightDir, norm);
        float spec = pow(max(dot(viewDir, reflectDir), 0.0), 32);
        vec3 specular = specularStrength * spec * lightColor;

        //Combine results
        vec3 result = (ambient + diffuse + specular) * objectColor;
        FragColor = vec4(result, 1.0);
    }
}

```

The next stage was to use this updated fragment shader in my c++ program. To do this I had to change the **objectType** variable depending on what was being rendered. This should have been as simple as adding the line:

```

C/C++
glUniform1i(glGetUniformLocation(shaderProgram, "objectType"), 1);

```

With the 1 being interchangeable with a 0 depending on whether a planet or the grid was being rendered.

Testing

I therefore carried out these tests again as I had updated the fragment shader, which could affect the grid as well as the planets:

Test ID	Description	Success criteria	Pass or Fail	Evidence
STGR 1	The grid appears in the window successfully and with a clear grid shape.	SC20	Pass	EVIDENCE VIDEO 18
STGR 2	The grid renders alongside the planets.	SC20	Fail - no planets were rendered.	EVIDENCE VIDEO 18

This worked for the grid and it was successfully rendered. However, planets would not render using this. This can be seen in **EVIDENCE VIDEO 18**.

Space-Time Grid Version 6

Build

As the VAO and VBO of the space-time grid are stored within the class, and the rendering for this was working, I wanted to reproduce that with the planet class. I therefore added the methods **DrawPlanet** and **MakeVBOVAO** to my planet class. Here is what they looked like, these methods take the code I was using in the main loop and package it into a method. I also had to add a destructor method for cleanup.

```
C/C++
void MakeVBOVAO(){
    glGenVertexArrays(1, &VAO);
    glGenBuffers(1, &VBO);
    glBindVertexArray(VAO);
    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    glBufferData(GL_ARRAY_BUFFER, sphereVertices.size() * sizeof(float),
sphereVertices.data(), GL_STATIC_DRAW);

    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float),
(void*)0);
    glEnableVertexAttribArray(0);
    glBindVertexArray(0);
}

void DrawPlanet(GLuint shaderProgram, GLuint objectColorLoc) {
    glUseProgram(shaderProgram);

    glm::mat4 model = glm::mat4(1.0f);
```

```

        model = glm::translate(model, glm::vec3(position.x, position.y,
position.z));
        model = glm::scale(model, glm::vec3(radius));
        GLuint modelLoc = glGetUniformLocation(shaderProgram, "model");

        glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));

        glUniform3f(objectColorLoc, 0.1f, 0.6f, 1.0f);
        glUniform1i(glGetUniformLocation(shaderProgram, "objectType"), 0); //
Set objectType to 1 for planet

        glBindVertexArray(VAO);
        glDrawArrays(GL_TRIANGLES, 0, sphereVertexCount);
        glBindVertexArray(0);
    }

    ~Planet() {
        glDeleteVertexArrays(1, &VAO);
        glDeleteBuffers(1, &VBO);
    }

```

The purpose of this was not to fix my problem but instead to make it much easier to navigate my code.

I eventually found that my camera position was the reason for my error. I was using the Y direction as my up vector. This meant I was looking down on the grid's edge and I couldn't see it. I therefore fixed this and updated the camera position to give a better view of the 3D element of the simulator:

```

C/C++
glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
glUseProgram(shaderProgram);
glUniform3f(lightPosLoc, 0.0f, 3000.0f, 0.0f);
glUniform3f(lightColorLoc, 1.0f, 1.0f, 1.0f);
glUniform3f(viewPosLoc, 15000.0f, 15000.0f, 15000.0f);

glm::vec3 cameraPos(15000.0f, 15000.0f, 15000.0f);
glm::vec3 cameraLookAt(0.0f, 0.0f, 0.0f);
glm::vec3 cameraUp(0.0f, 0.0f, 1.0f);
glm::mat4 view = glm::lookAt(cameraPos, cameraLookAt, cameraUp);
GLint viewLoc = glGetUniformLocation(shaderProgram, "view");
glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));

```

Testing

Test ID	Description	Success criteria	Pass or Fail	Evidence
STGR 1	The grid appears in the window successfully and with a clear grid shape.	SC20	Pass	EVIDENCE VIDEO 19
STGR 2	The grid renders alongside the planets.	SC20	Pass - planets could be rendered with the grid.	EVIDENCE VIDEO 19

Both of these tests passed so I moved on to the next stage in development of the space-time grid.

Space-Time Grid Version 7

Build

The next step in creating the space-time grid was to create the **UpdateGrid** method. This method is going to take the **planets** vector as a parameter. The **UpdateGrid** method will calculate displacement for each vertex of the original flat grid, so I will store the vertices of the flat grid as an attribute in my space-time grid class.

The function loops through each vertex and calculate the displacement from each planet using the equation:

$$\text{Displacement} = -k * (\text{mass}/[\text{distance}^2])$$

Where:

- k is a constant I can change later to calibrate the displacement

It then adds the displacement from each planet onto the total displacement for each vertex.

```
C/C++
void UpdateGrid(std::vector<Planet>& planets){
    vertices = baseVertices;

    float k = 240000.0f; // Gravitational constant for grid distortion
    float c = 1000.0f;    // Damping factor to control distortion

    for (int i = 0; i < vertices.size(); i += 3){
        float x = vertices[i];
        float y = vertices[i+1];
```

```

        float z = vertices[i+2];

        float totalDisplacement = 0.0f;

        for(int j = 0; j < planets.size(); j++){
            float dx = x - planets[j].position.x;
            float dy = y - planets[j].position.y;
            float distance = sqrt(dx*dx + dy*dy); // Avoid division
by zero

            float displacement = (-k * planets[j].mass) /
(pow(distance, 2) + c); // Gravitational distortion formula
            totalDisplacement += displacement;
        }

        vertices[i+2] = z + totalDisplacement; // Update
z-coordinate based on total displacement
    }
}

```

Within the rendering loop, I had to add these 2 lines of code:

```

C/C++
grid.UpdateGrid(planets);
grid.MakeVB0VA0();

```

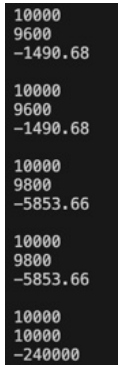
Testing

I carried out the STGR4 test here and skipped STGR3, as I don't have a working trail path in this iteration of my project.

Test ID	Description	Success criteria	Pass or Fail	Evidence
STGR 4	The grid vertices change their displacement depending on the distance from massive planets.	SC21 SC22	Pass - the grid successfully gave a visual representation of space-time, with each vertex displaced more with massive	EVIDENCE VIDEO 20

			objects closer.	
--	--	--	-----------------	--

I also carried out STGU1 to test that each of the vertices were being calculated correctly.

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail	EVIDENCE
STGU1	Normal	Position of planet: (10000, 10000) Mass of planet: 1000	This ensures that the displacement of the vertices is calculated correctly. If this is incorrect it could lead to an inaccurate depiction of space-time .	Vertex 1 coordinates: (10000, 10000, -240000) Vertex 2 coordinates: (10000, 9800, -5853.66) Vertex 3 coordinates: (10000, 9600, -1490.68)	As expected	SC21 SC22	Pass	

Trail Paths Version 4

Build

At this stage, I wanted to return to the trail path as I was confident I could get it working. After taking another look at how I had tried to create it, I realised I should be using a draw method within the trail path instead of using OpenGL functions within the main rendering loop to do this. I therefore added this **DrawTail** method to the trail class:

```
C/C++
void DrawTrail(unsigned int shaderProgram, int objectColorLoc) {
    glUniform4f(objectColorLoc, 1.0f, 1.0f, 0.0f, 1.0f); // Yellow color
    for trail
        glBindVertexArray(vao);
        glDrawArrays(GL_LINE_STRIP, 0, currentLength);
        glBindVertexArray(0);
}
```

Next I wanted to use composition to include a trail into each of my planets. I did this by creating a trail attribute. I wanted to do this so it was easier to keep track of every trail, by including it within its planet. This is the updated attributes for my planet class:

```
C/C++
class Planet{
public:
    std::string name;
    float mass;
    vec2 position;
    vec2 velocity;
    vec2 acceleration;
    bool trailPath;
    float radius;
    int sphereVertexCount;
    std::vector<float> sphereVertices;
    unsigned int VAO, VBO;
    TrailPath trail;
```

In the main render loop I can now run these lines of code to render each trail. These lines are within the loop that renders the planets:

```
C/C++
planets[i].trail.AddPoint(planets[i].position.x, planets[i].position.y);
planets[i].trail.UpdateBuffer();
planets[i].DrawPlanet(shaderProgram, objectColorLoc);
```

Next, I wanted to render the trail paths a different colour to the space-time grid, I therefore added a condition to the fragment shader to colour the trail paths yellow and included this line of code inside the **DrawTrail** method to tell the fragment shader to colour it this way:

```
C/C++
glUniform1i(glGetUniformLocation(shaderProgram, "objectType"), 2);
```

Testing

Test ID	Description	Success criteria	Pass or Fail	Evidence
TPR1	The path of the planet can be rendered in the window successfully.	SC5	Fail - the paths bugged and did not effectively show the path of the planet.	EVIDENCE VIDEO 21

Trail Paths Version 5

Build

To try and fix the problems I was having I looked towards the loop that renders the trail paths. Instead of using the loop that renders the planets to update and draw the trail paths, I used a separate loop. Here is what the loop looked like:

```
C/C++
for(int i = 0; i < planets.size(); i++){
    planets[i].trail.AddPoint(planets[i].position.x,
    planets[i].position.y);
    planets[i].trail.UpdateBuffer();
    planets[i].trail.DrawTrail(shaderProgram, objectColorLoc);
}
```

Testing

After doing this I repeated my test TPR1 and carried out tests TPR2 and TPR3.

Test ID	Description	Success criteria	Pass or Fail	Evidence
TPR1	The path of the planet can be rendered in the window successfully.	SC5	Pass	EVIDENCE VIDEO 22
TPR2	This works alongside the space-time grid successfully.	SC23	Pass	EVIDENCE VIDEO 22
TPR3	The path has a maximum length, and when it reaches this maximum length, the points at the end disappear.	SC24	Pass - in the video the points at the end of the path get taken away gradually.	EVIDENCE VIDEO 22

Trail Paths Version 6

Build

The next step in fulfilling my design is allowing the user to choose whether or not they want a planet to have a trail path.

The first step to implement this is to add the boolean variable **WithTrailPath** to my ImGui defaults, this started as false. I added a checkbox to my other controls that would change the default variable to true when ticked. I then added an if statement within my add planet if statement which would only initialise the trail path if this checkbox was ticked. Here is how the ImGui checkbox and updated if statement look:

```
C/C++
ImGui::Checkbox("Trail Path", &WithTrailPath);

if (ImGui::Button("Add Planet")) {
    vec2 newPosition = {newPositionX[0], newPositionY[0]};
    vec2 newVelocity = {newVelocityX[0], newVelocityY[0]};
    vec2 newAcceleration = {0.0f, 0.0f};
    Planet newPlanet(name, newMass, newPosition, newVelocity,
newAcceleration, true);
    planets.push_back(newPlanet);
    planets.back().sphereVertices = planets.back().GenerateSphere(64, 64);
    planets.back().MakeVBOVAO();
    if (WithTrailPath){
        planets.back().trail.Init();
        planets.back().trail.AddPoint(newPositionX[0],newPositionY[0]);
    }
}
```

Testing

I then carried out the final test for rendering the trail paths:

Test ID	Description	Success criteria	Pass or Fail	Evidence
TPR4	The user can choose whether or not they want a trail path for each planet and this dictates whether or not the trail path is shown.	SC5	Pass	EVIDENCE VIDEO 23

Review

The purpose of stage 5 was to add a working 2D space-time grid and trail paths to the simulation, as outlined in my design. I successfully created the classes for these which are largely based on the class definitions in my design. However, I did need to make a few tweaks in the development process. That being said, both these features work successfully and pass all relevant tests so I can consider stage 5 a success and move onto stage 6.

Stage 6: Usability Features

Planet Names / Labels

Build

To create the labels for my planets, I started with creating the label class. This label class requires 3 methods:

1. **Label()** - this constructor method will initialise the label when it is created.
2. **UpdateLabelPosition()** - updates the position of the label on the viewport.
3. **DrawLabel()** - uses ImGui functions to draw the label.

Here is how the label class looks:

```
C/C++
class Label{
public:
    std::string text;
    float x;
    float y;

    Label(std::string text = "", const glm::mat4& projection =
glm::mat4(1.0f), const glm::mat4& view = glm::mat4(1.0f), float planetX =
0.0f,
        float planetY = 0.0f){
        this->text = text;
        this->x = 0.0f;
        this->y = 0.0f;
        UpdateLabelPosition(projection, view, planetX, planetY);
    }

    void UpdateLabelPosition(const glm::mat4& projection, const
glm::mat4& view, float planetX, float planetY) {
        glm::vec4 clipSpacePos = projection * view * glm::vec4(planetX,
planetY, 0.0f, 1.0f);
        glm::vec3 ndcPos = glm::vec3(clipSpacePos) / clipSpacePos.w;

        x = (ndcPos.x * 0.5f + 0.5f) * width;
        y = (1.0f - (ndcPos.y * 0.5f + 0.5f)) * height;
    }
}
```

```

        void DrawLabel(ImGuiWindowFlags labelFlags) {
            ImGui::SetNextWindowPos(ImVec2(x, y), ImGuiCond_Always);
            ImGui::Begin(text.c_str(), nullptr, labelFlags);
            ImGui::Text("%s", text.c_str());
            ImGui::End();
        }
};

```

The next step to implement the labels was to use composition and add a label attribute to each planet. In the planet constructor method, I added this line, to assign the label's text attribute immediately when the planet is instantiated:

```

C/C++
this->label.text = name;

```

Next, I added this to the add planet if statement in order to run the labels constructor:

```

C/C++
planets.back().label = Label(name, projection, view, newPosition.x,
newPosition.y);

```

I then looped through each planet, drawing its label, using this for loop:

```

C/C++
for (int i = 0; i < planets.size(); i++){
    planets[i].label.UpdateLabelPosition(projection, view,
planets[i].position.x, planets[i].position.y);
    planets[i].label.DrawLabel(labelFlags);
}

```

Testing and Debugging

Initially this gave me an output that did not align with what I wanted. Each window was a default ImGui window which meant that it had the following undesirable features:

1. Titlebar
2. It could be resized by the user.
3. It started as a window that was too big.
4. The window had a scrollbar.

I therefore modified the **labelFlags** variable to introduce flags that would stop these features from occurring:

```
C/C++
ImGuiWindowFlags labelFlags = ImGuiWindowFlags_NoTitleBar |
ImGuiWindowFlags_NoResize | ImGuiWindowFlags_AlwaysAutoResize |
ImGuiWindowFlags_NoScrollbar;
```

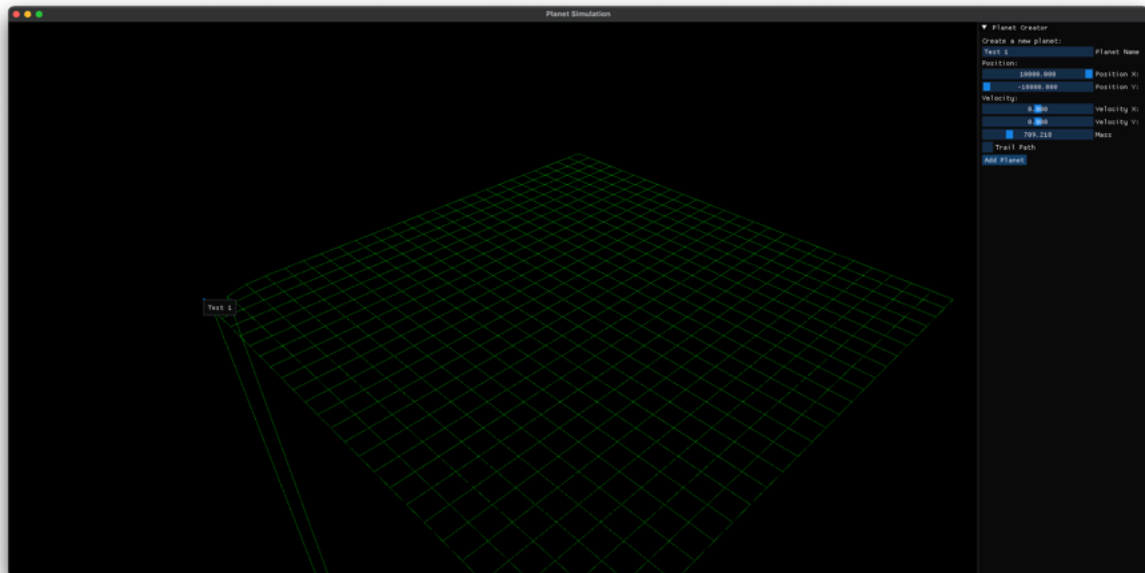
In order to test this I'm going to use my sliders to render the planets at each corner of the grid. The test will be successful if the label also renders in this location.

Test ID	Test Type	Test Data	Description	Expected	Actual	Success Criteria	Pass/Fail	Evidence
TL1	Normal	Left corner of my grid, coordinates (10000, -10000)	These tests are testing that the class correctly takes in the position it is given and updates the labels position accordingly, outputting this to the display.	Label renders on the left corner of the grid.	As expected, it renders on the left hand side of the grid.	SC13	Pass	Screen shot 1 (below)
TL2	Normal	Right corner of my grid, coordinates (-10000, 10000)		Label renders on the right corner of the grid.	As expected, it renders on the right hand side of the grid.	SC13	Pass	Screen shot 2 (below)
TL5	Normal	"Test 1"	These tests are testing that the class correctly stores and outputs the	"Test 1"	"Test 1"	SC13	Pass	Screen shot 1 (below)
TL6	Normal	"Mars"		"Mars"	"Mars"	SC13	Pass	Screen shot 2 (below)

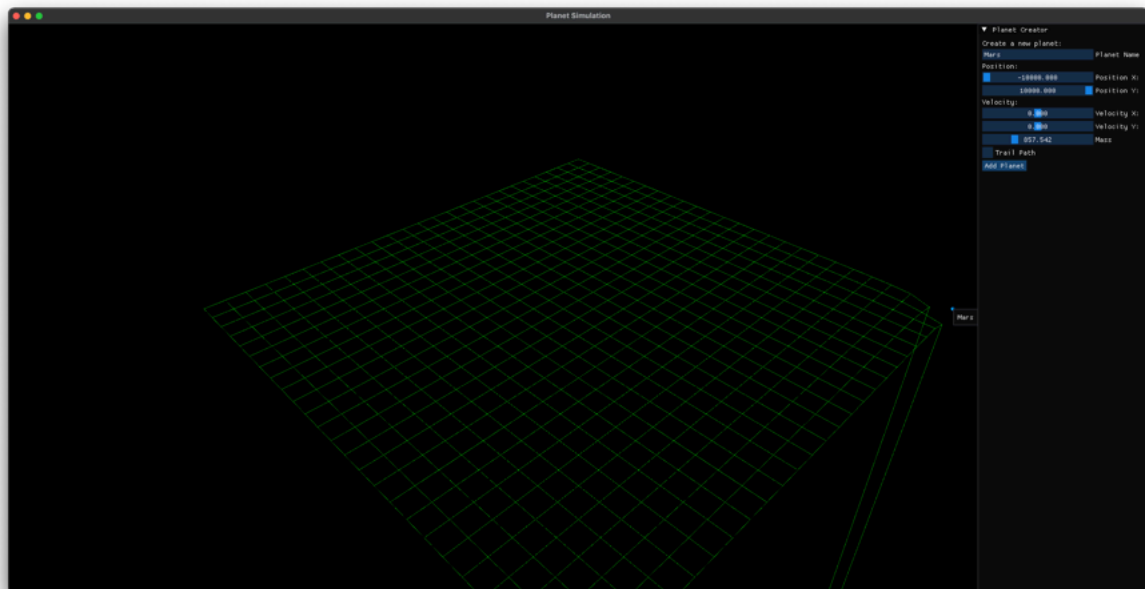
			name of each planet.					
--	--	--	----------------------------	--	--	--	--	--

Here are the relevant screenshots for evidence:

Screenshot 1:



Screenshot 2:



The Pause/Play Button

Build

The first stage in implementing a pause/play button is to introduce the boolean value **IsRunning** into my ImGui defaults:

```
C/C++
bool IsRunning = true;
```

This boolean value starts as true as I want the simulation to be running at first. This feature will work by only running calculations that update the position of the planets when this boolean value is true. I therefore updated the loop that updates and draws the planets to this:

```
C/C++
for(int i = 0; i < planets.size(); i++){
    planets[i].acceleration = vec2{0,0};
    if (IsRunning){
        for(int j = 0; j < planets.size(); j++){
            if (i != j){
                planets[i].updateAcceleration(planets[j].position,
planets[j].mass);
            }
            planets[i].updateVelocity(vec2{planets[i].acceleration.x * dt,
planets[i].acceleration.y * dt});
            planets[i].updatePosition(vec2{planets[i].velocity.x * dt,
planets[i].velocity.y * dt});
        }

        planets[i].DrawPlanet(shaderProgram, objectColorLoc);
    }
}
```

I then added this button to my ImGui code. This button changes its display after being pressed, so when the simulation is running it displays "Pause Simulation" and when the simulation is paused it displays "Resume Simulation":

```
C/C++
if (ImGui::Button(IsRunning ? "Pause Simulation" : "Resume Simulation")) {
    IsRunning = !IsRunning;
}
```

Testing

I then carried out the test TUI2. If this test passes I have succeeded in creating a pause/resume button and I can move onto the next user interface feature.

Test ID	Description	Success criteria	Pass or Fail	EVIDENCE
TUI2	The user can use a button to pause the simulation, and with the same button resume the simulation.	SC7	Pass	EVIDENCE VIDEO 24

Review

This was a very successful build as it only required one iteration. I'm happy with how this button works. There is the possibility that I change the position of this button later in development to make the user interface function better for the user. For now, however, this section of my development is complete.

The Reset Button

Build

Building the rest button involved a simple ImGui button that would clear the **planets** vector when pressed. This would remove all the planets from the vector and therefore return the space-time grid to its original state as each vertex would experience no force from any planets. This is what the code for the button looked like:

```
C/C++
if (ImGui::Button("Reset Simulation")) {
    planets.clear();
}
```

Testing

Test ID	Description	Success criteria	Pass or Fail	EVIDENCE
---------	-------------	------------------	--------------	----------

TUI3	The user can reset the simulation using a button on the interface.	SC8	Pass	EVIDENCE VIDEO 25
------	--	-----	------	--------------------------

Review

Although this may have been a simple build, I can be happy that it only took me one iteration to complete and it runs successfully.

The Delete Button

Build

The delete button and the drop down menu that is used to select the planet relies on the variable **selectedPlanetIndex**. This variable is used to display all the planets in the drop down menu using their name and converting to the ImGui string data type using **.c_str()**. The for loop checks whether each planet had been selected and if it has been it becomes the selected planet and **selectedPlanetIndex** is assigned its position in the **planets** list.

This variable is then used in the if statement to validate that this variable is within the accepted range. It is then used within the ImGui button statement to erase the planet selected in the drop-down menu. Finally, this variable is assigned the value -1, so that it does not fall within the valid range and remove other planets in subsequent iterations of the rendering loop.

```
C/C++
if (ImGui::BeginCombo("Select Planet to Delete", (selectedPlanetIndex >= 0
&& selectedPlanetIndex < planets.size()) ?
planets[selectedPlanetIndex].name.c_str() : "None")){
    for (int i = 0; i < planets.size(); i++){
        bool SelectedPlanet = (selectedPlanetIndex == i);
        if (ImGui::Selectable(planets[i].name.c_str(),
SelectedPlanet)){
            selectedPlanetIndex = i;
        }
    }
    ImGui::EndCombo();
}

if (selectedPlanetIndex >=0 && selectedPlanetIndex < planets.size()){
    if (ImGui::Button("Delete Selected Planet")){
        planets.erase(planets.begin() + selectedPlanetIndex);
        selectedPlanetIndex = -1; //Reset selection after deletion
    }
}
```

```
}
```

Testing

Test ID	Description	Success criteria	Pass or Fail	EVIDENCE
TUI1	The user can select a planet from a drop down menu and remove this from the simulation.	SC9	Pass	EVIDENCE VIDEO 26

Review

This build used slightly more complicated ImGui commands to function correctly. However, it was completed successfully only requiring one iteration and passing test TUI1. I can therefore move onto the next stage of my development.

Educational Value

Build

As I outlined in my design, I want to display different educational messages depending on what is currently happening in the simulation. I'm going to achieve this by using if statements to check how many objects are in my **planets** vector.

- If there are no objects in the vector, I will display a message that introduces the simulation and suggests that the user add a planet using the controls.
- If there is just one object in the vector, I will display a message that explains what the grid represents and a fact relevant to this.
- If there is more than one object in the vector, I will display a message that explains a bit about gravity.

I will package this into a function to ensure code maintainability.

This function will revolve around the variable **EducationalMessage**, which will be a global variable and have the string data type.

Here is how the function looks:

```
C/C++  
void DecideEducationalMessage(const std::vector<Planet>& planets){
```

```

if(planets.size() == 0){
    EducationalMessage = "Welcome to my planet simulation. To get
started, use the panel on the right to create a new planet by specifying its
name, position, velocity, and mass. You can also choose to enable a trail
path to visualize the planet's trajectory.";
}
else if (planets.size() == 1){
    EducationalMessage = "You've added your first planet! Observe how it
the space-time grid distorts around it, simulating gravitational effects.
This shows how mass doesn't just sit in space-time - it tells space-time how
to curve, and curved space-time tells mass how to move. Add another planet
to see gravitational interactions!";
}
else if (planets.size() >= 2){
    EducationalMessage = "With multiple planets in the simulaiton, you
can see how they influence each others trajectories through gravitational
forces. Each planet's mass distorts the space-time grid, affecting the paths
of nearby planets. Experiment with different masses and initial velocities
to explore various orbital dynamics!";
}
}

```

The next step was to use ImGui to display this message:

```

C/C++
DecideEducationalMessage(planets);
ImGui::SeparatorText("Educational Information");
ImGui::TextWrapped("%s", EducationalMessage.c_str());

```

Testing

Test ID	Description	Success criteria	Pass or Fail	EVIDENCE
TUI4	Educational prompts are displayed to the user.	SC11	Pass	EVIDENCE SCREENSHOTS 2,3,4
TUI5	The educational prompts change depending on what is in the simulation.	SC12	Pass	EVIDENCE SCREENSHOTS 2,3,4

This can be seen in **EVIDENCE SCREENSHOTS 2,3,4** which show the different messages with different numbers of planets.

Camera Position

Build

In my design, I outlined that I would implement camera control by using mouse feedback and the mouse's scrollwheel with a camera that orbits the origin. However, upon reflection, I think that the best way to implement this is to use sliders for each direction that the camera can move in. My justification for this is that I think this will be clearer for the user as it will be well labelled. In addition to this, sliders integrate naturally with my other ImGui controls, making it easier to tweak these later if I need to..

This started by adding these lines to my ImGui defaults, so they could be edited by sliders. I needed to add all the components of the camera coordinates. I kept the X and Z defaults as 15000 because I wanted the camera positioned out from and above the grid. However, I changed the y coordinate default to 0.0f because it would be easier for the user to understand. This is because the x slider would be forwards/backwards; the y slider would be left/right and the z slider would be up/down:

```
C/C++  
float cameraX = 15000.0f;  
float cameraY = 0.0f;  
float cameraZ = 15000.0f;
```

Next I used the ImGui slider function to create sliders for each, allowing the user to vary the coordinates between -30000 and 30000:

```
C/C++  
hoImGui::SeparatorText("Camera Control");  
ImGui::SliderFloat("Camera X", &cameraX, -30000.0f, 30000.0f);  
ImGui::SliderFloat("Camera Y", &cameraY, -30000.0f, 30000.0f);  
ImGui::SliderFloat("Camera Z", &cameraZ, -30000.0f, 30000.0f);
```

Testing

Test ID	Description	Success criteria	Pass or Fail	EVIDENCE
---------	-------------	------------------	--------------	----------

TUI6	The user can navigate the worldspace using the camera controls.	SC10	Pass	EVIDENCE VIDEO 27
------	---	------	------	--------------------------

This can be seen in **EVIDENCE VIDEO 27**

Review

As all tests associated with implementing camera movement have passed, this section is now finished. As this was the final stage in my development, I can declare that the bulk of my development is finished.

Calibrations

Space-time grid

However, there are some minor changes I would like to make before I move onto evaluation. This is related to SC22 - the space-time grid gives an accurate depiction of how gravity works.

The first of these calibrations was to change the colour of the space-time grid. I wanted the grid to be a translucent white colour to make it easier for the user to view other objects in the simulator. This simply involved the colour of the space-time grid within the fragment shader:

```
C/C++
if (objectType == 1){
    FragColor = vec4(1.0, 1.0, 1.0, 0.25); // White for grid
}else if (objectType == 0){
    //code for other object type...
```

Next, I wanted to increase the amount of divisions of the space-time grid because currently it doesn't feel like a continuous fabric but more like a series of space-out vertices. This was as simple as editing the **divisions** variable to 100.

The final change I wanted to make to the space-time grid was switching from an inverse squared relationship between displacement and distance to an inverse relationship. This is because I didn't like how planets on the grid affected its displacement. I also decreased the k constant within this method to 2400 rather than 240000. Because of these changes I will have to change the **updateGrid()** tests for post-development testing. Here is the updated **updateGrid()** method:

```

C/C++
void UpdateGrid(std::vector<Planet>& planets){
    vertices = baseVertices;

    float k = 2400.0f; // Gravitational constant for grid distortion
    float c = 1000.0f; // Damping factor to control distortion
    intensity

    for (int i = 0; i < vertices.size(); i += 3){
        float x = vertices[i];
        float y = vertices[i+1];
        float z = vertices[i+2];

        float totalDisplacement = 0.0f; //Initially no displacement

        for(int j = 0; j < planets.size(); j++){
            //dx and dy are the differences in x and y coordinates
            between the grid point and the planet
            float dx = x - planets[j].getPositionX();
            float dy = y - planets[j].getPositionY();
            float distance = sqrt(dx*dx + dy*dy); // Pythagorean
            theorem to calculate distance

            float displacement = (-k * planets[j].getMass()) /
            (distance + c); // Gravitational distortion formula
            totalDisplacement += displacement; //Accumulate
            displacement from all planets
        }

        vertices[i+2] = z + totalDisplacement; // Update
        z-coordinate based on total displacement
    }
}

```

Aesthetic Calibrations

To make the user interface look more polished I added style to the ImGui controls. This involved adding rounding to the sliders and text-boxes. To implement this I created an ImGui Style which I called **style**. I then added the colours I wanted to this, which were almost indistinguishable to what I had before. Here is what the code looked like for that:

```

C/C++
ImGui::CreateContext();

```

```
ImGui::StyleColorsDark();

ImGuiStyle& style = ImGui::GetStyle();
ImVec4* colors = style.Colors;

//Dark theme with blue accents
colors[ImGuiCol_WindowBg] = ImVec4(0.06f, 0.06f, 0.10f, 0.8f);
colors[ImGuiCol_Header]   = ImVec4(0.2f, 0.4f, 0.8f, 0.55f);
colors[ImGuiCol_Button]   = ImVec4(0.2f, 0.4f, 0.8f, 0.6f);
colors[ImGuiCol_ButtonHovered] = ImVec4(0.3f, 0.5f, 1.0f, 0.8f);
colors[ImGuiCol_Text]     = ImVec4(0.9f, 0.9f, 1.0f, 1.0f);

style.FrameRounding = 6.0f;
style.WindowRounding = 8.0f;
```

These calibrations can also be seen in **EVIDENCE VIDEO 27**

Review

The purpose of stage 6 was to add usability features and perform any final calibrations needed in the simulator. I successfully carried this out, with all tests passing eventually, even if debugging was needed.

As I have now carried out all parts of my development, I can declare my development as entirely complete and move onto post-development testing and evaluation.

Evaluation

All tests I carried out in the post development stage can be found in the Appendices. This includes my stakeholder tests which includes student questions and a teacher interview.

Evaluating Success Criteria

Success Criteria	Relevant Tests	Success	Justification
SC1 - Planet's name can be customised using a text box.	TPC1, PD1, PD14, MLRT11, VT1-VT6	Full	All tests associated with SC1 passed. The constructor method in my planet class effectively takes input from the ImGui textbook and instantiates the planet with that name. In addition to this, the necessary validation is carried out to ensure that users don't enter invalid values. Therefore, this success criteria is a full success.
SC2 - Planet's mass can be customised using a slider.	TPC1, MLRT10, PD2, PD3	Full	All tests associated with SC2 passed. The constructor method in my planet class effectively takes input from the ImGui slider and instantiates the planet with that mass. The constructor method also effectively takes the mass and uses this to instantiate the radius of the planet with a basic calculation.
SC3 - The planet's initial position can be customised using a slider for each component.	TPC1, MLRT8, PD4, PD5	Full	All tests associated with SC3 passed. The constructor method in my planet class effectively takes input from the ImGui sliders and instantiates the planet with that initial position. The DrawPlanet method then uses this position to render the planet in the given position in the simulation.
SC4 - The planet's initial velocity can be customised using a slider for each component.	TPC1, MLRT9, PD6, PD7	Full	All tests associated with SC4 passed. The constructor method in my planet class effectively takes input from the ImGui sliders and instantiates the planet with that initial position. The UpdatePosition method which is used in the update loop then uses this initial velocity to move the

			planet across the environment. This is visualised with the DrawPlanet method.
SC5 - The user can select whether or not they want a planet to have a trail path.	TPC1, TPR1, TPR4, PD8, PD9	Full	All tests associated with SC5 passed. The constructor method in my planet class effectively takes input from the ImGui checkbox and instantiates the planet with that initial position. Then if statements are used in the instantiation button statement to dictate whether or not a trail path is rendered.
SC6 - Once the user has customised a planet, they can add it to the environment using a button.	MLRT6, MLRT11, PD10,	Full	All tests associated with SC6 passed. The planet's attributes are successfully captured when the planet is added to the environment whether that be with its radius or velocity.
SC7 - Ability to pause and play the simulation using a button.	TUI2 PD11 PD12	Full	All tests associated with SC7 passed. The pause play button successfully changes the state of the boolean variable IsRunning , which is used in if statements within the rendering loop to dictate whether or not calculations are run to update the acceleration, velocity and position of all the planets.
SC8 - Ability to reset the simulation using a button.	TUI3 PD13	Full	All tests associated with SC8 passed. The reset button successfully removed all planets from the list of planets, which resets the simulation as with no planets, the space-time grid returns to its original position.
SC9 - Ability to delete a given planet using a drop-down menu and a button.	TUI1 PD14 PD15	Full	All tests associated with SC9 passed. The user can select a planet from the drop-down menu using its name. The user can then delete this planet using the relevant button.
SC10 - Ability to move the camera position around the environment.	PD16 SH5	Full	All tests associated with SC10 passed. The user can use sliders in each directly to navigate and explore the 3D environment. In addition to this, my stakeholders rated the camera usability a 4.2/5. This means that test SH5 also passed.

SC11 - Educational messages are displayed to the user.	TUI4 PD17	Full	All tests associated with SC11 passed. Educational messages are displayed to the user on the user interface panel.
SC12 - Educational messages change depending on what is in the simulation.	TUI5 PD18	Partial	All tests associated with SC12 passed. The educational messages change depending on what is in the 3D workspace for the user. There are 3 different messages which change depending on the number of planets in the list of planets. However, my teacher interview tests did not pass as my teacher said that the messages could be a lot more specific. So I therefore would not class this as a full success.
SC13 - Labels for all planets in the simulation.	VT1-VT6 PD1 PD19	Full	All tests associated with SC13 passed. The labels correctly show the name of each planet, which are validated after being entered into the text box. The labels then follow the planet across the screen, making it clear to the user where each planet is. The render loop successfully takes the position of each label's associated planet and updates the labels position to match this every frame.
SC14 - The simulator must be able to render the planets in the correct position on the screen.	TPP1-3 PD5	Full	All tests associated with SC14 passed. The planets are rendered as spheres in the correct position. In the post development videos you can see how changing the initial position sliders changes its position on the space-time grid, which can be used as a reference to see where planets are positioned. In addition to this, planets moving across the environment have their position updated correctly every time-step in accordance with their velocity and acceleration.
SC15 - The simulator must accurately simulate gravitational	TPD1-3 TPG1+2 TPA1-3 TPRF1-3	Partial - all tests pass apart	As not all tests associated with SC15 pass, this must be considered a partially met success criteria. I will talk about this in more detail later on, but I think that this

attraction between multiple planets using Newton's law of universal gravitation.	TPV1-3 TPP1-3 TPUA1+2 TG1 MLRT5 PD20 Teacher interview question 5	from TG1	was due to me using floats, which only store data in 32 bits. As the calculations for calculating the updated position for every planet require many steps and as floats can only work in 7 decimal places, any rounded values that lose accuracy will be passed down into the next stage of the calculation. This means that the errors gradually build up larger and larger. However, the teacher interview was a question as Dr Mahoney rated my simulation a % in terms of accuracy.
SC16 - The simulator must update each planet's velocity, position and acceleration in real-time based on the forces acting on it.	TPD1-3 TPG1+2 TPA1-3 TPRF1-3 TPV1-3 TPP1-3 TPUA1+2 MLRT2-4 PD7	Pass	All tests associated with SC16 passed. The planet class successfully takes inputs such as the list of planets and uses this to calculate the velocity, position and acceleration of the planet. The methods within the planet do this successfully, with them all being packaged into a single method which I can call for every planet in the list every time step.
SC17 - Show planets with lighting and shading to emphasise their 3D form.	MLRT1	Pass	All tests associated with SC17 passed. There is screenshot and video evidence of the planets being rendered with a 3D appearance due to my use of shaders and lighting position within the program.
SC18 - The customisation of planets should be captured in the simulator by differences in the visual characteristics of the planets such as size, position and velocity/speed.	TPC1 MLRT2+3 PD2 PD7	Pass	All tests associated with SC18 passed. The mass is represented by the radius for each planet. A larger radius/size = greater mass. In addition to this, planets created with a greater velocity can clearly be seen to move with a greater initial speed in the evidence videos. As well as this, the initial position of planets is controlled by sliders, which can also be seen in the evidence videos.
SC19 - Maintain a frame rate high enough for smooth animation.	PD21 SH1 SH2	Pass	All tests associated with SC19 passed. The simulation gets a frame-rate that is smooth and allows the user to see the simulation in good detail. This is captured in video evidence.

			In addition to this, stakeholders rated my simulation a 4.6/5 for the performance and responsiveness of the simulation.
SC20 - A space-time grid is rendered which initially is just a flat grid.	STGR1-3 PD22	Pass	All tests associated with SC20 passed. The space-time grid class is effectively used to create the vertices of the grid and this is captured in video and screenshot evidence.
SC21 - The space-time grid warps around planets which are added to the simulation.	STGR4 STGU1 PD23	Pass	All tests associated with SC21 passed. The program effectively uses the UpdateGrid method to update the position of each vertex and renders this to the window every time-step. This is captured in video evidence.
SC22 - The space-time grid gives an accurate depiction of how gravity works.	Teacher Interview question 5.	Pass	As Dr Mahoney ranked my simulation a % for accuracy of my simulator, this test passes and SC22 is fulfilled.
SC23 - Trail paths render behind planets that the user has selected to have a trail path.	TP1 TPR2 PD24	Pass	All tests associated with SC23 passed. This is seen in the video evidence of the trail paths rendering alongside the space-time grid and the planets. This works with the checkbox which the user can use to select whether or not they would like a rendered trail path.
SC24 - Trail paths have a set length and points are removed as necessary to maintain this set length.	TP2 TPR3 PD25	Pass	All tests associated with SC24 passed. The program effectively uses the trail path class which contains functions which correctly add and remove points from the path to maintain a set length. This can be seen in the videos when the path follows the planet without getting longer.
SC25 - A web-based simulation that users can access via the web.	TWe1-3	Fail	All tests associated with SC25 failed. The program does not run on the web as I could not get this to work. This will be something I discuss later on in evaluation.
SC26 - The simulator is simple and clear.	SH3 SH4	Pass	All tests associated with SC26 passed.

SC27 - The user interface controls are responsive and work without delay.	SH2	Pass	All tests associated with SC27 passed. There is video evidence of the user interface working with delay.
---	-----	------	--

Unmet Success Criteria and How they Could be Addressed

Out of my 25 success criteria, 23 were met, 2 were partially met (SC12 + SC15) and 1 failed (SC25). For each of these I am going to cover why they were not met and how this could be addressed in further development. I will cover SC12 in the usability section of my evaluation because I consider the educational messages a usability feature.

SC15

The failure with SC15 is a very minor one, but a failure nonetheless. My values for the position and velocity of the planets was slightly off, even in the first time-step of the simulation. In addition to this, the distance and force between two planets in the simulation was calculated with slight variation to what I expected. I therefore assume that these slight variations are passed through each of the attributes which are modified by gravitational calculations (acceleration, velocity, position). While this may be such a miniscule difference for the first few time-steps in the simulation, these errors seemed to build up as the simulation continued.

Take the distance calculation for example. In the first time-step there is only a 0.05% difference in the actual (chatBot calculated) value versus the value my simulation calculated. This would make no difference at all to the simulation as it is purely visual. However, the force calculation had a much larger 0.12% difference from the actual value. I think this is because the force calculation used not only the difference between the planets in its calculator, but also the mass of both planets.

Going further, by just the 5th time step, the last one I used in my tests, there was a 1.15% difference in the chatBot calculated force and my simulator's value of force. This build up of error is somewhat significant as if you run the simulation for any large amount of time, this could lead to significant differences in the positions of planets.

That being said, as these calculations are repeated every time-step, the simulation still accurately represents gravity and gives a suitable visual representation of it. However, it would be unsuitable to use my simulation to predict the long-term behaviour of gravitational systems. Since my simulator is designed for the use of students and not scientists, this does not mean a total failure of my simulation.

I think this partial error arises from my use of floats. Therefore, I used chatGPT to explore the differences in accuracy between c++ floats and chatGPT's level of accuracy:

Prompt	Evidence																				
What level of accuracy do c++ floats have compared to chatGPT?	What level of accuracy do c++ floats have compared to chatGPT Excellent question — and you're thinking about this in exactly the right way. Let's compare C++ float, C++ double, and ChatGPT's (and Python's) arithmetic precision, since they all handle numbers differently. 1. C++ Float Precision <table><thead><tr><th>Type</th><th>Bits Used</th><th>Significant Digits</th><th>Range</th><th>Typical Use</th></tr></thead><tbody><tr><td>float</td><td>32 bits (single precision)</td><td>~7 decimal digits</td><td>-10^{+38} to 10^{-38}</td><td>Fast, less memory, less precision</td></tr><tr><td>double</td><td>64 bits (double precision)</td><td>~15-16 decimal digits</td><td>-10^{+308} to 10^{-308}</td><td>Default for most calculations</td></tr><tr><td>long double</td><td>80-128 bits (platform-dependent)</td><td>~18-19+ digits</td><td>Depends on compiler & CPU</td><td>Very high precision (rarely needed)</td></tr></tbody></table> Example In C++: <pre>cpp float f = 1.0f / 3.0f; // 0.33333334 double d = 1.0 / 3.0; // 0.3333333333333333</pre> The float is accurate to about 7 digits, while double keeps about 15-16 digits. 2. ChatGPT's (and Python's) Precision ChatGPT uses double precision floats (IEEE 754 standard), which are equivalent to C++ double. Python's float is also a double precision float — the same as C++ double.	Type	Bits Used	Significant Digits	Range	Typical Use	float	32 bits (single precision)	~7 decimal digits	-10^{+38} to 10^{-38}	Fast, less memory, less precision	double	64 bits (double precision)	~15-16 decimal digits	-10^{+308} to 10^{-308}	Default for most calculations	long double	80-128 bits (platform-dependent)	~18-19+ digits	Depends on compiler & CPU	Very high precision (rarely needed)
Type	Bits Used	Significant Digits	Range	Typical Use																	
float	32 bits (single precision)	~7 decimal digits	-10^{+38} to 10^{-38}	Fast, less memory, less precision																	
double	64 bits (double precision)	~15-16 decimal digits	-10^{+308} to 10^{-308}	Default for most calculations																	
long double	80-128 bits (platform-dependent)	~18-19+ digits	Depends on compiler & CPU	Very high precision (rarely needed)																	

To summarise what information the chatbot gave me: ChatGPT uses double precision floats which are equivalent to a c++ double with 15-16 significant digits of accuracy. My calculation however uses c++ floats with 7 significant digits of accuracy. I think this could have been significant enough to cause the errors in my calculations. What happened is that my floats were rounded to their 7 digits of accuracy. Many of these floats were all used together in calculations surrounding the position of each planet. Therefore, the rounding built up into discrepancies between my values from the simulation and chatGPT's calculated values.

How could I address this in further development?

In order to solve this problem, I would have to use a data type with a greater level of accuracy. OpenGL doesn't support the use of doubles fully. This is mainly because hardware would not be able to handle the extra level of accuracy. Therefore, I cannot simply change all of my data types to doubles and expect this to work. However, what I could do is use doubles for all the attributes of the planets. I would therefore carry out all the calculations using doubles and increasing the level of accuracy of my calculations. As these values cannot be used in the rendering pipeline, I would have to convert them all to floats. I could do this using the casting to change floats to doubles.

Here is an example of how I would do this:

```

C/C++
//I have a double with the variable name Position
float PositionFloat = static_cast<float>(Position);

```

If I was inputting these values into functions, I can cast the variables at this stage. Take the OpenGL translate function for example, which expects to take floats.

C/C++

```
//In this example position.x, position.y and position.z are doubles from the planets attributes  
model = glm::translate(model, glm::vec3(static_cast<float>(position.x),  
static_cast<float>(position.y), static_cast<float>(position.z)));
```

Therefore, in order to implement using doubles into my program I would carry out these two steps:

1. Change all planet attributes to doubles
2. Use casting to cast these doubles into floats which OpenGL can use.

This would fix my problem and after these changes I think that test TG1 would pass.

SC25

Success criteria 25 was a complete failure. I didn't manage to make the simulator web-based. I think that there were several reasons for this.

Firstly, I don't think I had a good enough understanding of what running my code on the web would entail. This can be seen in my development when I had to switch from using an older OpenGL to the modern version of OpenGL. When I first started my development, I did not realise that Emscripten did not support the use of older OpenGL versions. I think this was a positive process because it allowed me to get to grips with the graphics pipeline and how OpenGL works all while using the more simple older functions, however, the whole process of switching versions could have been avoided.

This lack of understanding was particularly evident when Emscripten failed to work on the first try. As no errors were returned, there was no where I could look to try and fix errors. I am still unaware of what caused the error, but I suspect it is because I didn't prepare the code effectively. (Reference 1) This article talks about several steps in preparing their code for compilation with Emscripten and dealing with dependencies.

How could I address this with further development?

Therefore, in further development I would focus on two things before compiling with Emscripten:

1. Dealing with dependencies. In my code I have 4 main dependencies: OpenGL, GLFW, glm and Dear ImGui. All of these can be compiled for the web using Emscripten, including ImGui. This means as long as I deal with these correctly, the program should be able to run on the web.
2. Prepare my code for Emscripten compilation. What I didn't do in my development was edit my render loop for Emscripten use. This is something that must be done in order for it to work.

I think if I had done this I would have been able to get my program to run on the web.

What impact will this have on users?

Although I didn't meet SC25 in my development, this is not a total failure in terms of the project as a whole. Users within my school would still be able to access the simulation by downloading an executable file and it would still provide educational value to these users. However, users outside my school would definitely struggle with accessing it. One solution to this would be having a website which the users could access which contained the downloadable file. Users could then download and run the program.

Usability Feature Evaluation

In the following table I've isolated each of the usability features, outlined whether or not they are a success or failure and justified why. These do line up with some of my success criteria, but here I have a greater focus on how they succeed as usability features rather than in the program as a whole.

Feature	Success	Justification
Textbox that allows the user to input the planet name.	Full	All validation tests (VT1-6) for this feature were successful, preventing the user from inputting invalid data and presenting them with a message if they try to input invalid data. The user can select the textbox with a left click of their mouse and use their keyboard to enter data. In addition to this all usability success criteria related to this feature (SC26+27) were fulfilled. The textbox fulfills SC26 because it is clearly labelled, with the function of the textbox being positioned to the right of it. As well as this, all technology users are familiar with textboxes, making this very simple for them to use. SC27 was fulfilled by this feature because there is no delay between the user selecting the box with their mouse and being able to enter data using their keyboard.
Sliders that allow the user to edit the properties of each planet.	Full	No validation is needed for these sliders because the user can only select valid values. The sliders function correctly, with the user being able to left click and drag on the handle to use the slider. This feature fulfills SC26 because each slider is very clearly labelled, even when attributes are split into 2 components (X and Y). The clarity and ease of use is enhanced by the slider displaying its

		current value. This feature fulfills SC27 because the sliders are very responsive to the user's input. The displayed value changes in real-time without lag or delay and the editing of the slider is a smooth process.
Checkbox that allows the user to select whether or not they want a trail path to be displayed.	Full	No validation is needed for this checkbox because the user can only select it or de-select it. The checkbox functions correctly, with the user being able to select and de-select it. This feature fulfils SC26 because the checkbox is very clearly labelled. This feature fulfils SC27 because as soon as the user selects the box, a tick appears in it, signalling to the user that they have selected this option. Similarly, when the user de-selects it, there is no delay and this is displayed to the user immediately.
Button that allows the user to add the customized planet to the simulation.	Full	No validation is needed to this button because the user can only press it. The button functions correctly. This feature fulfils SC26 because the button is very clearly labelled. This feature fulfils SC27 because when the user presses the button, a planet is added to the simulation without delay. This can happen multiple times over a small time period and the planets will similarly be added without delay.
Button that pauses and plays the simulation.	Full	No validation is needed to this button because the user can only press it. The button functions correctly. This feature fulfils SC26 because the button is very clearly labelled with the function of the button signalled with text on the button that changes depending on whether or not the simulation is running. This feature fulfils SC27 because the button works without delay. As soon as the user presses it, the text on the button changes - either from pause to play or from play to pause - and the simulation changes state.
Button that resets the simulation.	Full	No validation is needed to this button because the user can only press it. The button functions correctly.

		This feature fulfils SC26 because the button is very clearly labelled with the function of the button signalled with text on the button. This feature fulfils SC27 because when the button is pressed, the simulation resets without delay.
Educational messages displayed which change depending on what is happening in the simulation.	Partial	This feature fulfils SC26 because the display is labelled with its function ("Educational Information"). However, Dr Mahoney did say that the prompts could be more specific to what I have in the simulation. Therefore, this feature is not a full success.
Sliders which allow the user to position the camera as they wish.	Full	No validation is needed for these sliders because the user can only select valid values. The sliders function correctly, with the user being able to left click and drag on the handle to use the slider. This feature fulfills SC26 because each slider is very clearly labelled. The clarity and ease of use is enhanced by the slider displaying its current value. This feature fulfills SC27 because the sliders are very responsive to the user's input. The displayed value changes in real-time without lag or delay and the editing of the slider is a smooth process.

Aesthetic Features

The first change I made to make the user interface more pleasing to the eye was to introduce separators between the sections of the controls. This makes it clear to the user what parts of the interface fall under what section (fulfilling SC26). In addition to this, I think that the sliders within the planet customisation section are well spaced, allowing for easy navigation for the user.

The second change I made was rounding the sliders, buttons, text input boxes and drop-down menus for a more polished and sleek look. This may not have a strict purpose but does make the simulator look better.

In terms of colour of the user interface, I chose to stick with the ImGui defaults. This is because they are fairly neutral colours which do not interfere with the user's view of the worldspace. I wanted the focus to be on the simulation running but without having a very boring user interface that might put a user off. I therefore think I've done the right thing in keeping the original colours.

For the colours within the simulation, I choose blue for the planets as it matches with the user interface, giving a polished feel to the simulator. The most appropriate colour for the

space-time grid was a semi-translucent white. This is because it would enable the user to partially see through it, important when viewing the simulation from certain angles. In addition to this, the colour of the space-time grid means that it doesn't obstruct the view of the planets or the trails paths. It was important that the colour of the trail-paths was different from both the space-time grid and the planets. I therefore chose a bright yellow, making the trail-paths very obvious and easily allowing the user to track the planets that they want to.

How I could address unmet usability features?

Educational Messages - SC12

From interviewing my teacher, I found that the main problem with my user interface was the fact that the educational messages could have been a lot more specific to what was happening in the simulation. He talked about how in other simulators he has used, for example the [NSTMF Gravity Simulator](#), the messages are directly related to the state of orbits and trajectories in the simulation. He said that while it was good to have different messages for a different number of planets in the simulation, it would be better to base this off other factors within the simulation.

In my simulation, 2 planets can be on opposite sides of the worldspace but the message says: 'With multiple planets in the simulation, you can see how they influence each other's trajectories through gravitational forces.' This is inaccurate as if they are too far away from each other the interaction will not be visible to the user. I therefore think that if I was to carry out further development, I would change this as it results in the partial failure of SC12.

How could I address this with further development?

I could address this by using more if and else if statements and changing the ones I have. Taking the example above, I could display a different message if two planets are too far away from each other. This would be fairly simple to implement for a two planet system as I could use the distance method to calculate the distance between them, comparing it to a value within the if statement.

To make the messages more specific I could also introduce using the acceleration of planets to predict what is going on in the simulation. As planets are added to the simulation with no acceleration, the only way that can gain acceleration is due to interaction with other planets. Therefore, I could introduce an if statement which checks if any planets have an acceleration above a certain value, then display a message about gravitational acceleration/interaction if this is the case.

I think this would be the best way of approaching it without huge complexity, however, if I wanted to make this function more advanced there are several things I could do. An example of this would be checking if a planet follows a roughly circular path. I could use trigonometry to check if the velocity of the planet was perpendicular to the central planet it is orbiting. As well as this I could keep a bank of the planet's previous positions and use a standard deviation calculation to check if the velocity is staying fairly constant without too much

variation. These two in combination would allow me to check for gravitational orbits in planets. This would make the educational messages more specific to the system on the screen.

Limitations of the Solution

My finished project has a few limitations. The first limitation is the limitation on accuracy. I talked about this in my analysis, mentioning that I will use Euler's method of integration, which I did, and saying that this can become inaccurate over time. We have already seen how using floats has made the simulation slightly inaccurate as time goes on. I choose to use Euler's method for calculations because it is easy to implement. It works well for my simulation as I wasn't focused on accuracy. It also performs less calculations every time step than other methods, meaning I have a smooth simulation. That being said, there are integration methods - such as Verlet and Runge-Kutta - which would be a lot more accurate, especially in longer periods of simulation.

In addition to this, my simulation only runs in 2 dimensions. This is a limitation because this is not realistic, with gravity acting in all directions. I chose to do this because it simplifies the simulation for the user and 2D simulations still provide educational value to the user. The user can still view the simulation as if it was 3D, but the planets only move across a 2 dimensional plane. Adding a 3rd dimension would have added an extra layer of complexity, especially for the space-time grid as it is difficult to implement a 3D grid that warps around planets. I also think this would have been disruptive to the user's view. Furthermore, only simulation in 2D reduces computation power required, allowing for a smooth responsive and smooth simulation.

Another limitation of my simulation is that there is no collision handling between planets. Instead of planets colliding in my simulation, they just overlap each other, which is very unrealistic. I choose to do this because I think it is interesting to see what happens when the planets get very close to each other and fling each other across the worldspace. In addition to this, if a user feels that it is necessary to remove a planet, then they can do so using the drop-down menu. Nevertheless, the lack of collision handling is definitely a limitation of my solution.

Finally, the user cannot edit the color or appearance of planets in the simulation. While they can track the movement of planets using their label, I think it would be a good feature to have if the user can edit the colour of each planet before adding it to the simulation. This would also allow them to set up systems similar to real-life systems such as our solar system, with each planet having its own colour.

In order to increase the accuracy of my simulation I have already talked about switching from using floats in my calculations to using doubles. However, another thing I could do is use a different integration method. Verlet integration is an example of a more accurate computational integration method I could use. This involves calculating the next position of a

planet or object using the current position, the previous position and the acceleration. This would be more accurate than Euler integration because it calculates the next position while conserving energy and momentum over time.

How could I address this with further development?

If I were to develop the project again I would implement a feature on the user interface that allows the user to customise the colour of each planet they add to the simulation. I could do this by introducing a colour attribute to my planets. This could be edited by the user using an ImGui colour editor, which provides sliders for each RGB value interactively. I could then upload this colour into the shaders when I needed to render each planet.

Finally, if I was developing the project further I would add collision detection to my program. This could be a function that loops through every planet, checking whether or not every other planet is within its radius. If so there would be a collision. At this point I could then combine the planets into one big one with the mass of both added together, or delete both planets from the simulation.

Maintenance Issues

In order to make my code maintainable, I've separated it into logical components. This is largely done with my classes, which store the data I need for creating objects to add to the simulation. My shaders, classes, structs, variables, functions and main loop are all split up logically to ensure that the code is easy to understand.

In addition to this, I've used encapsulation effectively with getters and setters for each attribute. Although I don't use all these getters and setters within the program, they are important to have if I want to carry out any more development and add features which might require the use of these.

I have also used composition within my planet class so that each planet has a label and a trail path associated with it. This promotes maintainability because it keeps classes focused and makes them easier to test and debug in future development.

A maintainability issue I have is that I have a lot of global variables. This reduces maintainability of my code for several reasons. First, it is harder for a reader to understand where data comes from, making it difficult to make changes in the future. In addition to this, if data can be changed anywhere in the program, it can make it difficult to keep track of where variables are changing in the program. Furthermore, the use of global variables makes it difficult to test a single section of my code as it would require me to run the entire program in order to have access to the global variables.

Another maintainability issue I have is that if in further development of the program, I add more features, this could get very complicated and messy. This would make it very difficult to test and debug the system, as well as follow and understand the code. Within the main function, control logic is spread between functions, ruining the flow and meaning that several parts of the program would have to be edited if changes want to be made.

How could I address this with further development?

Many of my global variables should not be changed throughout the program. I can therefore make many of these constants using the `const` command. This makes the use of them a lot more safe because they cannot be changed in the program accidentally. The values that can change are the camera position values. In order to make these more maintainable, I could package them into a struct, so that they are all grouped together. It is then easier to change and develop this further later, increasing the maintainability.

Another thing I could do to increase the maintainability of the global variables is to introduce a simulation manager class. Instead of having global variables defined at the beginning of my program, they could be encapsulated within the attributes of my simulation manager class.

Introducing a simulation manager would also make the main loop of my code more maintainable. Adding features would be a lot easier to implement as I could add them into the simulation manager class as a method then simply add this to the main loop. This would improve the readability and flow of my code as it becomes more clear what each line does.